ESP32开发实战

快速入门

物联网嵌入式开发全栈指南

目 录

内容简介 1

前言 2

如何使用本书 4

第一部分 入门篇 6

第1章 认识ESP32 7

第2章 开发环境搭建 10

第3章 点亮第一盏灯：GPIO输出 14

第4章 电子基础与开发板资源 18

第二部分 基础篇 21

第5章 GPIO 数字输入输出 22

第6章 PWM 与呼吸灯 26

第7章 ADC 模拟信号采集 29

第8章 定时器与中断 33

第9章 UART 串口通信 38

第10章 I2C 总线通信 41

第11章 SPI 总线通信 45

第三部分 进阶篇 49

第12章 常用传感器实战 50

第13章 Wi-Fi 网络入门 54

第14章 HTTP 通信实战 58

第15章 MQTT 物联网协议 62

第16章 蓝牙 BLE 通信 66

第17章 FreeRTOS 多任务 70

第18章 文件系统与数据存储 75

第19章 低功耗设计 79

第四部分 精通篇 82

第20章 OTA 远程升级 83

第21章 ESP-IDF 深入解析 87
第22章 内存管理与优化 90
第23章 底层驱动开发 93
第24章 实时性与性能优化 97
第25章 安全与加密 100
第五部分 实战篇 103
第26章 实战项目一：智能家居控制中心 104
第27章 实战项目二：物联网环境监测站 108
第28章 实战项目三：智能门锁与安防系统 112
第29章 实战项目四：智能小车 116
第30章 实战项目五：云端物联网平台 119
附录 124
附录A 常用函数速查 125
附录B 引脚对照表 129
附录C 常见问题与排错 131
附录D 术语表 132
附录E 参考资料与延伸阅读 133
附录F 实验器材与工具清单 135
附录G 30 天学习路线图 137
附录H 开发板选型与购买指南 139
附录I 全书知识点自测题 141
结语：从入门到精通的旅程 144

内容简介

本书是一本系统讲解 ESP32 物联网开发的实战教程。全书按照"入门—基础—进阶—精通—实战"的完整路线组织：从开发环境搭建与点亮第一盏 LED 讲起，系统覆盖 GPIO、PWM、ADC、定时器与中断、UART/I2C/SPI 三大总线等核心外设；进而深入 Wi-Fi、蓝牙 BLE、MQTT、FreeRTOS 多任务、文件系统与低功耗设计等物联网关键技术；再到底层，剖析 ESP-IDF 框架、内存管理、驱动开发、性能优化与安全加密；最后通过智能家居、环境监测站、智能门锁、智能小车与云端物联网平台五个完整项目，把所学知识融会贯通。

全书共三十章正文与九个附录，配有大量可直接运行的代码示例、电路连接说明与常见问题排错指南。书中内容遵循"原理讲解—代码示例—实验验证—常见问题"的统一结构，既适合零基础的嵌入式爱好者循序渐进地系统学习，也适合有一定基础的开发者按需查阅、快速进阶。

前言

在过去的十年里，物联网（Internet of Things，IoT）从一个略显遥远的概念，迅速成长为渗透到智能家居、工业自动化、智慧农业、可穿戴设备、环境监测等各个领域的现实技术。而在这一波浪潮的底层，有一款芯片几乎成为了"物联网开发板"的代名词——它就是乐鑫科技（Espressif）推出的 ESP32。

ESP32 之所以能够在全球开发者中如此流行，原因并不复杂：它以一颗芯片同时集成了高性能双核处理器、Wi-Fi 和蓝牙双模无线连接，以及丰富的外设接口，而价格却低到令人难以置信的程度。更关键的是，它的软件生态极为成熟——无论是面向初学者的 Arduino 框架，还是面向专业工程师的 ESP-IDF 官方开发框架，乃至轻量易用的 MicroPython，都让不同水平的开发者都能找到适合自己的切入点。

然而，正因为 ESP32 的生态过于庞大、资料过于庞杂，许多初学者在入门时反而感到迷茫：不知道该从哪里学起，不知道哪些概念是必须掌握的，不知道如何从"点亮一盏灯"一步步走到"搭建一个真正的物联网系统"。市面上的教程要么过于零散，要么只停留在 Arduino 层面的简单示例，很少有一本书能够系统性地覆盖从基础到进阶再到实战的完整路径。

这正是本书诞生的初衷。本书旨在为 ESP32 开发提供一条清晰、完整、循序渐进的进阶路线：从最基础的电子知识和开发环境搭建开始，逐步深入 GPIO、定时器、中断、通信总线等核心外设；再进入 Wi-Fi、蓝牙、MQTT、FreeRTOS 等进阶主题；最终通过内存管理、驱动开发、性能优化等深入内容，帮助读者建立对 ESP32 的底层理解，并在此基础上完成多个贴近真实场景的完整项目。

本书的特色可以概括为四点：

第一，**系统完整**。本书按照"入门篇—基础篇—进阶篇—精通篇—实战篇"五大部分组织，共三十章，覆盖了 ESP32 开发中几乎所有重要的知识点，形成了一条完整的知识链条。

第二，**循序渐进**。每一章都遵循"原理讲解—代码示例—实验验证—常见问题"的结构，让读者在理解原理的基础上动手实践，在实践中加深理解。新手可以按顺序逐章学习，有基础的读者也可以根据需要跳读。

第三，**双框架并重**。本书以 Arduino 框架作为入门主线，帮助读者快速建立信心和手感；同时在精通篇深入介绍官方 ESP-IDF 框架，帮助读者理解 ESP32 的底层机制，具备向专业开发过渡的能力。

第四，**贴近实战**。实战篇的五个项目并非玩具示例，而是可以直接改造落地的真实应用，涵盖了智能家居、环境监测、安防、机器人小车和云端物联网平台等热门方向。

关于阅读建议： 如果您是完全没有嵌入式基础的初学者，建议从第一部分开始按顺序阅读，并务必动手完成每一章的实验。如果您已经接触过 Arduino 或其他单片机，可以从第二部分开始，快速浏览基础内容后重点学习进阶篇和精通篇。如果您已经是一名有经验的嵌入式工程师，可以直接跳到第四部分和第五部分，了解 ESP32 的底层机制与实战技巧。

关于"黑白色"： 本书采用黑白灰的印刷配色设计，既适合打印成纸质书阅读，也适合在墨水屏等设备上查看。书中的电路连接图、时序图和示意图均以黑白灰的线条与灰度区分，不依赖彩色信息，请读者放心阅读。

需要说明的是，嵌入式技术迭代迅速，本书所涉及的开发板型号、库版本和接口细节以写作时的主流版本为准（Arduino ESP32 核心 2.x/3.x、ESP-IDF 5.x），读者在实际操作时如遇到版本差异，请以官方最新文档为准。

最后，嵌入式开发是一门"动手"的学问。纸面上的知识只有通过亲手敲代码、连电路、调程序，才能真正内化为自己的能力。愿这本书能成为您 ESP32 开发之路上的一盏明灯，陪您从第一盏 LED 点亮，走向属于自己的物联网产品。

现在，就让我们一起开始这段旅程吧。

如何使用本书

本书共分五大部分、三十章正文与四个附录，建议读者根据自身情况选择阅读路径。

第一部分 入门篇（第 1～4 章）：介绍 ESP32 是什么、有哪些型号、如何搭建开发环境、如何点亮第一盏 LED，并补充必要的电子基础知识。这是所有读者的起点。

第二部分 基础篇（第 5～11 章）：系统讲解 ESP32 最核心的外设能力——GPIO 输入输出、PWM、ADC、定时器与中断，以及 UART、I2C、SPI 三大通信总线。掌握本部分后，读者就具备了独立驱动绝大多数传感器和执行器的能力。

第三部分 进阶篇（第 12～19 章）：进入物联网的核心——传感器实战、Wi-Fi 组网、HTTP 通信、MQTT 协议、蓝牙 BLE、FreeRTOS 多任务、文件系统与低功耗设计。这是从"单片机玩家"迈向"物联网工程师"的关键一步。

第四部分 精通篇（第 20～25 章）：深入 ESP32 的底层世界——OTA 升级、ESP-IDF 框架解析、内存管理、底层驱动开发、实时性与性能优化、安全与加密。掌握本部分内容，读者将对 ESP32 的运行机制了然于胸。

第五部分 实战篇（第 26～30 章）：通过智能家居控制中心、环境监测站、智能门锁、智能小车、云端物联网平台五个完整项目，把前面所学融会贯通，产出可落地的作品。

附录：提供常用函数速查、引脚对照表、常见问题排错指南与术语表，方便读者随时查阅。

每一章都遵循统一的结构：**学习目标** 明确本章要掌握什么；**正文** 以原理讲解和代码示例交替推进；**本章小结** 回顾要点；**思考与练习** 提供课后巩固题目。代码示例均经过精简，保留了核心逻辑并附有详细注释，读者可在此基础上自由扩展。

硬件准备： 建议准备一块 ESP32 开发板（如 ESP32-DevKitC、ESP32-WROOM-32 或常见的 38 引脚开发板）、一块面包板、若干杜邦线、若干 LED、电阻（220 Ω、1 kΩ、10 kΩ 等）、按键、电位器，以及少量传感器模块（如 DHT11/DHT22 温湿度、BMP280 气压、MPU6050 六轴等）。这些器材价格低廉、易于获得，足以支撑全书绝大多数实验。

软件准备： 一台安装了 Windows、macOS 或 Linux 的电脑，并按照第 2 章的指引安装好 Arduino IDE 或 PlatformIO 即可。后续章节涉及的第三方库都会在相应位置说明安装方法。

准备好了吗？让我们翻开第一章，先来认识这位即将陪伴我们很久的"主角"——ESP32。

第一部分 入门篇

第1章 认识ESP32

1.1 什么是ESP32

ESP32 是乐鑫科技（Espressif Systems）推出的一系列低成本、低功耗的微控制器片上系统（SoC，System on Chip）。所谓"片上系统"，指的是它将中央处理器、内存、存储器、无线通信模块以及各种外设接口都集成在了一颗芯片内部。换句话说，一颗小小的 ESP32 芯片，就相当于一台"微型计算机"——它拥有自己的处理器、内存和操作系统，同时又比传统计算机小得多、便宜得多、省电得多。

ESP32 最引人注目的特点，是它把 **Wi-Fi 和蓝牙** 这两种无线通信能力直接集成进了芯片。这意味着，我们不需要外接任何无线模块，就能让一块几厘米见方的开发板连接互联网、与其他设备通信。这一特性使得 ESP32 天然适合物联网应用场景，也让它迅速成为全球最受欢迎的物联网开发芯片之一。

从性能上看，经典款 ESP32 搭载了 **Xtensa 双核 LX6 处理器**，主频最高可达 240 MHz，内置 520 KB 的静态随机存取存储器（SRAM）和 448 KB 的只读存储器（ROM），并支持高达 16 MB 的外部闪存（Flash）用于存储程序和数据。这样的配置对于一款几美元价位的芯片而言堪称豪华，足以运行实时操作系统和多任务并发程序。

除了强大的计算和无线能力，ESP32 还拥有极其丰富的外设资源，包括多达 34 个可编程的通用输入输出引脚（GPIO）、模数转换器（ADC）、数模转换器（DAC）、脉宽调制输出（PWM）、触摸传感器、多种通信接口（UART、I2C、SPI、I2S、CAN、SDIO 等），以及硬件加密引擎等。可以说，几乎任何你能想到的传感器、显示屏、电机、继电器，都能方便地接到 ESP32 上。

1.2 ESP32 的芯片家族

很多初学者以为"ESP32 只有一种"，其实乐鑫在 ESP32 之后又推出了一系列衍生产品，形成了一个庞大的芯片家族。了解它们之间的区别，有助于我们在项目中做出合适的选择。下表列出了几款常见的 ESP32 系列芯片及其主要特征。

芯片型号	处理器核心	主频	Wi-Fi	蓝牙	主要特点
ESP32	Xtensa 双核 LX6	240 MHz	802.11 b/g/n	4.2 BR/EDR + BLE	经典款，功能最全面
ESP32-S2	Xtensa 单核 LX7	240 MHz	802.11 b/g/n	无	主打 USB、低功耗
			802.11 b/g/n	BLE 5.0	

芯片型号	处理器核心	主频	Wi-Fi	蓝牙	主要特点
ESP32-S3	Xtensa 双核 LX7	240 MHz			增加 AI 向量指令，适合语音/图像
ESP32-C3	RISC-V 单核	160 MHz	802.11 b/g/n	BLE 5.0	采用开源 RISC-V 架构，性价比高
ESP32-C6	RISC-V 单核	160 MHz	802.11 ax (Wi-Fi 6)	BLE 5.3	支持 Wi-Fi 6 和 Thread/Zigbee

需要说明的是，本书主要以**经典款 ESP32**（尤其是常见的 ESP32-WROOM-32 模组和 38 引脚开发板）为讲解对象。这是因为经典款 ESP32 的教程、库和资料最为丰富，最适合入门。掌握了经典款之后，再迁移到 S2、S3、C3 等新芯片会非常轻松，因为它们共享同一套开发框架（ESP-IDF）和大多数编程接口。

1.3 ESP32 开发板解析

芯片是核心，但芯片本身尺寸极小、引脚密集，难以直接使用。因此，厂家通常会把芯片封装到一块"开发板"上，方便我们插线、供电和调试。最常见的经典 ESP32 开发板采用以下结构：

- **ESP32 芯片**：核心，位于开发板中央的金属屏蔽罩下方；
- **外部闪存（Flash）**：存储程序代码和文件系统，常见容量为 4 MB、8 MB 或 16 MB；
- **USB 转串口芯片**（如 CP2102 或 CH340）：负责将 USB 接口转换为串口信号，用于烧录程序和打印调试信息；
- **稳压电路**：将 USB 提供的 5 V 电压转换为芯片所需的 3.3 V；
- **两个按键**：一个用于复位（EN/Reset），一个用于进入下载模式（BOOT）；
- **两排排针**：引出芯片的 GPIO 引脚，方便用杜邦线连接外部器件；
- **电源指示灯与用户 LED**：电源指示灯上电即亮；部分开发板板载一颗可编程的用户 LED，通常接在 GPIO2 上。

典型的 38 引脚开发板按 0.9 英寸间距排列两排引脚，可以直接插在面包板上，非常适合原型开发。在后续章节中，我们会频繁用到这些引脚，具体的引脚定义与注意事项将在第 4 章详细讲解。

1.4 三大开发框架对比

给 ESP32 写程序，有三条主流的技术路线。它们各有优劣，本书会根据不同阶段的需要灵活选用。

Arduino 框架：对初学者最友好。它提供了大量封装好的库函数（如 `digitalWrite()`、`analogRead()`、`delay()`），语法简洁，一个程序就是一个 `setup()` 加一个 `loop()`，屏蔽了底层细节。Arduino 社区为 ESP32 提供了海量的第三方库，绝大多数传感器模块都能"即插即用"。它的缺点是性能和控制粒度不如官方框架精细，适合入门和快速原型开发。本书第一部分至第三部分主要以 Arduino 框架为主线。

ESP-IDF 框架：乐鑫官方推出的专业级开发框架，基于 FreeRTOS 实时操作系统，提供了完整而底层的 API，性能和灵活性最高，是工业级产品的首选。它的学习曲线相对陡峭，需要理解任务、队列、事件等概念，且使用命令行工具进行编译和烧录。本书第四部分将深入介绍 ESP-IDF。

MicroPython 框架：把 Python 解释器移植到了 ESP32 上，让开发者可以用 Python 语言编写嵌入式程序。它对熟悉 Python 的开发者非常友好，交互式开发效率高，但由于解释执行的特性，性能和实时性不如前两者，适合快速验证和教学场景。

对于初学者，我们的建议是：**先用 Arduino 框架入门，建立对硬件和编程的基本认识，再逐步过渡到 ESP-IDF，理解底层原理**。这也是本书贯穿始终的路线。

1.5 学习本章后你能做什么

学完本章，你已经对 ESP32 有了整体认识：知道它是一款集成 Wi-Fi 和蓝牙的物联网芯片，了解它背后的芯片家族，看清了开发板的结构，也明白了三条开发路线的区别。接下来，我们将着手搭建开发环境，让电脑能够"听懂"并"指挥"这块小小的开发板。

本章小结

- ESP32 是乐鑫推出的集成 Wi-Fi/蓝牙的物联网 SoC，经典款采用双核 240 MHz 处理器；
- ESP32 已发展出 S2、S3、C3、C6 等多个衍生型号，各有侧重；
- 开发板由芯片、Flash、USB 转串口、稳压电路、按键和排针等组成；
- Arduino、ESP-IDF、MicroPython 是三大开发框架，本书以 Arduino 入门、ESP-IDF 深入。

思考与练习

1. 请用自己的话说说，"片上系统（SoC）"和普通的单片机有什么区别？
2. ESP32-S3、ESP32-C3 与经典款 ESP32 相比，处理器架构有何不同？
3. 上网查一查你手头（或计划购买）的 ESP32 开发板，找出它使用的主芯片型号、Flash 容量和 USB 转串口芯片型号。

第2章 开发环境搭建

2.1 准备工作

在开始编写程序之前，我们需要让电脑具备编写、编译和烧录 ESP32 程序的能力，这一整套工具被统称为"开发环境"。本章将以最常用的 Arduino IDE 为例，逐步完成开发环境的搭建。Arduino IDE 是一款免费、开源、跨平台的集成开发环境，支持 Windows、macOS 和 Linux。

在开始之前，请先准备以下物品：

- 一台电脑（本书以 Windows 11 为例，macOS 和 Linux 的操作思路相同）；
- 一块 ESP32 开发板；
- 一根可传输数据的 USB 数据线（注意有些线只能充电不能传数据，务必使用带数据功能的线）；
- 可用的网络连接（用于下载软件和开发板支持包）。

2.2 安装 Arduino IDE

第一步，打开浏览器访问 Arduino 官网（www.arduino.cc），进入 Software 下载页面，根据您的操作系统下载对应的安装包。对于 Windows 用户，推荐下载 Windows 安装程序（.exe）版本；macOS 用户下载 .dmg 版本；Linux 用户可以使用系统软件源或官网提供的压缩包。

第二步，运行下载好的安装程序，按照安装向导的提示完成安装。安装过程中，如果系统弹出"是否允许此设备进行更改"的提示，请选择"是"；如果弹出驱动安装提示（如询问是否安装 USB 转串口驱动），请一并同意安装。

第三步，安装完成后启动 Arduino IDE。首次启动界面会显示一个默认的示例程序框架，包含 `void setup()` 和 `void loop()` 两个函数。这表示 IDE 已经可以正常工作了。

2.3 安装 ESP32 开发板支持包

Arduino IDE 默认只支持 Arduino 官方系列开发板，要让 IDE 识别 ESP32，我们需要额外安装 ESP32 的开发板支持包。这是 ESP32 入门过程中**最容易出错的一步**，请仔细跟随下面的操作。

第一步，打开 Arduino IDE，进入菜单栏的"文件 → 首选项"（macOS 为"Arduino IDE → Settings"）。在"其他开发板管理器地址"（Additional boards manager URLs）一栏中，填入以下链接：

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

如果这一栏已有其他地址，可以用英文逗号将多个地址隔开，然后点击"确定"保存。

第二步，进入菜单栏的"工具 → 开发板 → 开发板管理器"（Boards Manager）。在搜索框中输入 "esp32"，稍等片刻，即可看到 "esp32 by Espressif Systems" 这一项。点击"安装"按钮，等待下载和安装完成。由于支持包体积较大（数百 MB），这一步可能需要几分钟甚至更长时间，请耐心等待，并确保网络畅通。

> 【注意】 注意：如果您所在网络环境访问 GitHub 较慢，安装可能会超时失败。此时可以尝试更换网络、多次重试，或使用国内镜像源。安装失败时，请确认第一步中的 URL 是否填写正确，且没有多余的空格。

第三步，安装完成后，重新启动 Arduino IDE。此时在"工具 → 开发板 → ESP32 Arduino"子菜单下，应该能看到一长串 ESP32 开发板型号。

2.4 安装 USB 转串口驱动

要让电脑识别插上的开发板，还需要安装对应的 USB 转串口芯片驱动。大多数开发板使用 CP210x（硅实验室）或 CH340（南京沁恒）两种芯片之一，少数新开发板使用原生 USB（如 ESP32-S3 部分型号）。请根据您开发板上的芯片型号，到对应官网下载并安装驱动：

- **CP210x 系列**：到 Silicon Labs 官网搜索 "CP210x USB to UART Bridge VCP Drivers"，下载对应系统的驱动并安装；
- **CH340 系列**：到沁恒（WCH）官网搜索 "CH340" 驱动下载安装。

安装驱动后，用 USB 数据线将开发板连接到电脑。然后按下键盘上的 Win+X（macOS 用户直接跳过），打开"设备管理器"，展开"端口（COM 和 LPT）"一项。如果看到类似 "Silicon Labs CP210x USB to UART Bridge（COM3）" 或 "USB-SERIAL CH340（COM5）" 的条目，说明驱动安装成功，请记下括号中的 **COM 端口号**（如 COM3），稍后选择开发板端口时会用到。

> 【注意】 注意：如果设备管理器中该设备带有黄色感叹号，说明驱动未正确安装，请重新安装对应驱动；如果插上后完全没有反应，请先更换一根确定可以传输数据的 USB 线。

2.5 选择开发板与端口

现在，我们把 Arduino IDE 配置到具体的目标开发板上。

第一步，在"工具 → 开发板 → ESP32 Arduino"菜单中，选择与您开发板匹配的型号。对于最常见的经典款 38 引脚开发板，选择 "ESP32 Dev Module" 即可；如果您使用的是带 WROVER 模组的开发板，请选择对应的 WROVER 型号。

第二步，在"工具 → 端口"菜单中，勾选您刚才在设备管理器中看到的 COM 端口（如 COM3）。

第三步，保持"工具"菜单下的其他选项为默认值即可，通常无需修改。

至此，开发环境的基本配置就完成了。下面，我们用一个最简单的小程序来验证整条"编写—编译—烧录—运行"的链路是否打通。

2.6 第一个测试程序：串口问候

在 Arduino IDE 中新建一个文件，清空默认内容，输入以下代码：

```
// 第一个测试程序：通过串口打印问候语
void setup() {
  Serial.begin(115200);       // 初始化串口，波特率 115200
  delay(500);                 // 稍作等待，让串口稳定
  Serial.println("你好，ESP32! ");  // 打印一句问候语
}

void loop() {
  // loop 中无需做任何事，保持为空
}
```

这段程序的作用是：上电后初始化串口，并通过串口打印一句中文问候语。`Serial.begin()` 用于启动串口并设定通信速率，`Serial.println()` 用于打印一行文本。

接下来进行编译和烧录：

第一步，点击左上角的"对勾"图标（或按快捷键 Ctrl+R）进行编译。编译是把我们写的代码翻译成芯片能执行的机器指令的过程。如果代码无误，下方状态栏会显示"编译完成"并报告程序占用空间的大小。

第二步，点击"向右箭头"图标（或按快捷键 Ctrl+U）进行上传（烧录）。上传前，请先按住开发板上的 **BOOT 键**，然后点击上传，待编译完成、状态栏出现 "Connecting..." 字样后松开 BOOT 键。部分开发板无需手动按 BOOT 键也能自动进入下载模式，可以两种方式都尝试一下。

第三步，上传成功后，点击 IDE 右上角的"串口监视器"图标，将右下角的波特率设置为 **115200**。此时串口监视器窗口中应显示：

```
你好，ESP32!
```

如果看到这行文字，恭喜你——开发环境已经完整打通！如果屏幕上显示乱码，通常是波特率设置不一致导致的，请确认监视器右下角的波特率为 115200；如果串口没有任何输出，请检查端口选择是否正确、开发板是否已复位。

2.7 常见问题与排错

问题一：上传时一直提示 "Connecting..." 失败。 这是最经典的"下载模式"问题。请按住 BOOT 键再点击上传，看到 "Connecting..." 后松开；若仍失败，尝试在按住 BOOT 键的同时按一下复位键（EN）再松开。此外，请确认选择了正确的 COM 端口，且没有其他程序占用该端口。

问题二：开发板管理器搜不到 esp32。 请回到"首选项"，确认开发板管理器地址已正确填写并保存，然后重启 Arduino IDE 再试。

问题三：串口输出乱码。 检查两处波特率：代码中 `Serial.begin()` 的参数，以及串口监视器右下角的波特率设置，二者必须一致。

问题四：找不到端口。 通常是驱动未安装或数据线损坏。请回到 2.4 节检查驱动，并更换一根可传输数据的 USB 线。

本章小结

- 开发环境搭建包括安装 Arduino IDE、安装 ESP32 支持包、安装 USB 转串口驱动三步；
- 通过"选择开发板型号 + 选择端口"完成 IDE 到开发板的配置；
- 第一个串口测试程序验证了"编写—编译—烧录—运行"全链路；
- 下载模式（BOOT 键）和波特率是初学者最常见的两个坑。

思考与练习

1. 尝试修改测试程序中的问候语和波特率，重新编译上传，观察效果。
2. 在设备管理器中，观察拔插开发板前后端口列表的变化，确认您的端口号。
3. 如果条件允许，尝试在另一台电脑（不同操作系统）上完成环境搭建，比较流程差异。

第3章 点亮第一盏灯：GPIO输出

3.1 学习目标

"点亮一盏灯"几乎是所有嵌入式开发的第一课，它就像编程世界里的 "Hello World"。通过本章，你将掌握 ESP32 最基础也最重要的能力——**数字输出**：控制一个引脚的电压在高低两种状态之间切换，从而驱动 LED 亮灭。学完本章，你将能够：

- 理解数字信号与高低电平的概念；
- 学会使用 `pinMode()` 和 `digitalWrite()` 控制 GPIO 引脚；
- 完成点亮 LED、LED 闪烁两个经典实验；
- 初步了解 ESP32 GPIO 的电气特性与注意事项。

3.2 数字信号与高低电平

在数字电路中，一切信号最终都被抽象为两种状态：**高电平（High）** 和 **低电平（Low）**。对于 ESP32 这类 3.3 V 逻辑的单片机而言，高电平约等于 3.3 V，低电平约等于 0 V。我们用数字 1 表示高电平，用数字 0 表示低电平，这就是"数字信号"的含义。

当我们在程序中把一个引脚设置为高电平时，该引脚对外输出 3.3 V 电压；设置为低电平时，引脚输出 0 V。如果把一颗 LED 的正极通过限流电阻接到引脚、负极接到地（GND），那么当引脚输出高电平时，电流从引脚经 LED 流向地，LED 就会点亮；当引脚输出低电平时，两端没有电压差，LED 熄灭。这正是控制 LED 的原理。

> 【提示】 小知识：除了高电平点亮（称为"高电平有效"），也可以反过来接——LED 正极接 3.3 V 电源，负极经限流电阻接引脚，此时引脚输出低电平 LED 才亮，称为"低电平有效"。这两种接法在工程中都常用。

3.3 GPIO 与引脚编号

GPIO（General Purpose Input/Output）即"通用输入输出"，是单片机最核心的外设。ESP32 芯片拥有多达 34 个 GPIO 引脚，但受封装限制，常见开发板通常只引出其中一部分。每个 GPIO 引脚都有一个编号（如 GPIO0、GPIO2、GPIO5……），在程序中我们通过这个编号来操作对应的物理引脚。

需要特别注意的是，ESP32 的部分 GPIO 引脚有特殊用途，**并不是所有引脚都能随意使用**：

- **GPIO0**：兼作 BOOT 键，上电时电平决定芯片进入正常模式还是下载模式，不建议作为普通输出使用；
- **GPIO1 和 GPIO3**：分别是串口 TX0 和 RX0，用于 USB 串口通信和调试打印，通常不占用；
- **GPIO6～GPIO11**：连接片外 Flash，一般开发板不引出，不可使用；
- **GPIO34～GPIO39**：仅能作为**输入**使用，不能输出（没有内部上拉/下拉电阻）。

因此，在经典 38 引脚开发板上，最安全、最常用的输出引脚是 **GPIO2、GPIO4、GPIO5、GPIO12～GPIO19、GPIO21～GPIO23、GPIO25～GPIO27、GPIO32、GPIO33** 等。本书的实验默认使用 **GPIO2**（多数开发板的板载 LED 就接在 GPIO2 上），读者也可根据实际情况改用其他可用引脚。

3.4 实验一：点亮板载 LED

大多数 ESP32 开发板上都集成了一颗蓝色的用户 LED，它通常连接在 GPIO2 上（也有少数型号接在 GPIO13 或其他引脚，请以开发板电路图为准）。我们先利用这颗板载 LED 完成第一个实验，无需外接任何元器件。

新建程序，输入以下代码：

```
// 实验一：点亮板载 LED
#define LED_PIN 2   // 定义 LED 引脚为 GPIO2

void setup() {
  pinMode(LED_PIN, OUTPUT);  // 将 LED 引脚设置为输出模式
  digitalWrite(LED_PIN, HIGH); // 输出高电平，点亮 LED
}

void loop() {
  // 空循环，保持 LED 常亮
}
```

这段程序中出现了两个关键函数：

- `pinMode(pin, mode)`：设置某个引脚的工作模式。第一个参数是引脚号，第二个参数是模式，`OUTPUT` 表示输出模式，`INPUT` 表示输入模式。在使用一个引脚之前，必须先通过 `pinMode()` 声明它的用途。

- `digitalWrite(pin, value)`：设置某个输出引脚的电平。`HIGH` 表示高电平（3.3 V），`LOW` 表示低电平（0 V）。

`setup()` 函数在芯片上电或复位后只执行一次，用来完成初始化配置；`loop()` 函数则会反复不断地执行，是我们的主程序循环。

编译并上传这段程序，上传完成后，开发板上的蓝色 LED 会立即点亮并保持常亮。如果 LED 没有点亮，请检查引脚号是否正确（部分开发板板载 LED 在 GPIO13 或 GPIO2 反接），并确认代码已成功上传。

3.5 实验二：LED 闪烁

点亮常亮还不够有趣，接下来我们让 LED"呼吸"起来——有节奏地亮灭闪烁。这是嵌入式世界最经典的入门程序，几乎等同于编程界的 "Hello World"。

```
// 实验二：LED 闪烁（经典 Blink）
#define LED_PIN 2

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);  // 点亮
  delay(1000);                  // 延时 1000 毫秒（1 秒）
  digitalWrite(LED_PIN, LOW);   // 熄灭
  delay(1000);                  // 延时 1 秒
}
```

这里新引入的 `delay(ms)` 函数会让程序"暂停"指定的毫秒数。在上面的循环中，程序点亮 LED、等待 1 秒、熄灭 LED、再等待 1 秒，如此往复，LED 就以 1 秒为周期闪烁。

上传后观察效果：LED 每秒切换一次亮灭。你可以尝试把 `delay()` 的参数改小（如 100），LED 闪烁会明显加快；改大（如 5000），则会变慢。通过调整延时，就能直观地感受时间参数对程序行为的影响。

> 【注意】 注意：`delay()` 是一种"阻塞式"延时——在延时期间，芯片无法做任何其他事情。对于简单的 LED 闪烁这完全够用，但在复杂程序中滥用 `delay()` 会让程序变得"卡顿"。我们将在第 8 章学习用定时器和中断实现非阻塞的延时。

3.6 实验三：外接 LED 电路

板载 LED 只能用来验证程序，真正的项目往往需要外接 LED。下面我们来搭建第一个真实的硬件电路。

所需材料：LED 一颗、220 Ω（或 330 Ω）电阻一颗、杜邦线两根、面包板一块。

电路连接（以 GPIO5 为例）：

- 将 LED 的**长脚（正极，阳极）**通过 220 Ω 电阻连接到开发板的 GPIO5；
- 将 LED 的**短脚（负极，阴极）**连接到开发板的 GND（地）。

> 【提示】 为什么必须串联限流电阻？LED 的正向压降通常为 2～3 V，如果直接把 3.3 V 加在 LED 两端，电流会过大，轻则缩短 LED 寿命，重则当场烧毁，甚至可能损坏开发板引脚。串联一个 220 Ω 左右的电阻，可以把电流限制在约 (3.3 V − 2 V) ÷ 220 Ω ≈ 6 mA，既保证 LED 正常发光，又保护了器件。

代码：把实验二中的 `LED_PIN` 改为 5 即可，其余保持不变。上传后，外接的 LED 应当与程序逻辑同步闪烁。

接线检查要点：LED 有极性，长脚必须接电源侧（经电阻接引脚），短脚接地，接反则不亮；电阻应串联在 LED 与引脚之间，位置前后均可；确保杜邦线接触良好，面包板插孔对齐。

3.7 本章小结

- 数字信号只有高电平（3.3 V）和低电平（0 V）两种状态；
- `pinMode()` 设置引脚模式，`digitalWrite()` 输出高低电平，`delay()` 实现延时；
- ESP32 的 GPIO 并非全部可用，GPIO0、GPIO1、GPIO3、GPIO6～11、GPIO34～39 等有特殊限制；
- 外接 LED 时必须串联限流电阻，注意 LED 的极性。

思考与练习

1. 修改闪烁节奏，让 LED 亮 200 毫秒、灭 800 毫秒，观察效果。
2. 同时外接两颗 LED（如 GPIO4 和 GPIO5），编写程序让它们交替闪烁（流水灯效果）。
3. 查阅资料，说明"高电平有效"和"低电平有效"接法的区别，并各画一张电路连接示意图。

第4章 电子基础与开发板资源

4.1 为什么需要电子基础

ESP32 是一个"软硬结合"的平台：软件层面，我们写代码控制芯片；硬件层面，芯片通过引脚与外部电路连接。如果对电子知识一无所知，那么在接线、选器件、排查故障时就会处处碰壁。本章将用尽量通俗的语言，补充嵌入式开发必备的基础电子知识，并全面盘点 ESP32 开发板的引脚资源，为后续章节打下坚实基础。

4.2 电压、电流与电阻

电压（Voltage）：可以理解为"电势差"，是推动电荷流动的"压力"，单位是伏特（V）。ESP32 工作电压为 3.3 V，而 USB 口提供的是 5 V，因此开发板上需要用稳压电路把 5 V 降到 3.3 V。**需要特别牢记：ESP32 的 GPIO 引脚最高只能承受 3.3 V，绝对不能直接接入 5 V 信号，否则会烧毁芯片**。

电流（Current）：是电荷的定向流动，单位是安培（A）。电流有大小，也有方向。ESP32 单个 GPIO 引脚最大输出电流约为 40 mA（推荐不超过 20 mA 长期工作），因此**不能直接用引脚驱动大功率负载**（如电机、大功率 LED），必须通过三极管、场效应管或继电器等器件来"放大"驱动能力。

电阻（Resistance）：是导体对电流的阻碍作用，单位是欧姆（Ω）。电阻在电路中最重要的作用之一就是**限流**——保护 LED、引脚等器件不被过大的电流损坏，这正是上一章 LED 电路串联电阻的原因。

这三者之间的关系由著名的**欧姆定律**描述：**电压 = 电流 × 电阻**，即 $U = I \times R$。这是整个电子学最基础、最常用的公式，务必牢记。

4.3 面包板与杜邦线

面包板（Breadboard） 是原型开发中最常用的"免焊接"电路搭建工具。它的表面布满小孔，内部按一定规律用金属条连通。典型面包板的结构如下：

- 上下两侧各有一排横向连通的**电源轨**（通常用红色和蓝色线标出，红色接正极、蓝色接负极）；
- 中间区域分为左右两大块，每一列（纵向）的 5 个孔相互连通，用于插接元器件；
- 中间有一条凹槽，用于跨接集成电路（IC）等双列引脚器件。

使用面包板时，只需把元器件引脚插入对应插孔即可形成电气连接，修改和拆除都非常方便，特别适合快速验证电路。

杜邦线（Jumper Wire） 是连接开发板与面包板的导线，有公对公、母对母、公对母三种类型。连接 ESP32 开发板排针与面包板时，通常使用**公对公**杜邦线；连接带排针的传感器模块时，常用**母对母**杜邦线。颜色并没有功能含义，但习惯上用红色接电源正极、黑色接地（GND），便于识别。

4.4 ESP32 引脚资源总览

现在，让我们全面盘点经典 38 引脚 ESP32 开发板两侧的引脚资源。下表按功能分类列出常用引脚，帮助你在项目中快速选型。

功能类别	可用引脚	说明
安全数字输出	GPIO2、4、5、12、13、14、15、16、17、18、19、21、22、23、25、26、27、32、33	推荐优先使用，无特殊限制
仅输入引脚	GPIO34、35、36、39	只能作输入，无内部上下拉
串口 UART0	TX=GPIO1，RX=GPIO3	默认调试串口，勿占用
串口 UART2	TX=GPIO17，RX=GPIO16	可作第二路串口
I2C 默认	SDA=GPIO21，SCL=GPIO22	软件可改任意引脚
SPI 默认	MOSI=23，MISO=19，SCK=18，CS=5	软件可改任意引脚
ADC1 通道	GPIO32～GPIO39	模拟输入
ADC2 通道	GPIO0、2、4、12～15、25～27	模拟输入，但 Wi-Fi 使用时受限
DAC 输出	GPIO25、GPIO26	真模拟电压输出
触摸传感器	GPIO0、2、4、12～15、27、32、33	电容式触摸输入
特殊限制	GPIO0（BOOT）、GPIO1/3（串口）、GPIO6～11（Flash）	尽量避免占用

> 【提示】 提示：上表只是"默认/推荐"配置。得益于 ESP32 的 **GPIO 矩阵**（GPIO Matrix），绝大多数外设功能都可以通过软件映射到任意可用引脚上，这为 PCB 布局提供了极大的灵活性。这也是 ESP32 相比传统单片机的一大优势。

4.5 供电设计要点

ESP32 的供电是新手常踩的坑，以下几点务必注意：

- **开发板供电**：通过 USB 口供电最方便，5 V 输入由板上稳压电路转为 3.3 V。也可以从开发板的 **VIN（5V）** 引脚接入外部 5 V 电源，或从 **3V3** 引脚接入稳定的 3.3 V 电源（不推荐同时使用两路电源）。
- **传感器供电**：开发板的 3V3 引脚输出能力有限（通常数百 mA）。如果外接的传感器、模组较多或功耗较大，建议单独供电，并把所有设备的 **GND 共地**——这是电路能正常通信的前提。
- **瞬时功耗**：ESP32 在 Wi-Fi 收发时会出现短时电流尖峰（可达 500 mA 以上）。如果供电能力不足，会出现**反复复位、Wi-Fi 连接失败**等诡异问题。因此，使用电池或线性稳压器供电时，务必保证足够的电流余量。
- **电平匹配**：与 5 V 逻辑的外部器件（如部分传感器、继电器）通信时，要注意 3.3 V 与 5 V 的电平差异。输出到 5 V 器件通常没问题，但 5 V 器件输出的高电平直接接入 ESP32 引脚会损坏芯片，此时应使用**电平转换模块**。

4.6 本章小结

- 电压、电流、电阻及欧姆定律是电路分析的基础；
- 面包板与杜邦线是快速搭建原型电路的工具；
- ESP32 引脚资源丰富但有限制，选型时应避开特殊引脚；
- 供电稳定是 ESP32 正常工作的前提，尤其要注意 Wi-Fi 场景的电流需求。

思考与练习

1. 用欧姆定律计算：3.3 V 电压下，要得到约 10 mA 电流，应串联多大的限流电阻？
2. 列出至少 5 个不推荐作为普通输出使用的 ESP32 引脚，并说明原因。
3. 动手用面包板搭建一个"三颗 LED 流水灯"电路，并用杜邦线正确连接到开发板。

第二部分 基础篇

第5章 GPIO 数字输入输出

5.1 学习目标

第 3 章我们已经用 GPIO 做了输出（点亮 LED），本章将补全 GPIO 的另一半能力——**数字输入**，并学会处理输入中最棘手的问题：按键抖动与上下拉电阻。学完本章，你将能够：

- 使用 `digitalRead()` 读取引脚的高低电平；
- 理解上拉电阻、下拉电阻的作用；
- 编写按键控制 LED 的程序；
- 掌握软件消抖的原理与实现。

5.2 数字输入的基本原理

数字输入与数字输出正好相反：输出是"芯片对外输出电压"，输入则是"芯片读取外部施加在引脚上的电压电平"。当引脚上的电压接近 3.3 V 时，`digitalRead()` 返回 `HIGH`（1）；当电压接近 0 V 时，返回 `LOW`（0）。

最简单的输入器件是**按键（按钮）**。一个按键本质上是一个开关：按下时两端导通，松开时两端断开。常用的四脚轻触按键，实际上内部是两对引脚，对角各为一组导通端。接线时只需使用其中一组即可。

按键有两种基本接法：

- **下拉接法**：按键一端接 3.3 V，另一端接 GPIO 引脚，同时引脚接一个下拉电阻到 GND。松开时引脚被下拉电阻拉低（读到 0），按下时引脚被 3.3 V 拉高（读到 1）。
- **上拉接法**：按键一端接 GND，另一端接 GPIO 引脚，同时引脚接一个上拉电阻到 3.3 V。松开时引脚被上拉电阻拉高（读到 1），按下时引脚被拉到地（读到 0）。上拉接法更为常用，因为 ESP32 的 GPIO 内部就集成了可编程的上拉电阻，可以省去外部电阻。

> 【提示】 为什么不能把按键直接悬空接引脚？因为当按键松开时，如果引脚既不接高也不接低，就会处于"悬空"状态，引脚电平会因电磁干扰而随机跳变，导致读取结果不可靠。上拉/下拉电阻的作用，就是给引脚一个确定的"默认电平"。

5.3 按键控制 LED（基础版）

我们先做一个最简单的按键控制 LED：按下按键 LED 亮，松开 LED 灭。使用内部上拉，接线如下：

- 按键一端接 GPIO23，另一端接 GND；
- LED 经 220 Ω 电阻接 GPIO5（沿用第 3 章的接法）。

```
// 按键控制 LED（使用内部上拉）
#define BUTTON_PIN 23
#define LED_PIN 5

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);  // 输入模式 + 内部上拉
}

void loop() {
  int state = digitalRead(BUTTON_PIN);  // 读取按键状态
  if (state == LOW) {       // 按下时（上拉接法，按下为低电平）
    digitalWrite(LED_PIN, HIGH);  // 点亮 LED
  } else {
    digitalWrite(LED_PIN, LOW);   // 熄灭 LED
  }
}
```

这里用到了 `INPUT_PULLUP` 模式，它在将引脚设为输入的同时，内部接上了上拉电阻。因此按键未按下时，引脚被拉高，读到 `HIGH`；按下时按键把引脚接地，读到 `LOW`。程序据此判断按键状态并控制 LED。

上传后测试：按住按键 LED 亮，松开 LED 灭。如果现象相反，说明接线或逻辑需要调整。

5.4 按键抖动与消抖

如果我们在上面的程序里加入"每按一次，LED 状态翻转一次"的逻辑，很快就会发现一个诡异的现象：**明明只按了一下，LED 却可能连续翻转好几次**。这就是"按键抖动"在作祟。

按键抖动：机械按键在按下和松开的瞬间，触点并不会立刻稳定接触或断开，而是会在导通与断开之间快速震荡若干毫秒，产生一串高频的误信号。单片机读取速度极快，会把这一串震荡当作"多次按下"，从而造成误判。

解决抖动问题的方法称为"消抖"，分硬件和软件两种。**软件消抖**最为常用，其核心思想是：检测到按键状态变化后，等待一段时间（如 20～50 毫秒）让抖动结束，再读取一次确认状态。下面是一个标准的软件消抖翻转示例：

```
// 按键翻转 LED（带软件消抖）
#define BUTTON_PIN 23
#define LED_PIN 5

bool ledState = false;      // LED 当前状态
bool lastButton = HIGH;     // 上一次读取的按键状态
unsigned long lastDebounce = 0;  // 上次消抖时间戳
const unsigned long debounceDelay = 50; // 消抖延时 50 毫秒

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  int reading = digitalRead(BUTTON_PIN);

  if (reading != lastButton) {       // 状态发生变化，可能是抖动
    lastDebounce = millis();         // 重置消抖计时
  }

  // 状态稳定超过消抖时间，才认为是有效按键
  if ((millis() - lastDebounce) > debounceDelay) {
    if (reading == LOW) {            // 确认按下
      ledState = !ledState;          // 翻转 LED 状态
      digitalWrite(LED_PIN, ledState);
      while (digitalRead(BUTTON_PIN) == LOW); // 等待松开，防止一次按下触发多次
    }
  }

  lastButton = reading;
}
```

这段程序引入了 `millis()` 函数，它返回程序启动以来经过的毫秒数。通过记录状态变化的时间戳，并判断稳定持续时间是否超过 50 毫秒，来过滤掉抖动噪声。上传后测试：现在每按一次按键，LED 只翻转一次，表现稳定。

5.5 本章小结

- `digitalRead()` 读取引脚电平，`INPUT_PULLUP` 模式使用内部上拉；
- 上拉/下拉电阻为引脚提供确定的默认电平，避免悬空；
- 机械按键存在抖动，需用软件消抖（延时确认）或硬件消抖（并联电容）处理；
- `millis()` 是后续实现非阻塞逻辑的重要工具。

思考与练习

1. 分别用"上拉接法"和"下拉接法"实现按键控制 LED，比较两种接法的代码差异。
2. 实现"单击点亮、再次单击熄灭"的开关灯，并观察不加消抖时的异常现象。
3. 尝试把消抖延时改成 5 毫秒和 200 毫秒，观察并解释效果差异。

第6章 PWM 与呼吸灯

6.1 学习目标

前面我们用数字输出只能让 LED"全亮"或"全灭"，无法控制亮度。本章引入一个强大的概念——**PWM（脉宽调制，Pulse Width Modulation）**，它让我们可以"模拟"出介于 0 和 3.3 V 之间的任意电压效果。学完本章，你将能够：

- 理解 PWM 的原理（占空比、频率）；
- 使用 ESP32 的 `ledc` API 输出 PWM；
- 完成"呼吸灯"实验，让 LED 平滑地由暗变亮、由亮变暗；
- 了解 PWM 在电机调速、舵机控制、调光等场景的广泛应用。

6.2 PWM 原理

PWM 的核心思想非常巧妙：既然引脚只能输出高电平和低电平两种状态，那就让引脚**以极高的频率在高低电平之间快速切换**。如果在一个周期内，高电平占的时间长，等效的平均电压就高，LED 就显得亮；低电平占的时间长，平均电压就低，LED 就显得暗。只要切换频率足够快（远超人眼分辨能力），我们看到的就不再是"闪烁"，而是稳定的亮度变化。

PWM 有两个关键参数：

- **周期（Period）**：一个完整的"高—低"循环所占的时间，其倒数就是**频率（Frequency）**。对于 LED 调光，几 kHz 的频率即可；对于电机，常用十几 kHz；对于音频，则需要几十 kHz。
- **占空比（Duty Cycle）**：一个周期内高电平所占的比例，通常用百分比表示。占空比 0% 表示一直低电平，100% 表示一直高电平，50% 表示高低各占一半。

等效平均电压 = 高电平电压 × 占空比。例如 3.3 V 电源下，占空比 50% 的 PWM 等效输出电压约为 1.65 V。这就是 PWM "模拟"任意电压的奥秘。

> 【提示】 严格来说，PWM 输出的是"平均电压"，并不是真正的模拟电压。对于 LED、电机这类对电压纹波不敏感的负载，平均电压足够好用；但对于需要纯净直流电压的场景，就需要配合电容滤波，或使用真正的 DAC（数模转换器，见第 7 章）。

6.3 ESP32 的 ledc API

Arduino 框架中，`analogWrite()` 在 ESP32 上并不默认支持（或功能有限），官方推荐使用 **ledc（LED Control）** API 来产生 PWM。ESP32 内置了多路 **LEDC 硬件单元**，每路都包含独立的定时器和通道，可以产生稳定的硬件 PWM。其使用分四步：

1. 设置定时器：`ledcSetup(channel, freq, resolution)`；
2. 将定时器通道绑定到引脚：`ledcAttachPin(pin, channel)`；
3. 设置占空比：`ledcWrite(channel, duty)`；
4. （可选）解除绑定：`ledcDetachPin(pin)`。

其中 `resolution` 是占空比的分辨率位数，决定 `duty` 的取值范围。例如分辨率为 8 位时，`duty` 范围是 0～255；10 位时是 0～1023；12 位时是 0～4095。分辨率越高，亮度调节越细腻。

6.4 实验：呼吸灯

呼吸灯是 PWM 最经典的入门实验：让 LED 的亮度像呼吸一样平滑地渐亮渐暗，循环往复。电路沿用第 3 章的外接 LED（GPIO5 经 220 Ω 电阻接 LED），代码如下：

```
// 呼吸灯实验
#define LED_PIN 5
#define LEDC_CH 0          // LEDC 通道 0
#define FREQ 5000          // PWM 频率 5 kHz
#define RESOLUTION 8       // 分辨率 8 位（占空比 0~255）

void setup() {
  ledcSetup(LEDC_CH, FREQ, RESOLUTION); // 配置通道
  ledcAttachPin(LED_PIN, LEDC_CH);      // 绑定引脚
}

void loop() {
  // 渐亮：占空比从 0 增加到 255
  for (int duty = 0; duty <= 255; duty++) {
    ledcWrite(LEDC_CH, duty);
    delay(10);
  }
  // 渐暗：占空比从 255 减小到 0
  for (int duty = 255; duty >= 0; duty--) {
    ledcWrite(LEDC_CH, duty);
    delay(10);
```

```
  }
}
```

上传后，你会看到 LED 的亮度平滑地由暗变亮、再由亮变暗，仿佛在"呼吸"。试着把 `delay(10)` 改小，呼吸会加快；改大，则变慢。你也可以把分辨率改成 12 位（`RESOLUTION` 设为 12，循环上限改为 4095），观察亮度变化的细腻程度有何不同。

6.5 PWM 的应用场景

PWM 远不止调光这么简单，它在嵌入式领域应用极广：

- **LED 调光与调色**：调节 RGB LED 三路 PWM 的占空比，可以混合出任意颜色；
- **电机调速**：调节直流电机驱动电路的 PWM 占空比，控制转速；
- **舵机控制**：舵机通过特定频率（约 50 Hz）的 PWM 脉冲宽度（约 0.5～2.5 ms）来确定转动角度；
- **蜂鸣器发声**：通过不同频率的 PWM 驱动无源蜂鸣器，可以演奏音调；
- **加热/制冷控制**：调节加热丝、半导体制冷片的平均功率。

可以说，掌握了 PWM，你就掌握了"控制模拟世界"的一把万能钥匙。

6.6 本章小结

- PWM 通过高速切换高低电平、调节占空比来模拟平均电压；
- 平均电压 = 高电平电压 × 占空比；
- ESP32 使用 `ledcSetup`、`ledcAttachPin`、`ledcWrite` 三步产生硬件 PWM；
- 呼吸灯是 PWM 的经典实验，PWM 广泛应用于调光、调速、舵机、发声等场景。

思考与练习

1. 把呼吸灯改成"由亮到暗只运行一次后常灭"，观察占空比与亮度的对应关系。
2. 用三路 PWM 驱动一颗 RGB LED，尝试混合出红、绿、蓝、黄、青、紫、白七种颜色。
3. 查阅舵机资料，说说舵机的角度是如何由 PWM 脉冲宽度决定的。

第7章 ADC 模拟信号采集

7.1 学习目标

现实世界是"模拟"的——温度、光照、音量、压力等物理量都是连续变化的，而芯片只能处理"数字"的 0 和 1。要在二者之间架起桥梁，就需要**模数转换器（ADC, Analog-to-Digital Converter）**。本章将学习如何使用 ESP32 的 ADC 采集模拟电压，学完后你将能够：

- 理解 ADC 的基本原理（采样、量化、分辨率）；
- 使用 `analogRead()` 读取模拟电压；
- 用电位器（旋钮）实现调光、调节等交互；
- 了解 ESP32 ADC 的非线性与校准问题。

7.2 ADC 原理

ADC 的作用是把一个连续的模拟电压，转换成一个离散的数字值。这个过程可以拆成两个关键步骤：

- **采样（Sampling）**：每隔一定时间，读取一次当前的电压值；
- **量化（Quantization）**：把连续电压划分成若干等级，用最接近的整数表示。

ADC 的分辨率决定了它能把电压分成多少级。ESP32 的 ADC 是 **12 位**的，即可以把 0～3.3 V（量程）分成 2^12 = 4096 个等级，最小分辨电压约为 3.3 V ÷ 4096 ≈ 0.8 mV。也就是说，`analogRead()` 返回的数值范围是 0～4095：0 对应 0 V，4095 对应约 3.3 V。

> 【提示】 什么是"位（bit）"？这里的"12 位"表示数字结果用 12 个二进制位表示，取值范围是 0 到 2^12−1。位数越高，量化越精细，分辨率越高。相比之下，传统 Arduino Uno 的 ADC 只有 10 位（0～1023），ESP32 的精度要高得多。

7.3 电位器与分压

电位器（Potentiometer） 是一个阻值可调的电阻，通常有三个引脚：两端是固定电阻的两端，中间是滑动触点。旋转旋钮时，滑动触点的位置改变，中间引脚与两端之间的阻值随之变化。把电位器两端分别接 3.3 V 和 GND，中间引脚作为输出，就构成了一个**分压电路**——旋转旋钮，中间引脚输出的电压就在 0～3.3 V 之间连续变化。这个可变的模拟电压，正好可以用来测试 ADC。

7.4 实验一：读取电位器电压

接线（以 GPIO34 为例，输入引脚选 GPIO34～GPIO39 这类仅输入引脚更安全）：

- 电位器一端接 3V3，另一端接 GND；
- 电位器中间引脚接 GPIO34。

```
// 读取电位器模拟电压
#define POT_PIN 34

void setup() {
  Serial.begin(115200);  // 初始化串口
}

void loop() {
  int value = analogRead(POT_PIN);        // 读取 0~4095 的原始值
  float voltage = value * 3.3 / 4095.0;   // 换算成电压
  Serial.print("ADC值: ");
  Serial.print(value);
  Serial.print("  电压: ");
  Serial.print(voltage);
  Serial.println(" V");
  delay(200);
}
```

上传后打开串口监视器（波特率 115200），旋转电位器，你会看到 ADC 值在 0～4095 之间变化，对应电压在 0～3.3 V 之间变化。这是后续所有"传感器读数"的基础——绝大多数模拟传感器（光敏、气敏、土壤湿度、模拟温度传感器等）本质上都输出一个随物理量变化的电压，通过 ADC 就能把它变成程序里的数字。

7.5 实验二：电位器调光

结合第 6 章学的 PWM，我们可以做一个漂亮的交互：**旋转电位器调节 LED 亮度**。电路在上一个实验的基础上，再把 LED 经 220 Ω 电阻接到 GPIO5。

```
// 电位器调光
#define POT_PIN 34
#define LED_PIN 5
#define LEDC_CH 0

void setup() {
  ledcSetup(LEDC_CH, 5000, 8);   // 8 位分辨率
```

```
  ledcAttachPin(LED_PIN, LEDC_CH);
}

void loop() {
  int value = analogRead(POT_PIN);          // 0~4095
  int duty = map(value, 0, 4095, 0, 255);   // 映射到 0~255
  ledcWrite(LEDC_CH, duty);
  delay(10);
}
```

这里用到了 `map()` 函数，它把一个数值区间线性映射到另一个区间：`map(value, fromLow, fromHigh, toLow, toHigh)`。旋转电位器，LED 亮度随之连续变化。这个"采集模拟量 → 映射 → 控制输出"的模式，是嵌入式开发中最经典的信号处理链路之一。

7.6 ESP32 ADC 的注意事项

ESP32 的 ADC 虽然强大，但也有几个"个性"需要了解：

- **量程与衰减**：默认测量范围约为 0～1.1 V，要测到 3.3 V 需要设置**衰减（Attenuation）**。Arduino 框架已自动配置为 11 dB 衰减，使量程达到约 0～3.1 V（部分板实测到 3.3 V），因此我们直接读即可。
- **非线性**：ESP32 的 ADC 在量程两端（接近 0 和接近满量程）线性度较差，高精度应用需做多点校准。
- **ADC2 与 Wi-Fi 冲突**：ESP32 有两组 ADC——ADC1 和 ADC2。当 Wi-Fi 开启时，ADC2 的部分通道不可用（被 Wi-Fi 内部占用）。因此**在联网项目中，优先使用 ADC1（GPIO32～GPIO39）**。
- **输入阻抗**：ADC 输入阻抗有限，测量高内阻信号源时结果会偏低，必要时加运放缓冲。

7.7 本章小结

- ADC 通过采样和量化把模拟电压转换为数字值；
- ESP32 的 ADC 为 12 位，`analogRead()` 返回 0～4095；
- 电位器分压是测试 ADC 的经典电路；
- "采集→映射→输出"是模拟信号处理的核心链路；
- 联网项目应优先使用 ADC1（GPIO32～39），避免 ADC2 与 Wi-Fi 冲突。

思考与练习

1. 把电位器调光改为"调节 LED 闪烁频率"，旋钮越亮闪烁越快。

2. 用手边任何模拟传感器（如光敏电阻）做一个简易"光强计"，在串口打印读数。
3. 思考：为什么 12 位 ADC 返回的最大值是 4095 而不是 4096?

第8章 定时器与中断

8.1 学习目标

到目前为止，我们的程序都是"顺序执行"的：一行一行地跑，用 `delay()` 等待。这种写法虽然简单，但有一个致命缺陷——在 `delay()` 期间，芯片什么都做不了，一旦程序需要同时响应多个事件（比如一边闪灯、一边检测按键、一边读传感器），就会手忙脚乱。本章引入两个解决这一问题的利器：**定时器** 和 **中断**。学完后你将能够：

- 理解中断的概念与工作流程；
- 使用外部中断响应按键等突发事件；
- 使用硬件定时器实现精确、非阻塞的定时任务；
- 理解中断服务函数中的注意事项。

8.2 中断的概念

想象你正在专心读书，突然电话铃响——你会放下书去接电话，接完再回来继续读。**中断**就是这样一种机制：当某个外部事件（按键按下、引脚电平变化、定时器到期、串口收到数据等）发生时，芯片会暂停当前正在执行的程序，转而执行一段专门处理该事件的函数（称为**中断服务函数**，ISR，Interrupt Service Routine），处理完毕后再回到原来的位置继续执行。

中断的意义在于**实时响应**。如果没有中断，检测按键就得靠 `loop()` 反复轮询——轮询频率不够快就可能漏掉按键；而有了中断，无论程序在忙什么，按键一按下就能立刻被捕捉到。

中断虽然强大，但也要慎用。中断服务函数必须**尽量短小、快速**，因为它在执行期间会打断主程序，其他中断也可能被暂时屏蔽。下面这些操作在中断服务函数里要格外注意：不要用 `delay()`（会卡死），不要做耗时操作，不要打印大量串口信息，通常只需设置一个标志位，让主程序随后去处理。

8.3 外部中断实验

ESP32 的每一个 GPIO 引脚都可以配置为外部中断源，中断触发方式有四种：下降沿（HIGH→LOW）、上升沿（LOW→HIGH）、任意边沿变化、低电平触发。下面用外部中断做一个按键计数实验。

接线：按键一端接 GPIO23，另一端接 GND（使用内部上拉，按下时产生下降沿）。

```
// 外部中断按键计数
#define BUTTON_PIN 23

volatile int count = 0;   // 计数器，volatile 修饰见下文

// 中断服务函数
void IRAM_ATTR onPress() {
  count++;                  // 每触发一次中断，计数加一
}

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  // 绑定中断：下降沿触发
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), onPress, FALLING);
}

void loop() {
  Serial.print("按键次数: ");
  Serial.println(count);
  delay(500);
}
```

这段程序有两个关键点：

- `attachInterrupt(pin, isr, mode)`：把某个引脚的中断与处理函数绑定。`FALLING` 表示下降沿触发，即按键按下的瞬间触发一次。
- `IRAM_ATTR`：这是一个编译指令，把中断服务函数放到芯片的快速 RAM（IRAM）中执行，避免因 Flash 读取延迟导致中断响应异常。**ESP32 的中断服务函数建议都加上 `IRAM_ATTR`**。
- `volatile`：这是一个关键字，告诉编译器该变量可能被"意料之外"地改变（比如被中断修改），从而禁止编译器对它做过度的优化缓存，确保主程序每次都能读到最新值。

上传后，每按一次按键，串口里的计数值就加一。即使 `loop()` 里有 `delay(500)`，按键也能被及时捕捉——这正是中断相对于轮询的优势。

8.4 硬件定时器

外部中断解决的是"外部事件触发"，而**定时器（Timer）** 解决的是"定时触发"——让芯片每隔固定时间自动执行一次任务，且完全不影响主程序的运行。这比 `delay()` 高明得多。

ESP32 内置了 4 个 64 位硬件定时器。Arduino 框架下，常用 `esp_timer` 或 `Ticker` 库来使用定时器。下面以简洁的 **Ticker** 库为例，实现"LED 每 500 毫秒自动翻转一次"。

```
// 硬件定时器（Ticker 库）实现 LED 自动闪烁
#include <Ticker.h>

#define LED_PIN 5
Ticker ticker;

void toggleLED() {
  // 翻转 LED：读取当前状态取反
  digitalWrite(LED_PIN, !digitalRead(LED_PIN));
}

void setup() {
  pinMode(LED_PIN, OUTPUT);
  ticker.attach(0.5, toggleLED);  // 每 0.5 秒调用一次 toggleLED
}

void loop() {
  // 主循环空闲，可以去做别的事，LED 依然在自动闪烁
}
```

上传后，LED 以 0.5 秒为周期自动翻转闪烁，而 `loop()` 里空空如也——这说明定时任务已经完全"独立"于主循环运行。这正是非阻塞编程的核心思想：**把周期性任务交给定时器，让主循环保持清爽，随时响应其他事件**。

> 【注意】 注意：和外部中断一样，定时器回调函数也应尽量短小，避免在其中做耗时操作或调用 `delay()`。

8.5 中断与定时器实战：非阻塞呼吸灯 + 按键

我们把前面的知识综合起来，实现一个"进阶版呼吸灯"：LED 持续做呼吸灯效果，同时按键可以随时暂停/恢复呼吸。通过定时器驱动呼吸、中断捕捉按键，彻底摆脱 `delay()`。

```
// 非阻塞呼吸灯 + 按键暂停
#include <Ticker.h>

#define LED_PIN 5
#define BUTTON_PIN 23
#define LEDC_CH 0

Ticker ticker;
volatile bool paused = false;  // 暂停标志
int duty = 0;
int step = 5;                  // 每步占空比变化量

void IRAM_ATTR onPress() {
  paused = !paused;            // 按键翻转暂停状态
}

void breathe() {
  if (paused) return;          // 暂停时直接返回
  duty += step;
  if (duty >= 255 || duty <= 0) step = -step; // 到边界反向
  ledcWrite(LEDC_CH, duty);
}

void setup() {
  ledcSetup(LEDC_CH, 5000, 8);
  ledcAttachPin(LED_PIN, LEDC_CH);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), onPress, FALLING);
  ticker.attach_ms(20, breathe);  // 每 20 毫秒更新一次占空比
}

void loop() {
  // 主循环空置，呼吸与按键均由硬件机制驱动
}
```

这个例子展示了中断与定时器配合的威力：呼吸灯由定时器驱动，按键由中断捕捉，主循环完全空闲，程序结构清晰、响应及时、扩展性强。请务必亲手运行并体会这种"事件驱动"的编程思想，它是后续学习 FreeRTOS 多任务的重要铺垫。

8.6 本章小结

- 中断让程序能实时响应外部事件，中断服务函数必须短小，且应加 `IRAM_ATTR`；
- 被中断修改的共享变量要用 `volatile` 修饰；
- 硬件定时器（如 Ticker）实现非阻塞的周期任务，摆脱 `delay()` 的阻塞；
- 事件驱动思想是编写可靠嵌入式程序的基础。

思考与练习

1. 把外部中断的触发方式改为上升沿（RISING），观察计数时机有何变化。
2. 用 Ticker 实现三个不同频率的 LED 同时闪烁（如 200 ms、500 ms、1000 ms）。
3. 说说为什么中断服务函数里不能调用 `delay()`，会带来什么问题。

第9章 UART 串口通信

9.1 学习目标

串口（UART）是嵌入式世界最古老、最常用的通信方式之一，也是我们调试程序、与电脑交换数据、连接各种模块（GPS、蓝牙、指纹模块等）的"通用语言"。从第 2 章开始，我们其实一直在用串口打印调试信息。本章将系统学习 UART，学完后你将能够：

- 理解 UART 的通信原理与波特率概念；
- 使用 ESP32 的多路硬件串口；
- 实现 ESP32 与电脑的双向数据收发；
- 编写一个简单的串口指令控制程序。

9.2 UART 通信原理

UART（Universal Asynchronous Receiver/Transmitter，通用异步收发器）是一种**异步、全双工**的串行通信协议。所谓"串行"，是指数据一位一位地顺序传输；"异步"是指通信双方不共享时钟信号，而是各自按约定的速率（波特率）来解读数据；"全双工"是指收发可以同时进行，各用一根线。

一次 UART 通信至少需要三根线：

- **TX（发送）**：本方发送数据，接到对方的 RX；
- **RX（接收）**：本方接收数据，接到对方的 TX；
- **GND（共地）**：双方必须共地，这是所有通信的前提。

> 【提示】 接线的黄金法则：**TX 接对方的 RX，RX 接对方的 TX，GND 接 GND**。这是新手最容易接反的地方，务必牢记。

UART 的每个数据帧包含：一个起始位（低电平，标志数据开始）、若干数据位（通常 8 位）、可选的校验位、以及一个或多个停止位（高电平，标志数据结束）。通信双方必须约定一致的关键参数是**波特率（Baud Rate）**，即每秒传输的码元数，常用 9600、115200 等。波特率不匹配，收到的就是乱码——这也解释了第 2 章"乱码"问题的原因。

9.3 ESP32 的多路串口

经典 ESP32 提供 3 个硬件 UART：

- **UART0**：默认连接 USB 转串口芯片（GPIO1=TX，GPIO3=RX），用于烧录和调试打印，即 `Serial`；
- **UART1**：默认引脚被 Flash 占用，一般不直接使用；
- **UART2**：默认 GPIO17=TX、GPIO16=RX，可作为第二路串口连接外部设备。

在 Arduino 框架中，可以通过 `Serial`、`Serial1`、`Serial2` 三个对象访问这三路串口。其中 `Serial` 用于调试，`Serial2` 常用于连接 GPS、蓝牙等外部模块。得益于 GPIO 矩阵，串口引脚还可以映射到其他可用 GPIO。

9.4 实验：双向收发与串口指令

我们做一个实用的"串口指令控制"程序：电脑通过串口发送单个字符命令（如 `1` 开灯、`0` 关灯），ESP32 收到后执行并回复确认信息。

```
// 串口指令控制 LED
#define LED_PIN 5

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("指令：1=开灯  0=关灯");
}

void loop() {
  if (Serial.available() > 0) {   // 检查串口是否有数据到达
    char cmd = Serial.read();     // 读取一个字符
    if (cmd == '1') {
      digitalWrite(LED_PIN, HIGH);
      Serial.println("LED 已打开");
    } else if (cmd == '0') {
      digitalWrite(LED_PIN, LOW);
      Serial.println("LED 已关闭");
    } else {
      Serial.println("未知指令");
    }
  }
}
```

这里用到的关键函数：

- `Serial.available()`：返回接收缓冲区中待读取的字节数，大于 0 表示有数据到达；
- `Serial.read()`：读取一个字节（字符）；
- `Serial.write()` / `Serial.print()` / `Serial.println()`：发送数据。

上传后打开串口监视器，在输入框输入 `1` 并发送，LED 点亮且监视器回复"LED 已打开"；输入 `0` 则熄灭。注意串口监视器底部通常有"换行"选项，如果选择"无结束符"，程序也能正常工作，因为本例只读取单个字符。

9.5 串口的高级话题

数据格式：串口传输的是字节流，如果要发送数字、文本或结构化的数据，需要双方约定格式。例如发送传感器读数时，常用 `Serial.print(value)` 加分隔符（逗号、空格）的方式，接收方按分隔符解析；或使用更结构化的 JSON 文本。

硬件串口不够用：当需要连接多个串口设备时，可以使用 **SoftwareSerial（软件串口）** 库，用普通 GPIO 模拟串口时序。软件串口实现简单，但占用 CPU 资源、波特率上限较低，重要通信仍建议优先使用硬件串口。

调试技巧：养成在关键节点用 `Serial.print` 打印变量值和状态的习惯，是嵌入式调试最基本也最有效的手段。在第 9 章之后的所有项目中，请务必善用串口打印来观察程序运行状态。

9.6 本章小结

- UART 是异步、全双工、串行的通信协议，TX 接对方 RX、RX 接对方 TX、共地；
- 波特率是通信双方必须一致的关键参数；
- ESP32 有 3 路硬件串口，`Serial` 用于调试，`Serial2` 可连接外部设备；
- `Serial.available()`、`Serial.read()`、`Serial.print()` 是串口编程的核心函数。

思考与练习

1. 扩展串口指令程序，增加 `2`（闪烁模式）、`3`（呼吸灯模式）两个命令。
2. 用 `Serial2` 连接一个支持串口的模块（如 GPS），在串口监视器转发其输出。
3. 解释为什么通信双方必须共地，如果忘记共地会发生什么。

第10章 I2C 总线通信

10.1 学习目标

现实项目中，单片机常常需要同时连接多个传感器、存储器、显示屏。如果每个器件都要占用好几根独立的引脚，引脚很快就不够用了。**总线（Bus）** 就是为解决这个问题而生——让多个器件共享同一组通信线路。本章学习应用最广泛的 **I2C 总线**，学完后你将能够：

- 理解 I2C 的主从架构与总线机制；
- 使用 Wire 库读写 I2C 器件；
- 完成 I2C 地址扫描，识别总线上的设备；
- 驱动一个 I2C 器件（如 OLED 屏或传感器）读取数据。

10.2 I2C 总线原理

I2C（Inter-Integrated Circuit，集成电路间总线，读作"I方C"或"I two C"）由飞利浦公司发明，是一种**两线制**的串行总线。它的最大特点是只用两根线就能挂接多达上百个器件：

- **SDA（数据线）**：传输数据；
- **SCL（时钟线）**：由主机提供的时钟信号，同步数据传输。

I2C 采用**主从（Master-Slave）**架构：总线上有一个主设备（通常是单片机，如 ESP32）和多个从设备（传感器、存储器等）。主设备发起通信，每个从设备都有一个独一无二的 **7 位（或 10 位）地址**，主设备通过地址来"点名"要通信的对象。

I2C 的数据线是**开漏（Open-Drain）**结构，因此 SDA 和 SCL 都必须外接**上拉电阻**（典型值 4.7 kΩ）到电源正极。这也是 I2C 电路区别于其他总线的标志性特征。ESP32 的部分引脚有内部上拉，但外接上拉电阻通常更稳定可靠。

I2C 一次典型的读写流程是：主机发送**起始条件** → 发送从机地址和读写位 → 从机应答（ACK）→ 传输数据 → 主机发送**停止条件**。得益于成熟的库封装，我们在编程时无需关心这些底层细节。

10.3 Wire 库与引脚配置

Arduino 框架通过 **Wire 库** 提供 I2C 支持。ESP32 的 I2C 引脚可通过 `Wire.begin(sda, scl)` 灵活指定，默认是 GPIO21（SDA）和 GPIO22（SCL）。使用前需在 `setup()` 中初始化：

```
Wire.begin(21, 22);   // SDA=21, SCL=22
```

也可以不传参数使用默认引脚：`Wire.begin()`。多数 ESP32 开发板把 GPIO21/GPIO22 作为 I2C 的默认引脚，本书后续 I2C 实验均使用这两个引脚。

10.4 实验一：I2C 地址扫描

当我们拿到一个未知的 I2C 器件时，第一件事往往是确认它的 I2C 地址。下面这个经典的 **I2C 扫描器** 程序，会遍历所有可能的地址，报告总线上有哪些设备。

```
// I2C 地址扫描器
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  Serial.println("开始扫描 I2C 设备...");
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);   // 尝试向地址 addr 发送
    if (Wire.endTransmission() == 0) {  // 返回 0 表示有设备应答
      Serial.print("发现设备，地址 0x");
      Serial.println(addr, HEX);
    }
  }
  Serial.println("扫描完成");
}

void loop() {
}
```

上传并打开串口监视器，即可看到总线上所有设备的十六进制地址。请记住这些地址，后续编写读取代码时会用到。常见的 I2C 器件地址如：OLED 屏（SSD1306）通常为 0x3C，MPU6050 为 0x68，BMP280 为 0x76 或 0x77。

10.5 实验二：驱动 I2C 器件读取数据

下面以极其常见的 **BMP280 气压/温度传感器** 为例，演示如何用现成库驱动 I2C 器件。BMP280 可以测量温度、大气压，并推算海拔，广泛用于气象站、无人机等场景。

接线：BMP280 模块通常有 VCC、GND、SCL、SDA 四根线，分别接 3V3、GND、GPIO22（SCL）、GPIO21（SDA）。部分模块还有 CSB、SDO 引脚，本实验不使用。

安装库：在 Arduino IDE 中进入"工具 → 管理库"，搜索 "Adafruit BMP280"，安装 Adafruit BMP280 库（会自动附带依赖库）。这是 Arduino 生态的强大之处——几乎所有常见器件都有现成的开源库。

```
// 使用 BMP280 读取温度与气压
#include <Wire.h>
#include <Adafruit_BMP280.h>

Adafruit_BMP280 bmp;   // 创建传感器对象

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  if (!bmp.begin(0x76)) {   // 地址 0x76，失败则提示
    Serial.println("BMP280 未找到，请检查接线！");
    while (1);              // 卡住，等待检查
  }
}

void loop() {
  Serial.print("温度: ");
  Serial.print(bmp.readTemperature());  // 摄氏度
  Serial.print(" ℃  气压: ");
  Serial.print(bmp.readPressure() / 100.0F);  // 百帕
  Serial.print(" hPa  海拔: ");
  Serial.print(bmp.readAltitude(1013.25));  // 相对标准海平面气压
  Serial.println(" m");
  delay(1000);
}
```

上传后，串口会每秒打印一次温度、气压和海拔数据。用手捂住传感器，温度读数会逐渐上升；把传感器举高或放低，海拔读数会相应变化。这就是一个完整的"传感器数据采集"范例，几乎所有的 I2C 传感器都遵循同样的模式：**初始化对象 → begin 检查 → 调用 read 函数读取**。

10.6 I2C 的优缺点

优点：只用两根线即可挂接大量设备；协议简单、库生态成熟；支持多主多从。**缺点**：速度相对较慢（标准 100 kbps、快速 400 kbps、高速 3.4 Mbps）；总线上的器件地址可能冲突；上拉电阻选择不当会导致通信不稳定。当需要高速、大吞吐量通信（如显示屏刷屏、高速 ADC）时，就应改用下一章要讲的 SPI 总线。

10.7 本章小结

- I2C 是两线（SDA/SCL）主从架构的总线，需外接上拉电阻；
- 每个从设备有唯一地址，主设备按地址寻址通信；
- Wire 库提供 I2C 支持，`Wire.begin(sda, scl)` 初始化；
- I2C 扫描器是排查器件的利器；
- 驱动 I2C 器件的通用模式：创建对象 → begin 检查 → 调用读取函数。

思考与练习

1. 运行 I2C 扫描器，记录你手头所有 I2C 器件的地址。
2. 用 BMP280 连续采集 5 分钟数据，观察温度的变化趋势。
3. 查阅资料，说说 I2C 总线上为什么必须接上拉电阻。

第11章 SPI 总线通信

11.1 学习目标

当 I2C 的速度不够用时（例如驱动高分辨率显示屏、高速 SD 卡读写），就该请出**更快**的 **SPI 总线** 了。SPI 以其高速、全双工的特点，成为显示屏、存储卡、高速传感器等场景的首选。学完本章，你将能够：

- 理解 SPI 的四线制通信架构；
- 使用 Arduino 的 SPI 库进行通信；
- 理解 SPI 与 I2C 的异同，能根据场景选择合适的总线；
- 驱动一个 SPI 器件（如 SD 卡模块）。

11.2 SPI 总线原理

SPI（Serial Peripheral Interface，串行外设接口）是一种**四线制、全双工、高速**的同步串行总线，同样采用主从架构，通常是一个主机带一个或多个从机。它的四根线分别是：

- **SCLK（时钟线）**：由主机产生，同步数据传输节拍；
- **MOSI（Master Out Slave In）**：主机输出、从机输入的数据线；
- **MISO（Master In Slave Out）**：从机输出、主机输入的数据线；
- **CS/SS（片选线）**：用于选择要通信的从机，低电平有效。

> 【提示】 记忆技巧：MOSI 和 MISO 的方向是相对主机而言的——MOSI 是"主出从入"，MISO 是"主入从出"。

SPI 的关键优势在于**全双工**：主从双方可以在时钟的驱动下**同时**发送和接收数据，每来一个时钟脉冲就传输一位。相比之下，I2C 是半双工的（同一时刻只能一个方向传）。此外，SPI 没有复杂的寻址和应答机制，速度上限远高于 I2C，可达数十 MHz。

当总线上有多个从机时，每个从机需要一根独立的 **CS 片选线**，主机通过拉低对应从机的 CS 来选中它。这也是 SPI 相对 I2C 的一个代价：器件越多，占用的片选引脚就越多。

11.3 ESP32 的 SPI 引脚

ESP32 的默认 SPI 引脚（称为 VSPI）为：MOSI=GPIO23、MISO=GPIO19、SCLK=GPIO18、CS 可根据需要选任意 GPIO（常用 GPIO5）。Arduino 框架中，使用 `SPI.begin()` 即可初始化默认引脚。得益于 GPIO 矩阵，这些引脚同样可以重映射。

11.4 实验：驱动 SD 卡模块

SD 卡是嵌入式项目中存储数据（日志、图片、配置文件）的常用介质。下面用一个常见的 **Micro SD 卡模块** 演示 SPI 通信。这类模块集成了电平转换电路，可直接接 3.3 V 逻辑的 ESP32。

接线（以常见 Micro SD 模块为例）：

- VCC → 5V（模块上有稳压，接 5V 更稳定）或 3V3；
- GND → GND；
- CS → GPIO5；
- MOSI → GPIO23；
- MISO → GPIO19；
- SCK → GPIO18。

安装库：在"管理库"中搜索 "SD"，安装 Arduino 官方 SD 库。

```
// 使用 SPI 读写 SD 卡
#include <SPI.h>
#include <SD.h>

#define CS_PIN 5

void setup() {
  Serial.begin(115200);
  if (!SD.begin(CS_PIN)) {   // 初始化 SD 卡
    Serial.println("SD 卡初始化失败，请检查接线与卡格式！");
    return;
  }
  Serial.println("SD 卡就绪");

  File f = SD.open("/test.txt", FILE_WRITE);  // 打开（或创建）文件
  if (f) {
    f.println("Hello, ESP32 SD Card!");       // 写入一行文本
    f.close();
    Serial.println("已写入 test.txt");
```

```
  }

  f = SD.open("/test.txt");                    // 重新打开读取
  if (f) {
    Serial.println("读取内容: ");
    while (f.available()) {                    // 逐字节读取
      Serial.write(f.read());
    }
    f.close();
  }
}

void loop() {
}
```

上传后，串口会显示"SD 卡就绪"，写入并读回 test.txt 文件的内容。这就是 SPI 通信的典型应用——高速、大容量的数据存储。掌握了 SPI，你就可以驱动显示屏、读 SD 卡、操作高速 ADC/DAC 等大量外设。

11.5 SPI 与 I2C 对比

特性	I2C	SPI
线数	2 根（SDA/SCL）	4 根（SCLK/MOSI/MISO/CS）
速度	最高约 3.4 Mbps	可达数十 MHz
双工	半双工	全双工
多从机	按地址寻址，共享两线	每从机需一根片选线
上拉电阻	必须	不需要
典型场景	低速传感器、RTC、小屏	高速显示、SD 卡、高速采集

简单来说：**器件少、速度要求高、数据量大 → 用 SPI；器件多、引脚紧张、速度要求不高 → 用 I2C**。实际选型时还需结合具体器件支持的总线类型来定。

11.6 本章小结

- SPI 是四线、全双工、高速的同步总线，MOSI/MISO 方向相对主机；
- 多从机通过片选线 CS 区分，每个从机一根 CS；
- ESP32 默认 SPI 引脚为 MOSI=23、MISO=19、SCLK=18、CS=5；
- SD 卡是 SPI 的典型应用，演示了初始化、写入、读取的完整流程。

思考与练习

1. 用 SPI 连接一个 SD 卡，尝试写入并读回一个较大的文件（如 1 MB）。
2. 手绘一张 I2C 与 SPI 的对比表，注明各自适合的场景。
3. 查阅资料，说明 SPI 的四种工作模式（CPOL/CPHA）分别是什么意思。

第三部分 进阶篇

第12章 常用传感器实战

12.1 学习目标

从本章开始，我们正式踏入物联网的广阔天地。而物联网的第一步，是"感知世界"——通过各种各样的传感器，把温度、湿度、光照、距离、运动等物理量变成芯片能处理的数据。本章将把前面学到的 GPIO、ADC、I2C 等知识融会贯通，实战驱动几类最常用的传感器。学完后你将能够：

- 掌握数字传感器、模拟传感器、I2C 传感器的驱动套路；
- 读懂传感器数据手册（Datasheet）的关键参数；
- 使用温湿度传感器、超声波测距、运动传感器等完成数据采集。

12.2 数字传感器：DHT11/DHT22 温湿度

DHT11 和 DHT22（又称 AM2302）是最常见的温湿度传感器，其中 DHT22 精度更高、量程更宽。它们使用一根数据线传输数据，属于**单总线**协议。Arduino 生态下有成熟的 DHT 库可用。

接线：DHT11/DHT22 模块通常有 VCC、DATA、GND 三脚，分别接 3V3、GPIO4（数据脚可自选）、GND。数据脚建议接一个 4.7 kΩ 上拉电阻（很多模块已内置）。

安装库：在"管理库"搜索 "DHT sensor library"（作者 Adafruit）并安装。

```
// DHT22 温湿度读取
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT22   // 若用 DHT11 则改为 DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
}

void loop() {
  float h = dht.readHumidity();     // 读湿度
  float t = dht.readTemperature();  // 读温度（摄氏度）
```

```
  if (isnan(h) || isnan(t)) {       // 读数无效（NaN）时提示
    Serial.println("读取失败，请检查接线");
  } else {
    Serial.print("温度: "); Serial.print(t); Serial.print(" ℃  ");
    Serial.print("湿度: "); Serial.print(h); Serial.println(" %");
  }
  delay(2000);   // DHT22 采样间隔需 ≥2 秒
}
```

> 【注意】 注意：DHT 系列传感器读取频率不能太高（DHT22 最小间隔约 2 秒，DHT11 约 1 秒），否则读数会出错。此外，读取结果偶尔会失败（返回 NaN），程序里应做好判空处理。

12.3 超声波测距：HC-SR04

超声波测距模块 HC-SR04 通过发射超声波并测量回波时间来计算距离，广泛用于避障小车、液位检测等。它有两个引脚（Trig 触发、Echo 回波），测量原理是：给 Trig 一个 10 微秒的高电平脉冲触发测距，然后测量 Echo 引脚高电平持续的时间，距离 = 高电平时间 × 声速 ÷ 2。

```
// HC-SR04 超声波测距
#define TRIG_PIN 5
#define ECHO_PIN 18

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}

void loop() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);   // 触发 10us 脉冲
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH);  // 测量高电平持续时间（微秒）
  float distance = duration * 0.034 / 2;    // 声速约 0.034 cm/us

  Serial.print("距离: ");
```

```
  Serial.print(distance);
  Serial.println(" cm");
  delay(500);
}
```

这里用到了 `pulseIn()` 函数，它测量某个引脚上脉冲的持续时间。`delayMicroseconds()` 则用于微秒级延时。上传后，将手放在模块前方移动，串口会实时显示距离。HC-SR04 的测量范围约为 2～400 厘米。

12.4 运动传感器：MPU6050 六轴

MPU6050 集成了三**轴加速度计**和三**轴陀螺仪**，能检测物体的加速度和旋转角速度，是平衡车、无人机、姿态检测的核心器件。它通过 I2C 通信，地址通常为 0x68。

接线：VCC→3V3、GND→GND、SCL→GPIO22、SDA→GPIO21。

安装库：搜索 "Adafruit MPU6050" 安装。

```
// MPU6050 读取加速度与角速度
#include <Wire.h>
#include <Adafruit_MPU6050.h>

Adafruit_MPU6050 mpu;

void setup() {
  Serial.begin(115200);
  if (!mpu.begin()) {
    Serial.println("MPU6050 未找到! ");
    while (1);
  }
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);  // 加速度量程 ±8g
}

void loop() {
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);   // 读取数据

  Serial.print("加速度 X/Y/Z: ");
  Serial.print(a.acceleration.x); Serial.print(" / ");
  Serial.print(a.acceleration.y); Serial.print(" / ");
  Serial.println(a.acceleration.z);
  delay(500);
}
```

上传后，晃动或旋转模块，串口里的加速度数值会随之变化。静止水平放置时，Z 轴加速度约为 9.8 m/s^2（即重力加速度），这正是加速度计能检测"姿态"的原因——它测的是包括重力在内的合加速度。

12.5 传感器选型与数据手册阅读

随着学习的深入，你迟早要面对"没有现成库"的新传感器，这时就需要会读**数据手册（Datasheet）**。阅读数据手册时，应重点关注以下内容：

- **供电电压**：确认能否直接接 3.3 V，还是要 5 V 甚至更高；
- **通信接口**：是 I2C、SPI、UART 还是模拟/数字输出，决定接线和驱动方式；
- **地址与引脚定义**：I2C 器件的地址、各引脚的功能；
- **测量范围与精度**：决定能否满足项目需求；
- **时序要求**：如采样间隔、启动时间等限制。

掌握了这些要点，再配合厂商提供的示例代码或时序图，你就能驱动绝大多数传感器，而不再受限于"有没有现成库"。

12.6 本章小结

- 数字传感器（DHT）用单总线，注意采样间隔和 NaN 处理；
- 超声波测距用脉冲计时，`pulseIn()` 测量回波时长；
- I2C 传感器（MPU6050）遵循"创建对象 → begin → 读取"套路；
- 阅读数据手册重点关注供电、接口、地址、量程、时序。

思考与练习

1. 把 DHT22 与 HC-SR04 结合起来，做一个"环境监测"小装置，同时显示温湿度和距离。
2. 用 MPU6050 检测模块是否处于水平状态（通过 Z 轴加速度判断），并点亮 LED 提示。
3. 找一款你感兴趣的、没有现成库的传感器，试着读它的数据手册，写出驱动思路。

第13章 Wi-Fi 网络入门

13.1 学习目标

终于来到 ESP32 的"主场"——**Wi-Fi 联网**。正是 Wi-Fi 能力，让 ESP32 从一块普通的单片机，一跃成为物联网的核心枢纽。本章将学习让 ESP32 连接无线网络，学完后你将能够：

- 理解 Wi-Fi 的基础概念（SSID、IP 地址、MAC 地址）；
- 让 ESP32 作为 Station 连接到家庭/办公 Wi-Fi；
- 获取并显示设备的 IP 地址等网络信息；
- 理解 ESP32 的三种 Wi-Fi 工作模式。

13.2 Wi-Fi 基础概念

SSID（Service Set Identifier）：无线网络的名称，即我们在手机上看到的 Wi-Fi 名字。**密码**用于加密认证，常见加密方式为 WPA/WPA2。**MAC 地址**是每块网卡出厂时唯一分配的硬件地址，形如 `24:6F:28:xx:xx:xx`。**IP 地址**是设备接入网络后分配的逻辑地址，形如 `192.168.1.100`，由路由器通过 DHCP 自动分配。设备之间通过 IP 地址相互访问。

ESP32 的 Wi-Fi 支持三种工作模式：

- **Station（STA，站点模式）**：ESP32 作为"客户端"，连接到已有的路由器，就像手机连 Wi-Fi。这是最常用的模式。
- **AP（SoftAP，接入点模式）**：ESP32 自己"变身"成一个热点，让其他设备连接它。适合设备首次配网等场景。
- **STA+AP 共存**：同时工作于两种模式，既能联网又能提供热点。

13.3 连接 Wi-Fi 的经典代码

Arduino 框架下，使用 `WiFi.h` 库即可轻松连接网络。下面是一段几乎所有 ESP32 项目都会用到的"联网模板"：

```
// 连接 Wi-Fi 并显示 IP 地址
#include <WiFi.h>

const char* ssid = "你的WiFi名称";       // 替换为你的 Wi-Fi SSID
const char* password = "你的WiFi密码";  // 替换为你的 Wi-Fi 密码

void setup() {
```

```
  Serial.begin(115200);

  WiFi.mode(WIFI_STA);             // 设为站点模式
  WiFi.begin(ssid, password);      // 开始连接
  Serial.print("正在连接");

  while (WiFi.status() != WL_CONNECTED) {  // 循环等待连接成功
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("连接成功! ");
  Serial.print("IP 地址: ");
  Serial.println(WiFi.localIP());       // 打印本机 IP
  Serial.print("MAC 地址: ");
  Serial.println(WiFi.macAddress());    // 打印 MAC 地址
  Serial.print("信号强度(RSSI): ");
  Serial.println(WiFi.RSSI());          // 打印信号强度
}

void loop() {
}
```

这段程序的关键点：

- `WiFi.begin(ssid, password)` 发起连接，这是一个**异步**过程，需要几秒才能完成；
- `WiFi.status()` 返回当前连接状态，`WL_CONNECTED` 表示已连接，循环里不断检查直到成功；
- `WiFi.localIP()` 返回路由器分配的 IP 地址，这是后续所有网络通信的基础。

上传后，打开串口监视器，你会看到"正在连接……"的过程，成功后打印出 IP 地址等信息。请记下这个 IP 地址，后续章节的 HTTP、MQTT 实验都会用到。

13.4 连接的健壮性

真实的物联网设备不能"连一次就完事"，因为网络随时可能中断（路由器重启、信号变差、断网等）。因此，成熟的联网程序必须处理**断线重连**。下面是一个健壮性更好的版本，把重连逻辑放进 `loop()`，实现自动重连：

```
// 带自动重连的 Wi-Fi 连接
#include <WiFi.h>
```

```
const char* ssid = "你的WiFi名称";
const char* password = "你的WiFi密码";

void connectWiFi() {
  if (WiFi.status() == WL_CONNECTED) return;  // 已连接则返回
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n已连接，IP: " + WiFi.localIP().toString());
}

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {   // 检测断线
    Serial.println("连接断开，尝试重连...");
    connectWiFi();
  }
  // 主循环继续做其他事……
  delay(1000);
}
```

这样，即使网络中途断开，设备也能自动恢复连接，大大提高可靠性。

13.5 常见问题排查

连接一直转圈（打印……）不成功：最常见的原因是 SSID 或密码写错。请仔细核对，尤其是大小写和空格；确认路由器为 2.4 GHz（经典 ESP32 不支持 5 GHz 频段）；确认路由器没有开启 MAC 过滤。

连接成功但拿不到 IP：可能是路由器 DHCP 地址池耗尽，或信号太弱。试着靠近路由器，或给设备设置静态 IP。

信号强度 RSSI：`WiFi.RSSI()` 返回的是负数，单位 dBm，越接近 0 信号越强。低于 −80 dBm 时连接可能不稳定。

13.6 本章小结

- SSID 是网络名，IP 地址是设备入网后的逻辑地址，MAC 是硬件地址；
- ESP32 支持 STA、AP、STA+AP 三种 Wi-Fi 模式；
- `WiFi.begin()` 发起连接，`WiFi.status()` 判断状态，`WiFi.localIP()` 获取 IP；
- 真实设备必须实现断线自动重连，保证长期稳定运行。

思考与练习

1. 把 Wi-Fi 的 SSID 和密码改成你自己的网络，成功连接并记录 IP 地址。
2. 在代码中加入 LED 指示：连接中闪烁、连接成功常亮。
3. 尝试把 ESP32 设为 AP 模式，并用手机连接它（查阅 `WiFi.softAP()` 用法）。

第14章 HTTP 通信实战

14.1 学习目标

连上了网络，接下来就要"说话"了。**HTTP**（HyperText Transfer Protocol，超文本传输协议）是互联网上应用最广泛的协议，网页浏览、API 调用、数据上报都离不开它。本章学习让 ESP32 通过 HTTP 与服务器交换数据，学完后你将能够：

- 理解 HTTP 请求与响应、GET 与 POST 方法；
- 使用 HTTPClient 库发送 GET/POST 请求；
- 解析常见的 JSON 数据；
- 完成一个获取天气或时间的小项目。

14.2 HTTP 基础

HTTP 采用**请求—响应**模型：客户端（如 ESP32）向服务器发送一个**请求（Request）**，服务器处理后返回一个**响应（Response）**。请求中最重要的信息包括：

- **方法（Method）**：最常见的是 **GET**（获取数据）和 **POST**（提交数据）；
- **URL**：目标资源的地址，如 `http://api.example.com/data`；
- **请求头（Headers）**：附加信息，如内容类型、认证信息等；
- **请求体（Body）**：POST 请求携带的数据。

响应中最重要的信息是**状态码（Status Code）**，如 200 表示成功、404 表示资源不存在、500 表示服务器错误，以及**响应体**——服务器返回的实际数据，常见格式为 JSON 或 HTML。

14.3 HTTPClient 库与 GET 请求

Arduino 框架的 **HTTPClient** 库让 ESP32 发送 HTTP 请求变得非常简单。下面是一个 GET 请求的完整示例（以获取一个公开 API 的数据为例）：

```
// HTTP GET 请求
#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid = "你的WiFi名称";
const char* password = "你的WiFi密码";
const char* url = "http://api.example.com/data";  // 替换为真实 API 地址
```

```
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n已连接");
}

void loop() {
  HTTPClient http;
  http.begin(url);                  // 准备请求
  int code = http.GET();            // 发送 GET 请求，返回状态码

  if (code > 0) {                   // 有响应
    Serial.print("状态码: ");
    Serial.println(code);
    String payload = http.getString();  // 获取响应体
    Serial.println(payload);
  } else {
    Serial.print("请求失败: ");
    Serial.println(http.errorToString(code));
  }
  http.end();                       // 释放连接
  delay(10000);                     // 每 10 秒请求一次
}
```

这个程序的流程清晰：先 `http.begin(url)` 准备请求，再 `http.GET()` 发送请求并拿到状态码，最后 `http.getString()` 读取响应内容。上传后，串口会打印服务器返回的数据。

> 【注意】 注意：示例中的 URL 是占位符，实际使用时请替换为真实可访问的 API。您可以先在浏览器中打开该 URL，确认能正常返回数据，再写入代码。

14.4 POST 请求

当需要向服务器提交数据（如上报传感器读数）时，就使用 POST 方法。下面演示用 POST 上报数据：

```
// HTTP POST 请求（提交数据）
#include <WiFi.h>
```

```
#include <HTTPClient.h>

const char* ssid = "你的WiFi名称";
const char* password = "你的WiFi密码";
const char* url = "http://api.example.com/report";  // 替换为真实地址

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("已连接");
}

void loop() {
  HTTPClient http;
  http.begin(url);
  http.addHeader("Content-Type", "application/json");  // 声明数据格式

  // 构造 JSON 请求体
  String body = "{\"temperature\": 26.5, \"humidity\": 60}";
  int code = http.POST(body);   // 发送 POST 请求

  Serial.print("状态码: ");
  Serial.println(code);
  if (code > 0) {
    Serial.println(http.getString());
  }
  http.end();
  delay(10000);
}
```

注意 POST 请求的两处不同：通过 `addHeader()` 声明了数据格式为 JSON，以及用 `http.POST(body)` 携带请求体发送。

14.5 解析 JSON 数据

服务器返回的数据大多是 **JSON 格式**。JSON（JavaScript Object Notation）是一种轻量级的数据交换格式，用键值对组织数据，形如 `{"temperature": 26.5, "city": "Beijing"}`。Arduino 框架下，常用 **ArduinoJson** 库来解析和生成 JSON。

安装库：搜索 "ArduinoJson" 安装。

```
// 解析 JSON 响应
#include <ArduinoJson.h>

void setup() {
  Serial.begin(115200);
  // 假设这是服务器返回的 JSON 字符串
  String json = "{\"temperature\": 26.5, \"humidity\": 60, \"city\":
\"Beijing\"}";

  DynamicJsonDocument doc(1024);      // 分配 JSON 缓冲区
  deserializeJson(doc, json);         // 解析 JSON

  float temp = doc["temperature"];    // 按字段名取值
  int hum = doc["humidity"];
  const char* city = doc["city"];

  Serial.print("温度: "); Serial.println(temp);
  Serial.print("湿度: "); Serial.println(hum);
  Serial.print("城市: "); Serial.println(city);
}

void loop() {
}
```

`deserializeJson()` 把 JSON 字符串解析成可访问的对象，随后用 `doc["字段名"]` 即可取出对应值。把 JSON 解析和 HTTP 请求结合起来，你就能从任何开放 API 获取结构化数据了。

14.6 本章小结

- HTTP 采用请求—响应模型，核心是 GET/POST 方法、URL、状态码和响应体；
- HTTPClient 库用 `begin/GET/POST/getString` 完成请求；
- JSON 是主流数据格式，ArduinoJson 库负责解析与生成；
- 把 HTTP + JSON 结合，即可接入海量开放 API。

思考与练习

1. 找一个公开的天气或时间 API，用 ESP32 获取数据并在串口打印。
2. 结合第 12 章的 DHT22，把温湿度数据通过 HTTP POST 上报到你自己的服务器或云平台。
3. 解释 HTTP 状态码 200、404、500 分别代表什么含义。

第15章 MQTT 物联网协议

15.1 学习目标

如果说 HTTP 是"人与网页"之间的协议，那么 **MQTT** 就是"机器与机器"之间的协议。在物联网场景下，设备数量庞大、带宽有限、常常需要双向实时通信，MQTT 正是为此量身定制的轻量级消息协议。本章学习 MQTT，学完后你将能够：

- 理解 MQTT 的发布/订阅模型与核心概念；
- 搭建或使用 MQTT 服务器（Broker）；
- 使用 PubSubClient 库实现消息收发；
- 完成设备数据上报与远程控制的完整闭环。

15.2 MQTT 核心概念

MQTT（Message Queuing Telemetry Transport，消息队列遥测传输）是一种基于**发布/订阅（Publish/Subscribe）**模型的轻量协议。它的核心思想是"解耦"——设备之间不直接通信，而是通过一个中介 **Broker（消息代理/服务器）** 来转发消息。

理解 MQTT，只需掌握这几个概念：

- **Broker（代理服务器）**：消息的中转站，负责接收、存储、转发消息。常见的开源 Broker 有 Mosquitto、EMQX 等，也有大量免费云服务。
- **Topic（主题）**：消息的分类标识，用 `/` 分隔的层级结构表示，如 `home/livingroom/temperature`。设备通过主题来组织消息。
- **Publish（发布）**：向某个主题发送消息；
- **Subscribe（订阅）**：订阅某个主题，接收该主题下的所有消息；
- **QoS（服务质量）**：消息传输的可靠性等级，分 0（至多一次）、1（至少一次）、2（恰好一次）三档。

一个典型场景：温度传感器向主题 `home/temp` **发布**温度数据，而手机 App、云端平台等都**订阅**了这个主题，于是它们都能实时收到温度数据；反过来，手机向 `home/light` 主题发布"开灯"指令，订阅了该主题的智能灯泡就会执行。**发布者与订阅者互不感知，全靠 Broker 撮合**——这就是 MQTT 的魅力。

15.3 搭建 MQTT 服务器

要使用 MQTT，首先需要一个 Broker。有几种选择：

- **本地搭建**：在电脑上安装 Mosquitto（开源、轻量），即可拥有自己的 Broker；

- **云服务**：使用 EMQX Cloud、阿里云 IoT、腾讯云 IoT 等提供的 MQTT 服务；
- **公共测试 Broker**：如 `broker.emqx.io`（1883 端口）等免费公共服务器，适合学习测试。

本书实验以公共测试服务器为例，实际项目建议使用自建或云 Broker。需要了解的连接参数有：**服务器地址（Host）**、**端口（Port，默认 1883）**，部分服务器还需要用户名和密码。

15.4 PubSubClient 库与发布订阅

Arduino 框架下，最常用的 MQTT 库是 **PubSubClient**。下面实现一个完整示例：ESP32 订阅 `esp32/led` 主题接收开灯/关灯指令，同时每 5 秒向 `esp32/temp` 主题发布一条模拟的温度数据。

安装库：搜索 "PubSubClient"（作者 Nick O'Leary）安装。

```
// MQTT 发布与订阅
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "你的WiFi名称";
const char* password = "你的WiFi密码";
const char* mqtt_server = "broker.emqx.io";  // 公共测试服务器

#define LED_PIN 5

WiFiClient espClient;
PubSubClient client(espClient);

// 收到订阅消息时的回调函数
void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("收到消息，主题 [");
  Serial.print(topic);
  Serial.print("]: ");
  String msg = "";
  for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];
  Serial.println(msg);

  if (msg == "ON") digitalWrite(LED_PIN, HIGH);   // 收到 ON 开灯
  else if (msg == "OFF") digitalWrite(LED_PIN, LOW); // 收到 OFF 关灯
}
```

```
void reconnect() {
  // 循环直到连接成功
  while (!client.connected()) {
    Serial.print("连接 MQTT...");
    if (client.connect("esp32_client_001")) {   // 客户端 ID 需唯一
      Serial.println("成功");
      client.subscribe("esp32/led");            // 订阅控制主题
    } else {
      Serial.print("失败，状态码=");
      Serial.print(client.state());
      delay(3000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("WiFi 已连接");

  client.setServer(mqtt_server, 1883);  // 设置 Broker 地址和端口
  client.setCallback(callback);         // 设置回调函数
}

void loop() {
  if (!client.connected()) reconnect();  // 断线重连
  client.loop();                         // 处理 MQTT 后台任务（必须循环调用）

  // 每 5 秒发布一次模拟温度
  static unsigned long last = 0;
  if (millis() - last > 5000) {
    last = millis();
    float temp = 20 + random(0, 100) / 10.0;  // 模拟 20~30°C 温度
    char buf[16];
    dtostrf(temp, 1, 1, buf);                 // 浮点转字符串
    client.publish("esp32/temp", buf);
    Serial.print("已发布温度: "); Serial.println(buf);
  }
}
```

上传后，ESP32 会连接 Broker 并订阅 `esp32/led`。你可以用手机上的 MQTT 客户端 App（如 MQTTX）或电脑上的工具，向 `esp32/led` 发布 `ON` 或 `OFF`，即可远程控制 LED；同时 ESP32 也在持续发布温度数据，订阅 `esp32/temp` 就能看到。

15.5 MQTT 编程要点

客户端 ID 必须唯一：多个设备连接同一 Broker 时，客户端 ID 不能重复，否则会相互挤下线。生产环境建议用 MAC 地址拼接随机数。

`client.loop()` 必须常驻循环：它是 PubSubClient 的心跳与消息处理入口，缺少它会导致连接保活失败、收不到消息。

断线重连：和 Wi-Fi 一样，MQTT 连接也可能中断，`reconnect()` 中应实现自动重连并重新订阅。

主题设计：建议用有意义的层级命名，如 `产品/设备ID/数据类别`，并约定消息格式（纯文本或 JSON），便于扩展和维护。

15.6 本章小结

- MQTT 是基于发布/订阅模型的轻量物联网协议，核心是 Broker、Topic、Publish、Subscribe、QoS；
- PubSubClient 库的 `setServer/setCallback/connect/subscribe/publish/loop` 是核心 API；
- 回调函数处理收到的消息，`client.loop()` 必须循环调用；
- 客户端 ID 唯一、断线重连、主题设计是工程实践的关键。

思考与练习

1. 用 MQTTX 等工具，分别扮演发布者和订阅者，验证 MQTT 的发布/订阅机制。
2. 给 ESP32 增加"定时上报心跳"功能，并让断线后能自动重连重订阅。
3. 设计一套家庭物联网的主题命名规范，并说明设计理由。

第16章 蓝牙 BLE 通信

16.1 学习目标

除了 Wi-Fi，ESP32 还集成了**蓝牙**，其中对物联网应用最重要的是 **BLE（Bluetooth Low Energy，低功耗蓝牙）**。BLE 以其极低的功耗和便捷的近场连接，成为智能手环、传感器标签、门锁等设备与手机通信的首选。本章学习 BLE，学完后你将能够：

- 理解 BLE 的核心概念（GATT、服务、特征）；
- 让 ESP32 作为 BLE 外设广播数据；
- 用手机 App 连接 ESP32 并读写数据；
- 理解 BLE 与经典蓝牙、与 Wi-Fi 的区别。

16.2 BLE 基础概念

BLE 与传统蓝牙（BR/EDR，经典蓝牙）最大的区别在于**低功耗**：它专为间歇性传输小数据量而设计，一颗纽扣电池就能工作数月甚至数年。BLE 采用客户端-服务器模型，通信围绕 **GATT（Generic Attribute Profile，通用属性规范）** 展开，其核心概念如下：

- **Profile（规范）**：一组约定的服务集合，定义了一类设备的"标准长相"；
- **Service（服务）**：功能的逻辑分组，如"电池服务""心率服务"，每个服务有一个唯一 UUID；
- **Characteristic（特征）**：服务下的具体数据项，如"电池电量值"，也有唯一 UUID，并具备读、写、通知等属性；
- **Descriptor（描述符）**：对特征的附加说明。

通俗地说，可以把 BLE 设备想象成一个"数据结构"：**服务是文件夹，特征是文件夹里的文件**，手机 App 通过读写这些"文件"来交换数据。ESP32 在 BLE 中通常扮演 **外设（Peripheral）** 角色——它广播自己，等待手机（中央设备，Central）来连接。

16.3 ESP32 BLE 外设示例

下面用 Arduino 框架实现一个 BLE 外设：创建一个自定义服务，包含一个可读写的特征，手机可以读取和修改它（例如控制 LED）。

```
// ESP32 BLE 外设：可读写特征控制 LED
#include <BLEDevice.h>
#include <BLEServer.h>
```

```
#include <BLEUtils.h>
#include <BLE2902.h>

#define SERVICE_UUID        "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"

#define LED_PIN 5

BLECharacteristic *pCharacteristic;
bool deviceConnected = false;

// 连接/断开状态回调
class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer* s) { deviceConnected = true; Serial.println("已连
接"); }
  void onDisconnect(BLEServer* s) { deviceConnected = false;
Serial.println("已断开"); }
};

// 特征写入回调：手机写入数据时触发
class MyCallbacks : public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic* c) {
    std::string value = c->getValue();  // 获取写入的数据
    if (value == "ON")  digitalWrite(LED_PIN, HIGH);
    if (value == "OFF") digitalWrite(LED_PIN, LOW);
    Serial.print("收到写入: "); Serial.println(value.c_str());
  }
};

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);

  BLEDevice::init("ESP32_LED");           // 设备名
  BLEServer* pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService* pService = pServer->createService(SERVICE_UUID);  // 创建服务
  pCharacteristic = pService->createCharacteristic(
      CHARACTERISTIC_UUID,
      BLECharacteristic::PROPERTY_READ |
BLECharacteristic::PROPERTY_WRITE);  // 可读可写
```

```
  pCharacteristic->setCallbacks(new MyCallbacks());
  pCharacteristic->setValue("OFF");       // 初始值
  pService->start();                      // 启动服务

  BLEAdvertising* pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->start();                  // 开始广播
  Serial.println("BLE 已就绪，等待连接...");
}

void loop() {
  if (!deviceConnected) {                 // 断线后重新广播，等待新连接
    BLEDevice::startAdvertising();
    delay(500);
  }
  delay(100);
}
```

上传后，打开手机上的 BLE 调试 App（如 nRF Connect），扫描设备，找到名为 "ESP32_LED" 的设备并连接，然后找到我们创建的服务和特征，向特征写入 "ON" 或 "OFF"，即可远程控制开发板上的 LED。这就是一个最精简的 BLE 控制链路。

16.4 BLE 的典型应用与注意点

典型应用：可穿戴设备数据同步、室内定位（iBeacon）、智能门锁、遥控、传感器节点、配网等。BLE 尤其适合**手机近场交互**和**超低功耗**场景。

编程注意点：

- 自定义服务的 UUID 应使用随机生成的 128 位 UUID，避免与标准服务冲突；
- BLE 连接断开后要及时重新广播，否则设备会"失联"；
- 特征属性要按需设置：只读数据用 READ，指令用 WRITE，主动上报用 NOTIFY；
- 内存有限，注意 BLE 缓冲区大小，避免一次写入过大数据。

16.5 Wi-Fi 与 BLE 如何选择

Wi-Fi 和 BLE 各有千秋，选型要点如下：

维度	Wi-Fi	BLE
通信距离	较远（数十米）	较近（数米到数十米）

维度	Wi-Fi	BLE
功耗	高	极低
速率	高（Mbps 级）	低（Kbps 级）
联网	可直接上互联网	需经手机/网关中转
典型场景	大数据量、远程监控	近场交互、穿戴设备

一句话总结：**要上互联网、传大量数据 → 用 Wi-Fi；要省电、与手机近场交互 → 用 BLE**。而 ESP32 同时具备二者，让你在一个方案里灵活取用。

16.6 本章小结

- BLE 以低功耗著称，核心概念是 GATT 的服务、特征、描述符；
- ESP32 通常作为外设广播，手机作为中央设备连接；
- BLE 控制 LED 示例演示了服务创建、特征读写回调的完整流程；
- Wi-Fi 适合联网大数据，BLE 适合省电近场交互，按需选择。

思考与练习

1. 用 nRF Connect 连接你的 ESP32，尝试读写特征，观察 LED 变化。
2. 在 BLE 示例中增加一个"通知（Notify）"特征，让 ESP32 定时主动向手机推送数据。
3. 结合生活场景，设计一个用 BLE 实现的智能设备，并画出它的服务/特征结构。

第17章 FreeRTOS 多任务

17.1 学习目标

前面我们用定时器和中断实现了"伪并行"，但真正的"同时干多件事"，还得靠**实时操作系统（RTOS）**。ESP32 官方开发框架底层就运行着 **FreeRTOS**，它让我们可以创建多个"看起来同时运行"的任务。本章学习 FreeRTOS 多任务编程，学完后你将能够：

- 理解任务、调度、优先级等 RTOS 核心概念；
- 创建和运行多个任务；
- 使用队列、互斥锁等任务间通信与同步机制；
- 体会到多任务编程带来的结构优势。

17.2 什么是实时操作系统

普通程序是"单线程"的：代码从头到尾顺序执行。而**实时操作系统（RTOS，Real-Time Operating System）** 提供了一种"多线程"的运行方式——它把一个程序拆分成多个**任务（Task）**，每个任务都有自己的代码和堆栈，调度器（Scheduler）会在极短的时间片内轮流让各个任务运行，从宏观上看，就好像所有任务在**同时**执行。

"实时"并不意味着"最快"，而是指**在规定时间内能确定性地响应**。对于需要及时响应的场景（如按键中断、传感器采样、通信超时），RTOS 能保证关键任务不被长时间阻塞。

FreeRTOS 的关键概念包括：

- **任务（Task）**：一段独立的执行单元，有优先级和堆栈；
- **调度器（Scheduler）**：按优先级决定哪个任务获得 CPU；
- **队列（Queue）**：任务间传递数据的"管道"，先进先出；
- **互斥锁（Mutex）/信号量（Semaphore）**：保护共享资源、同步任务的机制；
- **时间片（Time Slice）**：同优先级任务轮流执行的单位时间。

17.3 创建多个任务

Arduino 框架默认已经把 `setup()` 和 `loop()` 跑在了一个 FreeRTOS 任务里，我们可以在其上再创建自己的任务。下面是一个经典的"三任务并行"示例：三个 LED 以不同频率独立闪烁。

```
// FreeRTOS 多任务：三颗 LED 独立闪烁
#define LED1 5
```

```
#define LED2 18
#define LED3 19

// 任务函数（每个任务是一个无限循环）
void taskBlink(void* param) {
  int pin = (int)param;          // 通过参数传入引脚号
  pinMode(pin, OUTPUT);
  while (1) {                    // 任务必须无限循环
    digitalWrite(pin, HIGH);
    vTaskDelay(200 / portTICK_PERIOD_MS);  // 阻塞延时
    digitalWrite(pin, LOW);
    vTaskDelay(200 / portTICK_PERIOD_MS);
  }
}

void setup() {
  // 创建三个任务，各自不同频率
  xTaskCreate(taskBlink, "LED1", 2048, (void*)LED1, 1, NULL);
  xTaskCreate(taskBlink, "LED2", 2048, (void*)LED2, 1, NULL);
  xTaskCreate(taskBlink, "LED3", 2048, (void*)LED3, 1, NULL);
}

void loop() {
  // 主循环可以空置，任务独立运行
}
```

关键 API 说明：

- `xTaskCreate(函数, 名称, 堆栈大小, 参数, 优先级, 句柄)`：创建任务。**堆栈大小**以字（word）为单位，2048 是常用值，任务复杂时需增大；**优先级**数值越大越优先。
- `vTaskDelay(ticks)`：让任务**阻塞**（让出 CPU）指定的时间片，与 `delay()` 不同，它不会浪费 CPU，其他任务可趁机运行。`portTICK_PERIOD_MS` 是每个时间片对应的毫秒数。

上传后，三颗 LED 各自独立闪烁，互不干扰。这就是多任务的魅力——每个任务只关心自己的事，代码清晰、互不阻塞。

17.4 任务间通信：队列

任务之间经常需要传递数据。例如，一个任务负责采集传感器，另一个任务负责把数据上报网络。**队列（Queue）** 就是任务间传递数据的标准方式，它像一个"管道"，一个任务往里放数据，另一个任务往外取。

```
// 使用队列在任务间传递数据
#include <WiFi.h>

QueueHandle_t queue;   // 队列句柄

// 生产者任务：采集数据放入队列
void taskProducer(void* param) {
  int counter = 0;
  while (1) {
    counter++;
    xQueueSend(queue, &counter, portMAX_DELAY);  // 发送数据
    vTaskDelay(1000 / portTICK_PERIOD_MS);
  }
}

// 消费者任务：从队列取出数据
void taskConsumer(void* param) {
  int data;
  while (1) {
    if (xQueueReceive(queue, &data, portMAX_DELAY) == pdTRUE) {  // 接收
      Serial.print("收到数据: ");
      Serial.println(data);
    }
  }
}

void setup() {
  Serial.begin(115200);
  queue = xQueueCreate(10, sizeof(int));   // 创建队列，容量 10，元素为 int
  xTaskCreate(taskProducer, "Producer", 2048, NULL, 1, NULL);
  xTaskCreate(taskConsumer, "Consumer", 2048, NULL, 1, NULL);
}

void loop() {
}
```

这个"生产者—消费者"模型是嵌入式系统的经典结构：生产者负责采集，消费者负责处理，二者通过队列解耦，速率不匹配时队列还能起到缓冲作用。

17.5 共享资源与互斥锁

当多个任务同时访问同一个资源（如串口、全局变量、I2C 总线）时，就可能发生冲突（竞态条件）。例如两个任务同时向串口打印，输出会乱成一团。此时需要用**互斥锁（Mutex）**来保护共享资源：任务在访问资源前"上锁"，访问完"解锁"，同一时刻只允许一个任务持有锁。

```
// 互斥锁保护串口输出
SemaphoreHandle_t mutex;

void taskA(void* param) {
  while (1) {
    xSemaphoreTake(mutex, portMAX_DELAY);   // 上锁
    Serial.println("AAAAAAAAAAA");
    xSemaphoreGive(mutex);                  // 解锁
    vTaskDelay(100 / portTICK_PERIOD_MS);
  }
}

void taskB(void* param) {
  while (1) {
    xSemaphoreTake(mutex, portMAX_DELAY);
    Serial.println("BBBBBBBBBBB");
    xSemaphoreGive(mutex);
    vTaskDelay(100 / portTICK_PERIOD_MS);
  }
}

void setup() {
  Serial.begin(115200);
  mutex = xSemaphoreCreateMutex();  // 创建互斥锁
  xTaskCreate(taskA, "A", 2048, NULL, 1, NULL);
  xTaskCreate(taskB, "B", 2048, NULL, 1, NULL);
}

void loop() {
}
```

上传后，串口里 A 和 B 的输出成串出现、不会交错——这就是互斥锁的效果。没有锁时，你会看到类似 "AAAAABBBBBAAAAA" 之外的乱序交错。

17.6 本章小结

- RTOS 通过任务和调度器实现"伪并行"，FreeRTOS 是 ESP32 的底层操作系统；
- `xTaskCreate()` 创建任务，`vTaskDelay()` 实现非阻塞延时；
- 队列（Queue）用于任务间传递数据，是生产者—消费者模型的基石；
- 互斥锁（Mutex）保护共享资源，避免竞态条件。

思考与练习

1. 修改三 LED 示例，让三颗 LED 以 200 ms、500 ms、1000 ms 三种不同周期闪烁。
2. 用队列实现"按键计数"：按键任务检测按键并发送事件，显示任务接收并打印。
3. 说说 `vTaskDelay()` 和 `delay()` 的本质区别，为什么 RTOS 中推荐前者。

第18章 文件系统与数据存储

18.1 学习目标

物联网设备常常需要把数据"记下来"：传感器历史数据、配置文件、Wi-Fi 密码、离线日志等。ESP32 的 Flash 里可以划分出一块区域作为**文件系统**，像电脑硬盘一样存放文件。本章学习文件系统的使用，学完后你将能够：

- 理解 SPIFFS 与 LittleFS 两种文件系统；
- 在文件系统上读写文件；
- 保存和读取配置数据（如 Wi-Fi 密码）；
- 把文件系统与数据采集结合，实现本地数据记录。

18.2 SPIFFS 与 LittleFS

ESP32 的片外 Flash 除了存放程序代码，还可以划出一部分空间作为文件系统使用。Arduino ESP32 核心支持两种文件系统：

- **SPIFFS**：早期的文件系统，简单易用，但不支持目录层级、断电后一致性较差；
- **LittleFS**：新一代文件系统，支持目录、掉电保护更好，是当前推荐的选择。

本书使用 **LittleFS**。在 Arduino IDE 中使用前，需要先在"工具 → Partition Scheme（分区方案）"中选择一个带文件系统分区的方案（如 "Default 4MB with spiffs" 或带 LittleFS 的方案），再在"工具"菜单里使用 LittleFS 上传工具把数据文件上传到 Flash（这一步用于上传初始文件，运行时读写则直接用代码即可）。

18.3 读写文件

LittleFS 的 API 与电脑上的文件操作非常相似。下面的示例演示了写入、读取和追加文件：

```
// LittleFS 文件读写
#include <LittleFS.h>

void setup() {
  Serial.begin(115200);

  if (!LittleFS.begin()) {           // 挂载文件系统
    Serial.println("LittleFS 挂载失败");
    return;
  }
  Serial.println("LittleFS 就绪");
```

```
  // 写入文件
  File f = LittleFS.open("/data.txt", "w");  // 以写模式打开
  if (f) {
    f.println("第一行数据");
    f.println("第二行数据");
    f.close();
    Serial.println("写入完成");
  }

  // 读取文件
  f = LittleFS.open("/data.txt", "r");       // 以读模式打开
  if (f) {
    Serial.println("文件内容: ");
    while (f.available()) {
      Serial.write(f.read());
    }
    f.close();
  }

  // 追加内容
  f = LittleFS.open("/data.txt", "a");       // 以追加模式打开
  if (f) {
    f.println("追加的第三行");
    f.close();
  }
}

void loop() {
}
```

这段代码覆盖了文件操作的三种基本模式：写（`"w"`，覆盖）、读（`"r"`）、追加（`"a"`，在末尾添加）。上传后，串口会打印出写入和读取的内容。

18.4 应用：保存 Wi-Fi 配置

文件系统最实用的场景之一，是**保存用户配置**——比如让设备记住上次连接的 Wi-Fi，下次上电自动重连。下面演示配置的保存与读取：

```
// 使用 LittleFS 保存和读取配置
#include <LittleFS.h>
#include <ArduinoJson.h>
```

```
void saveConfig(const char* ssid, const char* pass) {
  File f = LittleFS.open("/config.json", "w");
  if (!f) return;
  DynamicJsonDocument doc(256);
  doc["ssid"] = ssid;
  doc["pass"] = pass;
  serializeJson(doc, f);   // 直接把 JSON 序列化写入文件
  f.close();
  Serial.println("配置已保存");
}

void loadConfig() {
  File f = LittleFS.open("/config.json", "r");
  if (!f) {
    Serial.println("无配置文件");
    return;
  }
  DynamicJsonDocument doc(256);
  deserializeJson(doc, f);   // 从文件解析 JSON
  f.close();
  Serial.print("读取到 SSID: ");
  Serial.println(doc["ssid"].as<const char*>());
}

void setup() {
  Serial.begin(115200);
  LittleFS.begin();
  saveConfig("my_wifi", "12345678");  // 演示保存
  loadConfig();                       // 演示读取
}

void loop() {
}
```

这个模式非常实用：把配置以 JSON 形式存进文件，配合配网流程，就能做出"首次配置、后续自动连接"的智能设备。把数据记录到文件再配合时间戳，还能实现简单的本地数据日志功能。

18.5 本章小结

- LittleFS 是 ESP32 推荐的现代文件系统，支持目录和掉电保护；
- `LittleFS.open(path, mode)` 支持 `"w"` 写、`"r"` 读、`"a"` 追加三种模式；

· 文件系统常用于保存配置、记录日志、存储离线数据；
· 结合 ArduinoJson 可以方便地以 JSON 格式持久化配置。

思考与练习

1. 用 LittleFS 实现一个"计数器"，每次上电读上次的值并加一保存。
2. 把第 12 章的 DHT22 读数以 CSV 格式记录到文件，实现简易数据记录仪。
3. 对比 SPIFFS 与 LittleFS 的区别，说明为什么推荐 LittleFS。

第19章 低功耗设计

19.1 学习目标

许多物联网设备靠电池供电，功耗直接决定了设备的续航时间。ESP32 虽然功能强大，但"功耗大"也是它的短板之一。本章学习 ESP32 的低功耗技术，学完后你将能够：

- 理解 ESP32 的功耗来源与省电思路；
- 使用浅睡、深度睡眠等睡眠模式；
- 设计"定时唤醒采集上报"的低功耗方案；
- 了解进一步降低功耗的实用技巧。

19.2 功耗从哪来

ESP32 的功耗主要来自几个方面：**CPU 运行**（频率越高越耗电）、**Wi-Fi/蓝牙收发**（无线通信是耗电大户）、以及**外围器件**（传感器、LED 等）。经典款 ESP32 在正常运行 + Wi-Fi 连接时，电流可达 100～200 mA 以上，而深度睡眠时却能降到 **10 μA 以下**——差距达上万倍。因此，低功耗设计的核心思想就是：**尽可能让设备"睡觉"，只在必要时短暂"醒来"干活**。

19.3 ESP32 的睡眠模式

ESP32 提供多种睡眠模式，功耗与唤醒能力各不相同：

睡眠模式	大致电流	可唤醒方式	特点
Modem Sleep（调制解调器睡眠）	数十 mA	定时/无线	仅关闭无线，CPU 保持运行
Light Sleep（浅睡）	约 0.8 mA	定时器、GPIO、触摸	CPU 暂停，RAM 保持，唤醒快
Deep Sleep（深睡）	约 5～10 μA	定时器、GPIO、触摸	大部分断电，RAM 丢失，唤醒最慢

其中 **Deep Sleep（深度睡眠）** 的省电效果最显著，是电池供电设备的主力模式。它的代价是：唤醒后程序会**重新从头开始执行**（RAM 内容丢失），需要把状态持久化到 **RTC 内存** 或文件系统中。

19.4 定时唤醒采集上报

下面实现一个经典的低功耗数据采集方案：设备深度睡眠 60 秒 → 唤醒后连接 Wi-Fi 上报一次数据 → 再次进入深度睡眠，如此循环。这样设备大部分时间都在"睡觉"，续航大大延长。

```
// 深度睡眠 + 定时唤醒
#include <WiFi.h>

#define uS_TO_S_FACTOR 1000000ULL  // 微秒转秒的系数
#define TIME_TO_SLEEP 60           // 睡眠时长（秒）

void setup() {
  Serial.begin(115200);

  // 模拟采集与上报（实际项目中替换为真实的传感器读取和网络上报）
  Serial.println("唤醒，执行一次采集上报...");
  // ... 读取传感器、连接 Wi-Fi、上报数据 ...

  Serial.println("进入深度睡眠 60 秒");
  esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_S_FACTOR);  // 设置定时唤醒
  esp_deep_sleep_start();   // 进入深度睡眠（程序到此暂停，唤醒后重新从 setup 执行）
}

void loop() {
}
```

核心 API：

- `esp_sleep_enable_timer_wakeup(微秒)`：设置定时唤醒的时间；
- `esp_deep_sleep_start()`：立即进入深度睡眠。

上传并运行后，设备会每 60 秒"醒来"打印一次，其余时间处于极低功耗的深睡状态。你可以用万用表（或 USB 电流表）测一下深睡时的电流，感受省电效果的巨大差距。

19.5 其他省电技巧

除了睡眠模式，还有不少实用的省电手段：

- **关闭 Wi-Fi**：不需要联网时用 `WiFi.disconnect(true)` 并 `WiFi.mode(WIFI_OFF)` 彻底关闭无线，可省大量电流；

- **降低 CPU 频率**：`setCpuFrequencyMhz(80)` 把主频从 240 MHz 降到 80 MHz，省电且对多数任务够用；
- **降低 Flash 频率**、关闭不必要的 LED；
- **选用低功耗硬件**：选择更省电的传感器，用 MOS 管控制传感器供电，采样时才通电；
- **选择合适的芯片**：极致省电场景可选用 ESP32-S2/S3/C3 等功耗更优的衍生型号。

19.6 本章小结

- ESP32 功耗大头是 CPU 和无线，深度睡眠可降到 μA 级；
- 深度睡眠唤醒后程序从头执行，状态需持久化；
- "定时唤醒—采集—上报—再睡"是电池设备的标准省电范式；
- 关无线、降频、按需供电都是有效的省电手段。

思考与练习

1. 用 USB 电流表（或万用表）测量并对比 ESP32 正常运行与深度睡眠时的电流。
2. 实现一个"每 10 分钟唤醒采集温度并保存到文件"的低功耗记录仪。
3. 查阅资料，了解 ESP32 的 RTC 内存如何在深度睡眠中保存数据。

第四部分 精通篇

第20章 OTA 远程升级

20.1 学习目标

想象你的产品已经部署到了千里之外的客户现场，却突然发现了一个 Bug——难道要跑过去拆开设备用数据线重新烧录吗？当然不用。**OTA（Over-The-Air，空中升级）** 技术让你可以通过网络远程更新设备的固件。本章学习 OTA，学完后你将能够：

- 理解 OTA 的原理与意义；
- 使用 ArduinoOTA 库实现局域网内的固件升级；
- 了解基于 HTTP 的远程 OTA 方案；
- 掌握 OTA 的工程实践与注意事项。

20.2 OTA 原理

ESP32 的 Flash 采用**双分区（OTA 分区）** 机制：Flash 里通常有两个 App 分区（`app0` 和 `app1`），设备运行其中一个分区里的固件，OTA 时把新固件下载并写入另一个分区，校验通过后再切换启动分区，实现"无缝"升级。如果升级失败，还可以回退到旧分区继续运行，安全可靠。

OTA 的流程大致是：设备连接到网络 → 从服务器（或局域网主机）下载新固件 → 校验固件完整性 → 写入备用分区 → 重启并切换到新分区。整个过程无需物理接触设备，极大降低了运维成本，是物联网产品**必须具备**的能力。

20.3 ArduinoOTA 局域网升级

Arduino 框架提供了 **ArduinoOTA** 库，让设备可以在局域网内被 IDE 直接"无线烧录"。这对开发调试非常方便——不用反复拔插数据线。使用时只需在程序中加入 OTA 初始化：

```
// ArduinoOTA 局域网无线升级
#include <WiFi.h>
#include <ArduinoOTA.h>

const char* ssid = "你的WiFi名称";
const char* password = "你的WiFi密码";

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
```

```
  while (WiFi.waitForConnectResult() != WL_CONNECTED) {
    Serial.println("连接失败，重试...");
    delay(3000);
  }

  // 配置 OTA
  ArduinoOTA.setHostname("esp32-ota");       // 设备名
  ArduinoOTA.setPassword("admin123");        // 可选：OTA 密码
  ArduinoOTA.onStart([]() { Serial.println("OTA 开始"); });
  ArduinoOTA.onEnd([]() { Serial.println("OTA 完成"); });
  ArduinoOTA.onProgress([](unsigned int p, unsigned int t) {
    Serial.printf("进度：%u%%\r", p / (t / 100));
  });
  ArduinoOTA.onError([](ota_error_t e) {
    Serial.printf("OTA 错误[%u]\n", e);
  });
  ArduinoOTA.begin();

  Serial.print("OTA 就绪，IP: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  ArduinoOTA.handle();   // 必须在主循环中处理 OTA 请求
  delay(10);
}
```

上传这段程序后（第一次仍需用数据线），重新打开 Arduino IDE 的"工具 → 端口"，你会发现多了一个**网络端口**（如 "esp32-ota at 192.168.x.x"）。选择它，之后就可以像用数据线一样直接"上传"程序了——固件将通过 Wi-Fi 无线烧录，Arduino IDE 右上角还会显示上传进度。注意：**IDE 里要选择"网络端口"，而不是原来的 COM 口**。

20.4 远程 OTA（HTTP 方式）

局域网 OTA 适合开发调试，而真正的产品需要从互联网远程升级。远程 OTA 常用 **HTTP 下载固件** 的方式：设备定时或按指令，从服务器下载固件文件并写入备用分区。ESP32 的 `Update` 库提供了底层支持。核心代码如下：

```
// HTTP 远程 OTA（核心片段）
#include <HTTPClient.h>
#include <Update.h>
```

```
void otaUpdate(String url) {
  HTTPClient http;
  http.begin(url);
  int code = http.GET();
  if (code != 200) { Serial.println("下载失败"); return; }

  int len = http.getSize();
  bool canBegin = Update.begin(len);   // 开始更新，传入固件大小

  if (canBegin) {
    WiFiClient* stream = http.getStreamPtr();
    size_t written = Update.writeStream(*stream);  // 流式写入固件
    if (written == len && Update.end()) {          // 校验并结束
      Serial.println("OTA 成功，即将重启");
      ESP.restart();                               // 重启切换新固件
    } else {
      Serial.println("OTA 写入失败");
    }
  }
  http.end();
}
```

要点在于：先 `Update.begin(len)` 声明固件大小，再用 `Update.writeStream()` 流式写入（避免一次性占用大量内存），最后 `Update.end()` 校验完整性并切换分区，重启生效。生产环境中，还应在下载前校验固件的**签名/校验和**，防止固件被篡改或损坏（详见第 25 章安全相关内容）。

20.5 OTA 工程实践

- **务必保留回滚机制**：确保升级失败时能回退旧固件，避免设备"变砖"；
- **断点续传与重试**：网络不稳时下载可能中断，应支持重试；
- **版本管理**：设备上报当前版本号，服务器按版本分发合适的升级包；
- **分批灰度**：先小范围升级验证，再全量推送，降低风险；
- **校验固件**：升级前校验校验和/签名，防止损坏或恶意固件。

20.6 本章小结

- OTA 通过双分区机制实现远程固件升级，支持失败回滚；
- ArduinoOTA 库让局域网内无线烧录变得简单，开发调试利器；
- 远程 OTA 常用 HTTP 下载 + Update 库流式写入；

· 回滚、重试、版本管理、灰度、校验是工程化的关键。

思考与练习

1. 用 ArduinoOTA 完成一次局域网无线升级，体验网络端口烧录。
2. 给 OTA 程序加入"下载失败自动重试 3 次"的逻辑。
3. 设计一套"版本号 + 灰度升级"的方案，画出升级流程图。

第21章 ESP-IDF 深入解析

21.1 学习目标

前面我们一直使用 Arduino 框架，它简单易用，但屏蔽了大量底层细节。现在，让我们揭开 ESP32 的"引擎盖"，认识官方原生开发框架 **ESP-IDF**。学完本章，你将能够：

- 理解 ESP-IDF 的定位与架构；
- 搭建 ESP-IDF 开发环境；
- 用 ESP-IDF 编写并编译一个基础工程；
- 理解组件、Kconfig、构建系统等核心概念，为专业开发打下基础。

21.2 什么是 ESP-IDF

ESP-IDF（Espressif IoT Development Framework）是乐鑫官方推出的、基于 FreeRTOS 的完整开发框架。它不仅是 Arduino ESP32 核心的"底层"，更是工业级、量产级产品的标准开发方式。ESP-IDF 提供了：

- 完整的底层 API（Wi-Fi、蓝牙、外设、存储、加密等），控制粒度细、性能最优；
- 基于 **CMake** 的现代构建系统和 **组件（Component）** 化管理；
- 丰富的配置系统（menuconfig/Kconfig），可精细裁剪功能和资源；
- 配套的调试、监控、分析工具链（`idf.py` 命令行工具）。

如果说 Arduino 是"自动挡"，那么 ESP-IDF 就是"手动挡"——上手更难，但能完全掌控车辆的每一分动力。

21.3 搭建 ESP-IDF 环境

ESP-IDF 的环境搭建比 Arduino 复杂一些，主要有三种方式：

- **VS Code + ESP-IDF 扩展**：官方推荐的图形化方式，安装扩展后一键安装工具链，适合大多数开发者；
- **命令行安装**：克隆 ESP-IDF 仓库，运行 `install.sh`/`install.bat` 安装工具链，适合 Linux/macOS 用户；
- **Docker 容器**：官方提供镜像，环境隔离、开箱即用。

本书以命令行方式为例，说明 Linux/macOS 下的安装思路（Windows 用户建议使用 VS Code 扩展）：

```
mkdir -p ~/esp
cd ~/esp
git clone --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
git checkout v5.1        # 切换到稳定版本
./install.sh esp32       # 安装 esp32 目标的工具链
source export.sh         # 激活环境（每次打开终端都要执行）
```

安装完成后，`source export.sh` 会设置好 `idf.py` 等工具的环境变量。验证安装成功：运行 `idf.py --version`，能打印出版本号即表示就绪。

21.4 第一个 ESP-IDF 工程

我们创建一个最小工程，点亮 LED 并通过日志输出信息。

```
# 复制官方示例作为起点
cp -r $IDF_PATH/examples/get-started/blink ~/esp/my_project
cd ~/esp/my_project
idf.py set-target esp32     # 设置目标芯片
idf.py menuconfig           # 可选：图形化配置
idf.py build                # 编译
idf.py -p /dev/ttyUSB0 flash monitor  # 烧录并打开串口监控
```

`main/blink.c` 的典型内容如下：

```
// ESP-IDF 点灯示例（main/blink.c）
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"

#define LED_GPIO 5
static const char *TAG = "blink";

void app_main(void) {
    gpio_reset_pin(LED_GPIO);
    gpio_set_direction(LED_GPIO, GPIO_MODE_OUTPUT);

    while (1) {
        gpio_set_level(LED_GPIO, 1);
        ESP_LOGI(TAG, "LED ON");      // 打印日志
        vTaskDelay(1000 / portTICK_PERIOD_MS);
        gpio_set_level(LED_GPIO, 0);
```

```
        ESP_LOGI(TAG, "LED OFF");
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}
```

与 Arduino 相比，ESP-IDF 的入口是 `app_main()` 函数（替代了 `setup`/`loop`），并使用 `driver/gpio.h` 的 API 操作引脚，用 `ESP_LOGI` 打印带标签、分级的日志。烧录并打开监视器后，你会看到 LED 闪烁，同时终端输出带时间戳和日志级别的 `LED ON`/`LED OFF` 信息。

21.5 组件化与构建系统

ESP-IDF 工程最核心的概念是**组件（Component）**。一个组件是一个自包含的模块，包含源码、头文件和 `CMakeLists.txt` 构建脚本。工程由多个组件组成，`main` 也是一个组件。组件之间通过依赖关系组织，实现了高度的模块化与复用。

每个组件还可以通过 **Kconfig** 定义可配置选项。运行 `idf.py menuconfig` 时，就能在图形界面里调整这些选项（如引脚号、波特率、功能开关），配置结果保存在 `sdkconfig` 文件中，编译时生效。这种"配置驱动"的设计，让同一个代码库能灵活适配不同硬件和需求，是 ESP-IDF 工程化的精髓。

21.6 本章小结

- ESP-IDF 是乐鑫官方框架，基于 FreeRTOS 和 CMake，是量产产品的标准选择；
- 环境搭建可用 VS Code 扩展、命令行或 Docker；
- 入口是 `app_main()`，用 `idf.py` 完成配置、编译、烧录、监控全流程；
- 组件化 + Kconfig 配置系统是 ESP-IDF 工程化的核心。

思考与练习

1. 在你的电脑上完成 ESP-IDF 环境搭建，并编译运行 blink 示例。
2. 用 `idf.py menuconfig` 修改 blink 示例的引脚配置，观察 sdkconfig 的变化。
3. 对比 Arduino 和 ESP-IDF 的 LED 闪烁程序，说说二者的主要差异。

第22章 内存管理与优化

22.1 学习目标

嵌入式系统的资源是极其有限的。ESP32 的 SRAM 只有约 320 KB（可用部分），与动辄数 GB 的电脑内存相比微不足道。在这样紧张的内存里跑 Wi-Fi、BLE、多任务，稍有不慎就会"内存不足"甚至崩溃。本章学习 ESP32 的内存管理，学完后你将能够：

- 理解 ESP32 的内存结构与各类内存的用途；
- 掌握内存监控与排查手段；
- 识别并避免常见的内存问题（内存泄漏、堆碎片、栈溢出）；
- 运用优化技巧让程序稳定运行。

22.2 ESP32 的内存结构

ESP32 的内存分为几个层次，用途各不相同：

- **SRAM**：芯片内置的静态 RAM，共 520 KB，是程序运行的主要内存；
- **IRAM**：SRAM 中用于存放"指令"的部分，中断服务函数等对时序敏感、需要快速执行的代码会放进 IRAM（对应 `IRAM_ATTR`）；
- **DRAM**：SRAM 中用于存放"数据"的部分，普通变量、堆栈都在这里；
- **PSRAM**：部分模组（如 WROVER）外扩的伪静态 RAM，容量可达数 MB，适合存放图片、音频等大数据；
- **Flash**：片外非易失存储，存放代码和文件系统，读取速度比 RAM 慢。

理解这些内存的区分，是理解 ESP32 内存问题的前提。例如，为什么中断函数要加 `IRAM_ATTR`？因为默认代码存放在 Flash 里，而 Flash 读取有延迟，中断响应要求极快，所以要把中断代码常驻在 IRAM 里。

22.3 内存监控

要优化内存，先要"看得见"内存。Arduino 框架提供了几个常用函数：

```
void setup() {
  Serial.begin(115200);
  Serial.print("总堆内存: ");
  Serial.println(ESP.getHeapSize());          // 堆总大小
  Serial.print("可用堆内存: ");
  Serial.println(ESP.getFreeHeap());          // 当前可用堆
  Serial.print("最小可用堆(历史): ");
```

```
  Serial.println(ESP.getMinFreeHeap());       // 运行以来的最小可用值
  Serial.print("PSRAM 大小: ");
  Serial.println(ESP.getPsramSize());         // PSRAM（若支持）
  Serial.print("可用 PSRAM: ");
  Serial.println(ESP.getFreePsram());
}

void loop() {
  // 定期打印可用内存，观察内存变化
  Serial.printf("可用堆: %d\n", ESP.getFreeHeap());
  delay(5000);
}
```

其中 **`ESP.getMinFreeHeap()`** 尤为重要——它记录了程序运行以来堆内存的历史最低点，是判断内存是否"危险"的重要指标。如果这个值持续走低，说明程序存在内存问题。上传后观察这些数值，尤其注意在反复执行某些操作（如连接 Wi-Fi、发送请求）后，可用堆是否不断减少。

22.4 常见内存问题

内存泄漏（Memory Leak）：程序反复分配内存却不释放，可用内存逐渐减少，最终耗尽崩溃。常见原因是用 `malloc`/`new` 分配内存后忘了 `free`/`delete`，或某些库反复创建对象未释放。排查方法：在循环中反复执行可疑操作，观察 `getFreeHeap()` 是否持续下降。

堆碎片（Heap Fragmentation）：反复分配和释放不同大小的内存块，导致堆里出现许多"碎片"，虽然总空闲够，但找不到一块连续的大内存。对策：减少频繁的小块分配释放、使用固定大小的内存池。

栈溢出（Stack Overflow）：每个 FreeRTOS 任务都有独立且大小固定的栈，如果任务里定义了过大的局部数组、或递归过深，就会栈溢出，导致程序崩溃复位。对策：增大任务堆栈、避免在栈上分配大数组（改用堆或 static）。

22.5 优化技巧

- **用 `static` 或全局变量替代反复的动态分配**：避免频繁 malloc/free 造成的碎片；
- **大数组放 PSRAM**（若支持）：用 `ps_malloc()` 把大数据分配到 PSRAM，减轻内部 SRAM 压力；
- **合理设置任务堆栈**：`xTaskCreate` 时按需分配堆栈，既不浪费也不不足；
- **用 `const` 修饰常量数据**：让编译器和链接器把常量放进 Flash 而非 RAM；

- **及时释放不用的库资源**：如 HTTP 客户端用完 `http.end()`、文件用完 `f.close()`；
- **监控 `getMinFreeHeap()`**：把它作为产品测试的必查指标，提前发现隐患。

22.6 本章小结

- ESP32 内存分 SRAM（IRAM/DRAM）、PSRAM、Flash 等层次，用途各异；
- `ESP.getFreeHeap()`、`ESP.getMinFreeHeap()` 是内存监控的核心工具；
- 内存泄漏、堆碎片、栈溢出是三大常见问题，各有排查与对策；
- 减少动态分配、善用 PSRAM、合理栈大小、及时释放资源是优化要诀。

思考与练习

1. 写一个反复 `malloc` 不释放的程序，观察 `getFreeHeap()` 的下降趋势。
2. 在任务里定义一个 10 KB 的局部数组，观察是否触发栈溢出，并给出修复方案。
3. 查阅资料，说明 `IRAM_ATTR` 的作用以及为什么中断函数需要它。

第23章 底层驱动开发

23.1 学习目标

到目前为止，我们大多在使用现成的库。但高级工程师的能力，体现在**没有现成库时，能自己写出驱动**。本章将带你把驱动开发的"黑盒"打开，学完后你将能够：

- 理解驱动开发的基本思路（读时序图 → 写寄存器/时序）；
- 手动实现 I2C 器件和单总线器件的底层读写；
- 理解寄存器操作与位运算；
- 掌握编写健壮驱动的工程方法。

23.2 驱动开发的基本思路

一个外设的驱动，本质上是"**用芯片的引脚和总线，按数据手册规定的时序，与器件交换数据**"。因此，驱动开发的第一步永远是**精读数据手册（Datasheet）**，尤其是其中的：

- **引脚定义**：每个引脚的功能、电源、地址引脚；
- **寄存器表**：器件的内部寄存器地址、位含义、默认值（适用于 I2C/SPI 器件）；
- **时序图**：通信的时序要求（适用于单总线、自定义协议）；
- **读写流程**：要读取某个数据，需要先写哪些命令、再读哪些字节。

掌握了这些，再加上对总线底层时序的掌控，就能写出任何器件的驱动。下面以两个经典例子说明。

23.3 手动读写 I2C 寄存器

很多 I2C 传感器的数据都藏在寄存器里，读寄存器是驱动的基础操作。以 BMP280 这类器件为例，其寄存器读写模式是通用的：写寄存器时，先发"寄存器地址"再发"数据"；读寄存器时，先发"寄存器地址"，再发起重复起始并读取数据。下面是基于 Wire 库的底层寄存器读写函数：

```
// 手动读写 I2C 寄存器
#include <Wire.h>

#define DEV_ADDR 0x76   // 器件地址

// 写一个字节到寄存器
void writeReg(uint8_t reg, uint8_t value) {
  Wire.beginTransmission(DEV_ADDR);
```

```
  Wire.write(reg);       // 寄存器地址
  Wire.write(value);     // 数据
  Wire.endTransmission();
}

// 从寄存器读一个字节
uint8_t readReg(uint8_t reg) {
  Wire.beginTransmission(DEV_ADDR);
  Wire.write(reg);                  // 先写要读的寄存器地址
  Wire.endTransmission(false);      // 不产生停止位
  Wire.requestFrom(DEV_ADDR, 1);    // 请求读 1 字节
  return Wire.read();
}

// 从寄存器连续读多个字节
void readRegs(uint8_t reg, uint8_t* buf, uint8_t len) {
  Wire.beginTransmission(DEV_ADDR);
  Wire.write(reg);
  Wire.endTransmission(false);
  Wire.requestFrom(DEV_ADDR, len);
  for (uint8_t i = 0; i < len; i++) buf[i] = Wire.read();
}
```

有了这两个函数，再加上数据手册里的寄存器表，就能读写任何 I2C 器件。例如，读 BMP280 的 ID 寄存器（地址 0xD0），应当返回固定的芯片 ID（0x58），用这个"读 ID"技巧可以验证通信是否正常——这也是所有 I2C 驱动开发的第一步。

23.4 手动实现单总线协议

第 12 章的 DHT22 用的是"单总线"协议，很多教程直接调用现成库。现在我们来手动实现它，深入理解单总线的时序。单总线通过一根数据线上的高低电平持续时间来编码数据：主机先发出一个启动信号，从机回应后，用"高电平持续时间的长短"来表示 0 或 1。

```
// 手动读取 DHT22（单总线协议，核心片段）
#define DHTPIN 4

// 等待引脚变为指定电平，返回是否超时
bool waitFor(int level, unsigned long timeout_us) {
  unsigned long t = micros();
  while (digitalRead(DHTPIN) != level) {
    if (micros() - t > timeout_us) return false;
```

```
  }
  return true;
}

// 读取一个字节
uint8_t readByte() {
  uint8_t result = 0;
  for (int i = 0; i < 8; i++) {
    // 等待数据位开始
    waitFor(HIGH, 100);     // 跳过数据位的低电平
    delayMicroseconds(40);  // 采样点：40us 后判断
    result <<= 1;
    if (digitalRead(DHTPIN) == HIGH) result |= 1;  // 高电平长 => 1
    waitFor(LOW, 100);      // 等待本位结束
  }
  return result;
}

bool readDHT(float* temp, float* hum) {
  // 主机启动信号：拉低 18ms 再拉高
  pinMode(DHTPIN, OUTPUT);
  digitalWrite(DHTPIN, LOW);
  delay(18);
  digitalWrite(DHTPIN, HIGH);
  delayMicroseconds(40);
  pinMode(DHTPIN, INPUT);

  // 从机响应信号
  if (!waitFor(LOW, 100) || !waitFor(HIGH, 100) || !waitFor(LOW, 100))
    return false;

  // 读取 5 个字节：湿度高、湿度低、温度高、温度低、校验和
  uint8_t data[5];
  for (int i = 0; i < 5; i++) data[i] = readByte();

  // 校验：前 4 字节之和等于第 5 字节
  if ((uint8_t)(data[0] + data[1] + data[2] + data[3]) != data[4])
    return false;

  *hum = (data[0] * 256 + data[1]) / 10.0;
  *temp = ((data[2] & 0x7F) * 256 + data[3]) / 10.0;
```

```
  return true;
}
```

这段代码手动实现了单总线的启动、响应、逐位读取和校验。虽然比现成库复杂，但通过它你能真正理解"时序"的含义——**驱动开发的核心，就是把数据手册上的时序图翻译成精确的电平与延时操作**。

23.5 编写健壮驱动的工程方法

- **先读 ID 验证通信**：绝大多数器件有只读的 ID 寄存器，第一步先读 ID，确认地址、接线、时序无误；
- **做好超时处理**：所有等待循环都要设超时，防止器件异常时程序卡死；
- **校验数据**：利用器件提供的校验和/CRC 验证数据完整性；
- **失败重试**：一次读取失败可重试几次，增强鲁棒性；
- **分层设计**：把"底层时序"与"高层逻辑"分开，底层函数可复用。

23.6 本章小结

- 驱动开发的本质是按数据手册的时序与寄存器读写数据；
- I2C 寄存器读写有通用范式，读 ID 是验证通信的第一步；
- 单总线协议靠电平持续时间编码，手动实现能深入理解时序；
- 超时、校验、重试、分层是健壮驱动的工程保障。

思考与练习

1. 用寄存器读写函数，读出手头某 I2C 器件的 ID，并与数据手册对照。
2. 手动实现单总线读取 DHT11，并解释 0 和 1 在时序上的区别。
3. 给读 DHT 的代码加上"连续失败 3 次则报警"的鲁棒性处理。

第24章 实时性与性能优化

24.1 学习目标

当项目越来越复杂、对响应速度和稳定性要求越来越高时，"能跑"就不再够用，我们需要让程序"跑得好"。本章关注 ESP32 程序的**实时性与性能优化**，学完后你将能够：

- 理解实时性的含义与影响因素；
- 掌握任务优先级的合理设计；
- 运用测量与分析手段定位性能瓶颈；
- 应用编译优化、DMA 等性能优化技巧。

24.2 实时性的含义

实时性 不是"跑得快"，而是"**响应得及时且确定**"。一个实时系统，关键任务必须在规定的时间窗口内得到响应。例如，电机控制环必须在每个控制周期内完成计算，晚了就会失控；按键中断必须及时捕捉，否则会漏按。

影响 ESP32 实时性的主要因素有：

- **任务优先级设置不当**：低优先级任务被高优先级任务"饿死"；
- **阻塞操作**：在关键任务中使用 `delay()` 或长时间等待；
- **中断处理过慢**：中断服务函数耗时过长，影响其他中断；
- **无线协议栈的占用**：Wi-Fi/BLE 协议栈需要 CPU 时间，可能干扰实时任务。

24.3 任务优先级设计

FreeRTOS 的调度器按优先级调度：**优先级高的任务先运行，同优先级轮流运行**。合理的优先级设计是实时性的基础。几条经验法则：

- **关键、周期短、要求及时的任务给高优先级**：如控制环、按键检测；
- **耗时、可延迟的任务给低优先级**：如日志打印、数据上报；
- **避免"饿死"**：确保低优先级任务也能获得执行机会（可通过合适的延时或事件驱动）；
- **中断优先级与任务优先级分开考虑**：中断总能抢占任务，但中断本身也分优先级。

下面是一个优先级设计的对比示例：高优先级的按键任务能及时响应，即使低优先级的日志任务在忙。

```
// 任务优先级设计示例
void taskButton(void* param) {   // 高优先级：及时响应按键
  while (1) {
    if (digitalRead(23) == LOW) {
      Serial.println("按键按下");  // 简短处理
    }
    vTaskDelay(10 / portTICK_PERIOD_MS);
  }
}

void taskReport(void* param) {   // 低优先级：耗时上报
  while (1) {
    // 模拟耗时的网络上报
    Serial.println("开始上报...");
    vTaskDelay(5000 / portTICK_PERIOD_MS);
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(23, INPUT_PULLUP);
  xTaskCreate(taskButton, "btn", 2048, NULL, 3, NULL);  // 优先级 3
  xTaskCreate(taskReport, "rpt", 4096, NULL, 1, NULL);  // 优先级 1
}

void loop() {
}
```

24.4 测量与分析

优化不能靠猜，必须靠**测量**。常用的测量手段有：

- **时间戳测量**：用 `micros()`/`esp_timer_get_time()` 测量某段代码的耗时；
- **内存监控**：`ESP.getFreeHeap()`、`ESP.getMinFreeHeap()`；
- **系统统计**：`esp_get_free_heap_size()`、任务水印（`uxTaskGetStackHighWaterMark()` 查看栈剩余）；
- **专业工具**：ESP-IDF 提供 SystemView、perf 等性能分析工具，可查看 CPU 占用、任务调度时间线。

一个简单的耗时测量模板：

```
unsigned long start = micros();
someFunction();                       // 要测量的代码
unsigned long elapsed = micros() - start;
Serial.printf("耗时: %lu 微秒\n", elapsed);
```

把可疑的代码段逐个测量，就能精准定位瓶颈，而不是盲目猜测。

24.5 性能优化技巧

- **提高编译优化等级**：Arduino 默认优化等级为 -Os（优化体积），对计算密集任务可提高优化等级，换取更快的执行速度；
- **用硬件外设替代软件模拟**：优先用硬件 PWM、硬件 I2C/SPI/UART，而不是软件模拟；
- **使用 DMA 搬运数据**：大批量数据（如 ADC 采样、显示屏刷屏）用 DMA 传输，CPU 可同时干别的活；
- **减少不必要的浮点运算**：浮点运算较慢，能用整数运算就尽量用整数（如把小数放大成整数处理）；
- **避免频繁的字符串拼接**：尤其 `String` 的 `+` 拼接会产生大量临时对象，建议用字符数组或格式化函数；
- **合理使用中断和事件**：把轮询改成中断/事件驱动，减少 CPU 空转。

24.6 本章小结

- 实时性强调"及时且确定地响应"，受优先级、阻塞、中断等因素影响；
- 高优先级给关键短任务，低优先级给耗时任务，避免饿死；
- 优化先测量，用时间戳、内存监控、栈水印定位瓶颈；
- 提升优化等级、用硬件外设、DMA、减少浮点和字符串拼接是常用手段。

思考与练习

1. 用 `micros()` 测量 `digitalRead()` 和一次 I2C 读写的耗时，比较它们的差异。
2. 用 `uxTaskGetStackHighWaterMark()` 检查你某个任务的栈剩余空间。
3. 设计一个"高优先级控制任务 + 低优先级上报任务"的程序，验证优先级的效果。

第25章 安全与加密

25.1 学习目标

物联网设备常年暴露在互联网上，安全威胁无处不在：数据被窃听、固件被篡改、设备被劫持……安全问题一旦出事，后果远比功能 Bug 严重。本章学习 ESP32 的安全能力，学完后你将能够：

- 理解物联网面临的主要安全威胁；
- 使用安全的通信协议（HTTPS、TLS）；
- 理解 Flash 加密与安全启动；
- 掌握设备身份认证与密钥管理的基本方法。

25.2 物联网安全威胁

物联网设备面临的安全威胁主要有几类：

- **窃听（Eavesdropping）**：明文传输的数据被截获，泄露隐私和机密；
- **篡改（Tampering）**：传输中的指令被修改，如把"开锁"改成"开所有锁"；
- **伪造（Spoofing）**：攻击者冒充服务器或设备，发送恶意指令；
- **固件篡改**：设备固件被替换为带后门的版本；
- **密钥泄露**：硬编码在固件里的密码、密钥被逆向提取。

防御这些威胁，需要综合运用加密通信、身份认证、固件保护、密钥管理等多种手段，形成**纵深防御**。

25.3 加密通信：HTTPS 与 TLS

TLS（Transport Layer Security，传输层安全协议）是互联网加密通信的标准，HTTPS 就是"HTTP + TLS"。它提供**加密**（防窃听）、**完整性校验**（防篡改）和**身份认证**（防伪造）三重保障。ESP32 内置硬件加密加速器，支持 TLS，可方便地发起 HTTPS 请求。

Arduino 框架下，使用 `WiFiClientSecure` 类发起 HTTPS 请求：

```
// HTTPS 安全请求
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>

const char* ssid = "你的WiFi名称";
```

```
const char* password = "你的WiFi密码";
const char* url = "https://example.com/api";  // 注意 https

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);

  WiFiClientSecure client;
  client.setInsecure();   // 仅测试用：跳过证书校验（生产环境务必校验证书！）

  HTTPClient http;
  http.begin(client, url);
  int code = http.GET();
  Serial.print("状态码: "); Serial.println(code);
  if (code > 0) Serial.println(http.getString());
  http.end();
}

void loop() {
}
```

> 【注意】 安全提示：示例中的 `client.setInsecure()` 会跳过服务器证书校验，**只能用于测试**。生产环境必须配置受信任的 CA 证书并开启校验，否则中间人攻击仍然可以得逞。

25.4 Flash 加密与安全启动

ESP32 提供两项底层的固件保护机制：

- **Flash 加密（Flash Encryption）**：把 Flash 里的程序和数据加密存储，防止攻击者直接读出固件、逆向分析或提取密钥；
- **安全启动（Secure Boot）**：设备启动时校验固件的数字签名，只有通过签名验证的固件才能运行，防止被替换为恶意固件。

这两项机制配合使用，能有效防止固件被读取和篡改。它们通常通过 ESP-IDF 的 menuconfig 或乐鑫的量产工具进行配置，是物联网产品安全设计的重要组成部分。

25.5 设备身份与密钥管理

安全通信的前提是"身份可信"。物联网设备安全的关键实践包括：

- **一机一密**：每台设备使用独一无二的密钥/证书，而非所有设备共用一个密钥（否则一台被破解，全线沦陷）；
- **密钥安全存储**：密钥存放在加密的存储区（如 ESP32 的 eFuse），而非明文硬编码在代码里；
- **双向认证**：设备验证服务器，服务器也验证设备，防止伪造双方；
- **最小权限原则**：设备只获得完成任务所需的最小权限；
- **定期更新**：及时修补安全漏洞，更新证书和密钥。

乐鑫提供了从安全启动、Flash 加密到云端密钥管理（如 ESP RainMaker、云平台安全方案）的完整工具链，生产项目建议充分利用这些官方能力。

25.6 本章小结

- 物联网安全威胁包括窃听、篡改、伪造、固件篡改、密钥泄露等；
- TLS/HTTPS 提供加密、完整性、认证三重保障，ESP32 硬件加速支持；
- Flash 加密与安全启动保护固件不被读取和篡改；
- 一机一密、密钥安全存储、双向认证是设备安全的核心实践。

思考与练习

1. 用 ESP32 发起一次 HTTPS 请求，对比与 HTTP 请求的代码差异。
2. 查阅乐鑫文档，了解 Flash 加密和安全启动的配置流程。
3. 分析一个你熟悉的物联网产品，列出它可能面临的安全威胁与对策。

第五部分 实战篇

第26章 实战项目一：智能家居控制中心

26.1 项目概述

从本章开始，我们把前面学到的知识融会贯通，完成五个完整的实战项目。第一个项目是**智能家居控制中心**——一个可以通过手机远程控制的家庭设备中枢，能控制灯光、风扇等家用电器，并实时反馈状态。这是智能家居领域最典型、最经典的应用。

项目功能：

- 通过 MQTT 接收手机 App 的控制指令；
- 控制多路继电器，进而控制灯、风扇等电器；
- 实时上报设备状态和温湿度数据；
- 支持断线自动重连，长时间稳定运行。

技术栈：ESP32 + MQTT + 继电器模块 + DHT22 温湿度传感器 + OLED 显示屏（可选）。

26.2 硬件准备与接线

本项目需要以下硬件：

器件	数量	作用
ESP32 开发板	1	主控
继电器模块（2～4 路）	1	控制强电设备
DHT22 温湿度传感器	1	采集环境数据
OLED 显示屏（可选）	1	本地显示状态
杜邦线、电源	若干	连接与供电

接线：

- 继电器模块：IN1 → GPIO26，IN2 → GPIO27，VCC → 5V，GND → GND；
- DHT22：DATA → GPIO4，VCC → 3V3，GND → GND；
- OLED（可选）：SDA → GPIO21，SCL → GPIO22，VCC → 3V3，GND → GND。

> 【注意】 安全提示：继电器控制的是 220 V 强电，接线时务必断电操作，强电部分与弱电部分保持隔离。初学者建议先用 LED 或低压小电器替代强电负载，待电路和程序验证无误后再接真实电器。

26.3 软件设计

整个项目的软件架构遵循"**采集任务 + 控制任务 + 通信回调**"的清晰分层：

- **通信层**：MQTT 回调函数接收控制指令，更新设备状态；
- **采集任务**：定时读取温湿度，通过 MQTT 发布；
- **控制任务**：根据设备状态驱动继电器，并上报状态；
- **显示任务**：在 OLED 上显示运行状态。

下面是核心代码：

```
// 智能家居控制中心
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

const char* ssid = "你的WiFi名称";
const char* password = "你的WiFi密码";
const char* mqtt_server = "broker.emqx.io";

#define RELAY1_PIN 26
#define RELAY2_PIN 27
#define DHTPIN 4
#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);
WiFiClient espClient;
PubSubClient mqtt(espClient);

bool lightState = false;   // 灯的状态
bool fanState = false;     // 风扇的状态
unsigned long lastReport = 0;

// MQTT 回调：接收控制指令
void callback(char* topic, byte* payload, unsigned int length) {
  String msg = "";
  for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];
```

```
  if (String(topic) == "home/light/set") {
    lightState = (msg == "1");
    digitalWrite(RELAY1_PIN, lightState ? HIGH : LOW);
    mqtt.publish("home/light/state", lightState ? "1" : "0");
  } else if (String(topic) == "home/fan/set") {
    fanState = (msg == "1");
    digitalWrite(RELAY2_PIN, fanState ? HIGH : LOW);
    mqtt.publish("home/fan/state", fanState ? "1" : "0");
  }
}

void reconnect() {
  while (!mqtt.connected()) {
    if (mqtt.connect("home_center_001")) {
      mqtt.subscribe("home/light/set");   // 订阅控制主题
      mqtt.subscribe("home/fan/set");
    } else {
      delay(3000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(RELAY1_PIN, OUTPUT);
  pinMode(RELAY2_PIN, OUTPUT);
  digitalWrite(RELAY1_PIN, LOW);   // 初始关闭
  digitalWrite(RELAY2_PIN, LOW);
  dht.begin();

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);

  mqtt.setServer(mqtt_server, 1883);
  mqtt.setCallback(callback);
}

void loop() {
  if (!mqtt.connected()) reconnect();
  mqtt.loop();
```

```
  // 每 30 秒上报一次温湿度
  if (millis() - lastReport > 30000) {
    lastReport = millis();
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    char buf[32];
    sprintf(buf, "{\"t\":%.1f,\"h\":%.1f}", t, h);
    mqtt.publish("home/env", buf);
  }
}
```

26.4 测试与联调

上传程序后，用 MQTT 客户端工具（如 MQTTX）进行联调：

1. 订阅 `home/light/state`、`home/fan/state` 和 `home/env`，观察设备状态上报；
2. 向 `home/light/set` 发布 `1` 或 `0`，观察继电器动作（LED/小电器开合）和状态回传；
3. 检查断线重连：断开路由器后恢复，确认设备能自动重连并重新订阅。

测试通过后，这个控制中心就已经是一个可用的智能家居中枢了。你可以继续扩展：增加更多路继电器、加入定时开关、接入语音助手、增加本地按键控制等。

26.5 扩展方向

- 接入更多传感器（光照、人体红外、门磁），实现场景联动；
- 增加"定时任务"，如"每天 18:00 自动开灯"；
- 加入本地 Web 控制页面，不依赖云也能操作；
- 对接主流智能家居平台（如 Home Assistant），融入更大的生态。

26.6 本章小结

- 智能家居控制中心 = ESP32 + MQTT + 继电器 + 传感器；
- 软件分层：通信回调处理指令、采集任务上报数据、控制任务驱动输出；
- 控制强电务必注意安全，先用低压负载验证；
- 项目可向定时、联动、Web、语音等方向持续扩展。

第27章 实战项目二：物联网环境监测站

27.1 项目概述

第二个项目是**物联网环境监测站**——一台能采集温度、湿度、气压、空气质量等环境数据，并通过网络实时上报到云端、支持历史数据查询的监测设备。它广泛应用于智慧农业、仓储、机房、家庭环境等场景。

项目功能：

- 采集温湿度（DHT22）、气压（BMP280）等多种环境数据；
- 通过 HTTP/MQTT 上报到云端平台；
- 本地 OLED 实时显示数据；
- 支持离线数据缓存，网络恢复后补传。

技术栈：ESP32 + 多传感器 + HTTP/MQTT + LittleFS（离线缓存）。

27.2 硬件准备与接线

器件	数量	作用
ESP32 开发板	1	主控
DHT22	1	温湿度
BMP280	1	气压/海拔
OLED（SSD1306）	1	本地显示
可选：空气质量传感器（如 PMS 系列）	1	空气质量

接线：

- DHT22：DATA → GPIO4，VCC → 3V3，GND → GND；
- BMP280：SDA → GPIO21，SCL → GPIO22，VCC → 3V3，GND → GND；
- OLED：SDA → GPIO21，SCL → GPIO22（与 BMP280 共用 I2C 总线），VCC → 3V3，GND → GND。

> 【提示】 注意：BMP280 和 OLED 都挂在同一条 I2C 总线上，这正是 I2C 总线的优势——多个器件共享两线。请先用第 10 章的 I2C 扫描器确认二者的地址（通常 OLED 为 0x3C，BMP280 为 0x76/0x77）。

27.3 软件设计

本项目采用 FreeRTOS 多任务架构，各任务职责分明：

- **采集任务**：定时读取所有传感器；
- **显示任务**：把最新数据刷新到 OLED；
- **上报任务**：把数据上报云端，失败时缓存到本地文件；
- **重传任务**：网络恢复后补传缓存数据。

核心代码结构如下：

```
// 物联网环境监测站（核心结构）
#include <WiFi.h>
#include <Wire.h>
#include <DHT.h>
#include <Adafruit_BMP280.h>
#include <Adafruit_SSD1306.h>
#include <LittleFS.h>

// ... 引脚与对象定义 ...

// 全局数据（用互斥锁保护）
SemaphoreHandle_t dataMutex;
float g_temp, g_hum, g_pressure;

void taskCollect(void* param) {   // 采集任务
  while (1) {
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    float p = bmp.readPressure() / 100.0F;

    xSemaphoreTake(dataMutex, portMAX_DELAY);  // 上锁更新数据
    g_temp = t; g_hum = h; g_pressure = p;
    xSemaphoreGive(dataMutex);

    vTaskDelay(5000 / portTICK_PERIOD_MS);
  }
}

void taskDisplay(void* param) {   // 显示任务
  while (1) {
    xSemaphoreTake(dataMutex, portMAX_DELAY);
    // ... 把 g_temp/g_hum/g_pressure 显示到 OLED ...
```

```
    xSemaphoreGive(dataMutex);
    vTaskDelay(2000 / portTICK_PERIOD_MS);
  }
}

void taskReport(void* param) {    // 上报任务
  while (1) {
    xSemaphoreTake(dataMutex, portMAX_DELAY);
    // ... 构造 JSON 并上报云端；失败则写入 LittleFS 缓存 ...
    xSemaphoreGive(dataMutex);
    vTaskDelay(60000 / portTICK_PERIOD_MS);
  }
}

void setup() {
  // 初始化串口、WiFi、传感器、OLED、LittleFS、队列与互斥锁
  // 创建各任务
}

void loop() {
}
```

由于篇幅所限，这里只给出架构骨架，完整的传感器初始化、JSON 构造和 OLED 刷新代码，读者可结合第 10、12、14、17、18 章的内容自行补齐。**这个"采集—显示—上报—重传"的多任务架构，是绝大多数物联网数据采集设备的标准范式**，务必理解透彻。

27.4 云端展示

数据上报到云端后，还需要一个展示界面。你可以选择：

- 使用现成的物联网云平台（如阿里云 IoT、腾讯云 IoT、OneNET、EMQX Cloud），配置数据可视化大屏；
- 自建后端（如用 Node.js/Python 写接口 + 前端页面）展示曲线；
- 接入第三方可视化工具（如 Grafana + 时序数据库 InfluxDB）。

无论选择哪种，核心都是"设备上报 → 云端存储 → 可视化展示"这一条数据链路。

27.5 本章小结

- 环境监测站 = ESP32 + 多传感器 + OLED + 云上报 + 离线缓存；
- 多任务架构（采集/显示/上报/重传）是数据采集设备的标准范式；

- 互斥锁保护共享数据，队列/文件缓存解决离线问题；
- 云端展示是完整闭环的最后一步。

思考与练习

1. 完成环境监测站的完整代码，实现多传感器采集与 OLED 显示。
2. 加入离线缓存：断网时把数据写进 LittleFS，恢复后补传。
3. 选择一个云平台，把你的监测数据接入并生成可视化图表。

第28章 实战项目三：智能门锁与安防系统

28.1 项目概述

第三个项目是**智能门锁与安防系统**——一款结合了密码验证、门磁检测、异常报警的智能安防设备。智能门锁是智能家居中技术含量较高、安全要求最严格的产品之一，本项目将带你理解其中的核心技术。

项目功能：

- 通过手机 App（BLE 或 MQTT）远程开锁/上锁；
- 门磁传感器检测门的开关状态；
- 非法开门（未授权而门被打开）时触发报警；
- 记录开锁日志。

技术栈：ESP32 + BLE/MQTT + 门磁传感器 + 蜂鸣器报警 + 继电器（模拟锁体）。

28.2 硬件准备与接线

器件	数量	作用
ESP32 开发板	1	主控
门磁传感器（干簧管）	1	检测门开合
有源蜂鸣器	1	报警
继电器	1	模拟电控锁
LED 指示灯	2	状态指示

接线：

- 门磁传感器：一端接 GND，另一端接 GPIO13（使用内部上拉，门开时状态变化）；
- 蜂鸣器：正极接 GPIO12，负极接 GND；
- 继电器：IN → GPIO26，VCC → 5V，GND → GND；
- LED：绿色（已上锁）接 GPIO2，红色（报警）接 GPIO4。

28.3 软件设计

门锁软件的核心是一个**状态机（State Machine）**。设备在不同状态（已上锁/已解锁/报警中）之间切换，每个状态下对事件（开锁指令、门磁变化）的处理逻辑不同。状态机让复杂的逻辑变得清晰可控。

```
// 智能门锁状态机（核心片段）
enum LockState { LOCKED, UNLOCKED, ALARM };

LockState state = LOCKED;

// 门磁状态读取（假设门闭合时门磁为 HIGH）
bool doorClosed() {
  return digitalRead(13) == HIGH;
}

// 蜂鸣器报警
void alarm() {
  for (int i = 0; i < 10; i++) {
    digitalWrite(12, HIGH);
    delay(200);
    digitalWrite(12, LOW);
    delay(200);
  }
}

void loop() {
  bool closed = doorClosed();

  switch (state) {
    case LOCKED:
      digitalWrite(2, HIGH);          // 绿灯：已上锁
      if (!closed) {                  // 上锁状态下门被打开 => 非法开门
        state = ALARM;
        alarm();
      }
      // 收到开锁指令时：先解锁，再进入 UNLOCKED
      if (收到开锁指令) {
        digitalWrite(26, HIGH);       // 驱动继电器开锁
        state = UNLOCKED;
      }
      break;
```

```
    case UNLOCKED:
      digitalWrite(2, LOW);           // 绿灯灭
      if (closed) {                   // 门重新关上 => 自动上锁
        digitalWrite(26, LOW);        // 释放继电器（上锁）
        state = LOCKED;
      }
      break;

    case ALARM:
      // 报警处理：持续鸣叫，直到门重新关闭或收到复位指令
      if (closed) {
        state = LOCKED;               // 门关好，解除报警
      }
      break;
  }
}
```

这段状态机代码清晰地表达了门锁的核心逻辑：**上锁状态下门被打开 = 非法入侵 → 报警**；解锁后门重新关上 → 自动上锁。把"收到开锁指令"替换为 BLE 或 MQTT 的指令接收（参见第 16、15 章），即可实现远程开锁。

28.4 安全设计考量

门锁是安防设备，安全设计至关重要，绝不能用"玩具级"的代码糊弄。要点包括：

- **通信加密**：开锁指令必须走加密通道（BLE 加密连接或 MQTT over TLS），防止被截获和重放；
- **防重放攻击**：指令中加入时间戳或随机数，防止攻击者录制后重放；
- **身份认证**：验证开锁者的身份，而非任何能连上设备的人都能开锁；
- **本地优先**：门锁的本地控制（按键、密码）应独立于网络，断网也能正常开锁；
- **异常日志**：记录每次开锁的时间、方式，便于追溯。

28.5 扩展方向

- 加入键盘，实现密码开锁（密码错误多次则锁定报警）；
- 加入指纹模块，实现指纹开锁；
- 接入摄像头，报警时抓拍上传；
- 加入低功耗设计（深睡 + 门磁中断唤醒），用电池长期供电。

28.6 本章小结

- 智能门锁 = ESP32 + 门磁 + 蜂鸣器 + 继电器，核心是状态机；
- 状态机清晰表达"上锁—解锁—报警"的状态切换逻辑；
- 安防设备必须重视通信加密、防重放、身份认证等安全设计；
- 可扩展密码、指纹、摄像头、低功耗等能力。

第29章 实战项目四：智能小车

29.1 项目概述

第四个项目是**智能小车**——一款能自动避障、可遥控的移动机器人。智能小车是机器人学的入门经典，涉及电机驱动、传感器测距、运动控制和远程通信等多个领域。

项目功能：

- 手机/遥控器远程控制前进、后退、转向；
- 超声波传感器自动检测障碍并避障；
- 速度可调（PWM 调速）。

技术栈：ESP32 + 电机驱动模块 + 直流电机 + 超声波传感器 + BLE/MQTT 遥控。

29.2 硬件准备与接线

器件	数量	作用
ESP32 开发板	1	主控
电机驱动模块（如 L298N/TB6612）	1	驱动电机
直流减速电机	2～4	提供动力
超声波传感器 HC-SR04	1	检测障碍
电池组	1	供电

接线（以 L298N 驱动两轮为例）：

- L298N 电机输出 OUT1/OUT2 接左电机，OUT3/OUT4 接右电机；
- L298N 控制引脚：IN1→GPIO26、IN2→GPIO27（左），IN3→GPIO14、IN4→GPIO12（右）；
- 使能引脚 ENA→GPIO25、ENB→GPIO33（PWM 调速）；
- 超声波：Trig→GPIO5、Echo→GPIO18；
- 电池组给电机驱动供电，注意**共地**。

> 【提示】 供电要点：电机是"耗电大户"，务必用独立的电池组给电机驱动供电，ESP32 单独供电或通过驱动板的 5V 输出供电，且所有电源**共地**。切不可用 ESP32 开发板的 3V3 引脚直接驱动电机。

29.3 软件设计

小车的运动控制，本质是对两个电机的方向和速度的组合。我们把底层封装成几个基本动作函数，上层逻辑只需调用这些函数。

```
// 智能小车运动控制（核心片段）
#define ENA 25   // 左电机使能（PWM）
#define IN1 26
#define IN2 27
#define ENB 33   // 右电机使能（PWM）
#define IN3 14
#define IN4 12

const int SPEED = 200;   // 基础速度 0~255

void setMotor(int inA, int inB, int en, int speed) {
  if (speed >= 0) {           // 正转
    digitalWrite(inA, HIGH);
    digitalWrite(inB, LOW);
  } else {                    // 反转
    digitalWrite(inA, LOW);
    digitalWrite(inB, HIGH);
    speed = -speed;
  }
  ledcWrite(en, speed);       // PWM 调速（需先 ledcSetup/Attach）
}

void forward()  { setMotor(IN1, IN2, ENA, SPEED);  setMotor(IN3, IN4, ENB,
SPEED); }
void backward() { setMotor(IN1, IN2, ENA, -SPEED); setMotor(IN3, IN4, ENB, -
SPEED); }
void turnLeft() { setMotor(IN1, IN2, ENA, 0);      setMotor(IN3, IN4, ENB,
SPEED); }
void turnRight(){ setMotor(IN1, IN2, ENA, SPEED);  setMotor(IN3, IN4, ENB,
0); }
void stop()     { setMotor(IN1, IN2, ENA, 0);      setMotor(IN3, IN4, ENB,
0); }

void loop() {
  // 读取超声波距离
  float dist = readDistance();   // 复用第 12 章的 HC-SR04 代码

  if (dist < 20) {           // 前方 20cm 内有障碍 => 避障
```

```
    stop();
    turnLeft();              // 左转绕开
    delay(300);
  } else {
    forward();               // 无障碍 => 直行
  }
}
```

把超声波测距（第 12 章）和运动控制结合起来，小车就具备了"自动避障"的能力——遇到障碍自动转向绕行，否则直行。加上 BLE/MQTT 遥控（第 15、16 章），就能在自动模式和遥控模式间切换。

29.4 扩展方向

- 加入循迹传感器，实现循迹行驶；
- 加入更多超声波传感器，实现更精准的避障；
- 加入陀螺仪，实现走直线、转精确角度；
- 接入摄像头，实现视觉识别；
- 加入编码器，实现精确的速度和里程控制。

29.5 本章小结

- 智能小车 = ESP32 + 电机驱动 + 直流电机 + 超声波传感器；
- 运动控制封装为前进/后退/转向/停止等基本动作；
- 电机是耗电大户，必须独立供电并共地；
- 结合测距与遥控，实现避障、遥控等智能行为。

思考与练习

1. 搭建智能小车硬件，实现前进、后退、转向、停止四个基本动作。
2. 实现超声波避障，让小车在障碍环境中自主行驶。
3. 用手机 App 通过 BLE 遥控小车，实现手动与自动模式的切换。

第30章 实战项目五：云端物联网平台

30.1 项目概述

最后一个项目是**综合性的云端物联网平台**——把前面所有知识整合起来，构建一个"设备—云端—应用"完整闭环的物联网系统。这是全书的集大成项目，也是你走向真实产品开发的"毕业设计"。

项目功能：

- 多台 ESP32 设备接入云端，实时上报数据；
- 云端存储设备数据，提供 RESTful API；
- 手机/网页端实时监控设备、下发控制指令；
- 支持设备管理、历史数据查询、告警通知。

技术栈：ESP32（设备端）+ MQTT Broker（通信）+ 后端服务（存储/API）+ 前端（可视化）。

30.2 系统总体架构

一个完整的物联网平台通常包含四层，理解这四层架构，你就理解了物联网系统的全貌：

```
设备层（ESP32 传感器/执行器）
   │  MQTT 发布/订阅
   ▼
接入层（MQTT Broker，如 EMQX/Mosquitto）
   │  消息转发
   ▼
服务层（后端服务：数据存储、业务逻辑、REST API）
   │  HTTP/WebSocket
   ▼
应用层（手机 App / Web 前端 / 可视化大屏）
```

- **设备层**：ESP32 采集数据、执行指令（第 12～19 章的能力）；
- **接入层**：MQTT Broker 负责海量设备的消息接入与转发（第 15 章）；
- **服务层**：后端订阅 MQTT 消息存入数据库，对外提供查询和控制接口（第 14 章）；
- **应用层**：用户界面，展示数据、下发指令（第 14 章的 HTTP/WebSocket）。

30.3 设备端设计

设备端我们已在前面章节反复练习，这里给出一个规范的"设备模板"。每台设备统一遵循"主题规范 + 心跳 + 数据上报 + 指令响应"的协议约定：

```
// 物联网设备通用模板（约定主题协议）
// 设备 ID：dev_001
// 数据上报主题：dev/001/data
// 指令下发主题：dev/001/cmd
// 心跳主题：    dev/001/heartbeat

#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "你的WiFi名称";
const char* password = "你的WiFi密码";
const char* mqtt_server = "你的服务器地址";   // 自建或云 Broker

WiFiClient espClient;
PubSubClient mqtt(espClient);

void callback(char* topic, byte* payload, unsigned int length) {
  String msg = "";
  for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];
  Serial.print("收到指令: "); Serial.println(msg);
  // 解析并执行指令（如控制继电器、上报数据等）
  // 执行后向 dev/001/ack 回复确认
}

void reconnect() {
  while (!mqtt.connected()) {
    if (mqtt.connect("dev_001")) {
      mqtt.subscribe("dev/001/cmd");   // 订阅自己的指令主题
    } else {
      delay(3000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
```

```
  mqtt.setServer(mqtt_server, 1883);
  mqtt.setCallback(callback);
}

void loop() {
  if (!mqtt.connected()) reconnect();
  mqtt.loop();

  static unsigned long last = 0;
  if (millis() - last > 10000) {   // 每 10 秒
    last = millis();
    mqtt.publish("dev/001/heartbeat", "alive");          // 心跳
    mqtt.publish("dev/001/data", "{\"t\":25.6,\"h\":55}"); // 数据
  }
}
```

规范的主题协议（`dev/{ID}/data`、`dev/{ID}/cmd`、`dev/{ID}/heartbeat`）让设备与云端之间的协作清晰有序，也方便水平扩展——接入第二台设备时，只需把设备 ID 换成 `dev_002` 即可。

30.4 云端后端设计

后端服务是整个平台的大脑。它需要完成：订阅 MQTT 消息 → 解析数据 → 存入数据库 → 对外提供查询/控制 API。后端可以用任何你熟悉的语言实现，下面是 Node.js + MQTT + Express 的极简示例：

```
// 后端服务（Node.js 示例，核心片段）
const mqtt = require('mqtt');
const express = require('express');
const app = express();

// 连接 MQTT Broker
const client = mqtt.connect('mqtt://你的服务器地址');
client.on('connect', () => {
  console.log('已连接 Broker');
  client.subscribe('dev/+/data');       // 订阅所有设备的数据
  client.subscribe('dev/+/heartbeat');
});

// 收到设备数据时处理（示例：打印并存储）
client.on('message', (topic, payload) => {
  console.log(`[${topic}] ${payload.toString()}`);
```

```
  // 实际项目：解析 JSON，写入数据库（MySQL/InfluxDB 等）
});

// 提供控制 API：向指定设备下发指令
app.post('/device/:id/cmd', (req, res) => {
  const id = req.params.id;
  const cmd = req.body.cmd;
  client.publish(`dev/${id}/cmd`, JSON.stringify(cmd));
  res.json({ ok: true });
});

app.listen(3000, () => console.log('后端服务运行在 3000 端口'));
```

后端把 MQTT 的消息流和 HTTP 的请求响应连接起来：设备数据经 MQTT 进入后端入库，用户控制指令经 HTTP API 转成 MQTT 发布给设备。这"一进一出"的两条链路，构成了物联网平台的核心。

30.5 前端可视化

前端负责把数据"画出来"。最简单的做法是做一个网页，通过后端 API 拉取数据并绘制图表（可用 ECharts 等图表库），或通过 WebSocket 实时刷新。对于学习阶段，你甚至可以先用云平台自带的可视化工具快速看到效果，再逐步自研前端。

一个完整的可视化界面通常包括：

- **设备列表与在线状态**：哪些设备在线、哪些离线；
- **实时数据面板**：温度、湿度等数据的实时曲线；
- **控制按钮**：开关、模式切换等控制；
- **历史数据查询**：按时间段查看历史曲线；
- **告警提示**：数据超阈值时的通知。

30.6 从原型到产品

完成这个项目后，你就已经走完了一条完整的物联网产品开发链路。但从"原型"到"产品"还有一段路要走，需要补齐：

- **稳定性**：长时间运行、断网重连、异常恢复；
- **安全性**：TLS 加密、设备认证、权限控制；
- **可维护性**：OTA 升级、远程日志、设备管理；
- **规模化**：海量设备接入、消息堆积处理、成本优化；
- **合规性**：数据隐私、无线电认证等。

这些正是本书第四部分（精通篇）所铺垫的能力。当你把这些都做好了，你就完成了从"开发者"到"物联网工程师"的蜕变。

30.7 本章小结

- 物联网平台 = 设备层 + 接入层 + 服务层 + 应用层四层架构；
- 规范的 MQTT 主题协议是设备与云端协作的基础；
- 后端把 MQTT 消息流与 HTTP API 连接，是平台的核心；
- 从原型到产品还需补齐稳定性、安全性、可维护性、规模化与合规性。

思考与练习

1. 画出你的物联网平台的完整架构图，标明各层之间的通信协议。
2. 用 Node.js/Python 搭建一个最小后端，实现设备数据入库和控制 API。
3. 设计一套规范的主题命名协议，并说明如何扩展到多设备场景。

附录

附录A 常用函数速查

本附录汇总本书涉及的核心 API，方便读者随时查阅。函数按功能分类，均以 Arduino 框架为准。

A.1 数字输入输出

函数	说明
`pinMode(pin, mode)`	设置引脚模式，mode 为 `OUTPUT`、`INPUT`、`INPUT_PULLUP` 等
`digitalWrite(pin, value)`	输出数字电平，value 为 `HIGH` 或 `LOW`
`digitalRead(pin)`	读取引脚数字电平，返回 `HIGH` 或 `LOW`
`analogRead(pin)`	读取 ADC 值，返回 0～4095
`analogReadResolution(bits)`	设置 ADC 分辨率（默认 12 位）
`attachInterrupt(pin, isr, mode)`	绑定外部中断，mode 为 `FALLING`、`RISING`、`CHANGE` 等
`detachInterrupt(pin)`	解除外部中断

A.2 PWM（ledc）

函数	说明
`ledcSetup(channel, freq, resolution)`	配置 LEDC 通道的频率和分辨率
`ledcAttachPin(pin, channel)`	把通道绑定到引脚
`ledcWrite(channel, duty)`	设置占空比
`ledcDetachPin(pin)`	解除引脚绑定

A.3 时间与延时

函数	说明
`delay(ms)`	阻塞延时（毫秒）
`delayMicroseconds(us)`	阻塞延时（微秒）
`millis()`	返回程序启动以来的毫秒数
`micros()`	返回程序启动以来的微秒数

函数	说明
`vTaskDelay(ticks)`	FreeRTOS 非阻塞延时

A.4 串口

函数	说明
`Serial.begin(baud)`	初始化串口
`Serial.available()`	返回可读字节数
`Serial.read()`	读取一个字节
`Serial.print(x)` / `Serial.println(x)`	打印数据（后者带换行）
`Serial.write(byte)`	写入原始字节

A.5 I2C（Wire）

函数	说明
`Wire.begin()` / `Wire.begin(sda, scl)`	初始化 I2C
`Wire.beginTransmission(addr)`	开始向指定地址传输
`Wire.write(byte)`	写入一个字节
`Wire.endTransmission()`	结束传输
`Wire.requestFrom(addr, len)`	向指定地址请求读取 len 字节
`Wire.read()`	读取一个字节

A.6 SPI

函数	说明
`SPI.begin()`	初始化 SPI
`SPI.beginTransaction(settings)`	开始一次事务
`SPI.transfer(byte)`	收发一个字节
`SPI.endTransaction()`	结束事务

A.7 Wi-Fi（WiFi.h）

函数	说明
`WiFi.mode(m)`	设置模式：`WIFI_STA`、`WIFI_AP`、`WIFI_AP_STA`
`WiFi.begin(ssid, pass)`	连接 Wi-Fi
`WiFi.status()`	返回连接状态
`WiFi.localIP()`	返回本机 IP 地址
`WiFi.macAddress()`	返回 MAC 地址
`WiFi.RSSI()`	返回信号强度
`WiFi.disconnect()`	断开连接

A.8 MQTT（PubSubClient）

函数	说明
`client.setServer(host, port)`	设置 Broker 地址和端口
`client.setCallback(fn)`	设置消息回调函数
`client.connect(clientId)`	连接 Broker
`client.subscribe(topic)`	订阅主题
`client.publish(topic, payload)`	发布消息
`client.loop()`	处理后台任务（必须循环调用）
`client.connected()`	返回连接状态

A.9 文件系统（LittleFS）

函数	说明
`LittleFS.begin()`	挂载文件系统
`LittleFS.open(path, mode)`	打开文件，mode 为 `"r"` / `"w"` / `"a"`
`f.print(x)` / `f.println(x)`	写入文件
`f.read()`	读取一个字节
`f.available()`	返回可读字节数
`f.close()`	关闭文件

A.10 系统与内存

函数	说明
`ESP.getFreeHeap()`	当前可用堆内存
`ESP.getMinFreeHeap()`	历史最小可用堆
`ESP.getHeapSize()`	堆总大小
`ESP.restart()`	软件重启
`esp_deep_sleep_start()`	进入深度睡眠
`esp_sleep_enable_timer_wakeup(us)`	设置定时唤醒
`ESP.getFlashChipSize()`	Flash 容量

附录B 引脚对照表

下表列出经典 38 引脚 ESP32 开发板的常用引脚及其默认功能，供项目接线时快速查阅。表中的"默认功能"可通过软件重映射，具体以实际代码配置为准。

GPIO	默认/常见功能	备注
GPIO0	BOOT 按键	上电时决定启动模式，慎用
GPIO1	UART0 TX	调试串口发送，勿占用
GPIO2	板载 LED（多数板）	通用输出
GPIO3	UART0 RX	调试串口接收，勿占用
GPIO4	—	通用，常用作 DHT 数据脚
GPIO5	SPI CS	通用
GPIO12	—	通用
GPIO13	—	通用
GPIO14	—	通用
GPIO15	—	通用
GPIO16	UART2 RX	通用
GPIO17	UART2 TX	通用
GPIO18	SPI SCK	通用
GPIO19	SPI MISO	通用
GPIO21	I2C SDA	默认 I2C 数据线
GPIO22	I2C SCL	默认 I2C 时钟线
GPIO23	SPI MOSI	通用
GPIO25	DAC1	模拟输出/通用
GPIO26	DAC2	模拟输出/通用
GPIO27	—	通用
GPIO32	ADC1 CH4	模拟输入/通用
GPIO33	ADC1 CH5	模拟输入/通用
GPIO34	ADC1 CH6	仅输入

GPIO	默认/常见功能	备注
GPIO35	ADC1 CH7	仅输入
GPIO36	ADC1 CH0	仅输入
GPIO39	ADC1 CH3	仅输入

> 提示：GPIO6～GPIO11 连接片外 Flash，未在表中列出且不可使用。GPIO34～GPIO39 无内部上拉/下拉电阻，仅能作输入。

附录C 常见问题与排错

C.1 烧录与连接类

问题：上传时一直提示 "Connecting..." 后失败。 解决：按住 BOOT 键再点击上传，出现 "Connecting..." 后松开；或按住 BOOT 的同时按一下复位键（EN）再松开。同时检查端口选择是否正确、数据线能否传输数据。

问题：串口监视器输出乱码。 解决：确认代码中 `Serial.begin()` 的波特率与串口监视器右下角设置一致（本书统一用 115200）。

问题：电脑找不到 COM 端口。 解决：检查 USB 转串口驱动（CP210x/CH340）是否安装；更换可传输数据的 USB 线。

C.2 供电与硬件类

问题：设备反复复位，尤其开启 Wi-Fi 时。 解决：这是典型的供电不足。Wi-Fi 收发瞬时电流大，请使用稳定电源，避免用电脑 USB 供电能力不足的端口或劣质充电线。

问题：传感器读数为 0 或 NaN。 解决：检查接线（电源、地、信号线）；I2C 器件先运行地址扫描确认地址；DHT 系列注意采样间隔，读取失败（NaN）时加入重试。

问题：LED 不亮。 解决：检查 LED 极性（长脚正、短脚负）；确认限流电阻已串联；确认引脚号与代码一致。

C.3 网络类

问题：Wi-Fi 连接一直失败。 解决：确认 SSID 和密码无误（含大小写）；确认路由器为 2.4 GHz 频段（经典 ESP32 不支持 5 GHz）；靠近路由器重试。

问题：MQTT 连接失败或频繁掉线。 解决：确认 Broker 地址和端口；确认客户端 ID 唯一；确保 `client.loop()` 在循环中被调用；实现断线重连。

C.4 内存与稳定性类

问题：程序运行一段时间后崩溃复位。 解决：排查内存泄漏（观察 `ESP.getFreeHeap()` 是否持续下降）；排查栈溢出（增大任务堆栈）；检查是否频繁动态分配内存。

问题：中断或定时器回调中的代码表现异常。 解决：中断/回调函数应尽量短小，避免在其中使用 `delay()`、打印大量信息或做耗时操作。

附录D 术语表

本附录解释本书出现的主要专业术语，方便读者随时查阅。

ADC（模数转换器）：把模拟电压转换为数字值的电路。ESP32 的 ADC 为 12 位。

BLE（低功耗蓝牙）：专为低功耗设计的蓝牙技术，适合间歇性小数据量传输。

Broker（消息代理）：MQTT 协议中负责接收、存储、转发消息的服务器。

DAC（数模转换器）：把数字值转换为模拟电压的电路，是 ADC 的逆过程。

GPIO（通用输入输出）：单片机可编程控制输入或输出的引脚。

HTTP（超文本传输协议）：互联网上应用最广泛的应用层协议，采用请求—响应模型。

I2C（集成电路间总线）：两线制（SDA/SCL）的低速串行总线，支持多设备。

中断（Interrupt）：外部或内部事件触发时暂停当前程序、转而执行处理函数的机制。

ISR（中断服务函数）：中断触发后执行的函数，应尽量短小。

MQTT（消息队列遥测传输）：基于发布/订阅模型的轻量物联网消息协议。

OTA（空中升级）：通过网络远程更新设备固件的技术。

PWM（脉宽调制）：通过调节高低电平的占空比来模拟平均电压的技术。

RTOS（实时操作系统）：能提供确定性响应和多任务调度的操作系统，如 FreeRTOS。

SPI（串行外设接口）：四线制、全双工、高速的串行总线。

SSID（服务集标识符）：无线网络的名称。

UART（通用异步收发器）：异步、全双工的串行通信协议，即"串口"。

占空比（Duty Cycle）：PWM 信号中高电平占一个周期的比例。

波特率（Baud Rate）：串行通信中每秒传输的码元数，收发双方必须一致。

上拉/下拉电阻：为引脚提供默认高/低电平的电阻，避免悬空状态。

堆（Heap）：动态分配内存的区域。

栈（Stack）：存放函数调用、局部变量的内存区域，任务栈溢出会导致崩溃。

附录E 参考资料与延伸阅读

E.1 官方文档与资源

- **乐鑫科技官网**（www.espressif.com）：芯片、模组、开发板的权威资料入口；
- **ESP-IDF 编程指南**（docs.espressif.com/projects/esp-idf）：官方开发框架的完整文档，是进阶的必读；
- **Arduino ESP32 核心**（github.com/espressif/arduino-esp32）：Arduino 框架下 ESP32 支持的源码与文档；
- **ESP32 技术参考手册（TRM）**：芯片寄存器级的技术细节，底层开发的权威依据；
- **乐鑫官方论坛**（esp32.com）：官方技术支持社区，很多疑难问题都能在这里找到答案。

E.2 推荐书籍

- 《ESP32-C3 物联网工程开发实战》（乐鑫官方团队）：了解新芯片与工程实践；
- 《FreeRTOS 内核实现与应用开发实战指南》：深入理解实时操作系统；
- 《C 语言程序设计》（经典教材）：巩固 C 语言基础，是嵌入式开发的根基；
- 《电子技术基础》（模拟/数字电路）：补齐硬件与电路知识。

E.3 在线学习平台与社区

- **GitHub**：海量 ESP32 开源项目、库和示例，是最好的学习素材库；
- **Arduino Project Hub**：大量社区项目与教程，激发创意；
- **B 站 / YouTube**：众多 UP 主的 ESP32 视频教程，直观易懂；
- **知乎 / CSDN / 博客园**：中文技术文章与踩坑经验分享；
- **立创开源硬件平台 / 嘉立创 EDA**：开源硬件设计与 PCB 学习资源。

E.4 常用第三方库速查

库名	用途
PubSubClient	MQTT 客户端
ArduinoJson	JSON 解析与生成
DHT sensor library	DHT 系列温湿度传感器
Adafruit BMP280	BMP280 气压传感器

库名	用途
Adafruit MPU6050	MPU6050 六轴传感器
Adafruit SSD1306	SSD1306 OLED 屏
Ticker	定时任务
LittleFS	文件系统
ArduinoOTA	局域网 OTA 升级
WiFiClientSecure	TLS 加密通信

E.5 学习建议

- **边学边做**：每个知识点都动手验证，不要只看不练；
- **循序渐进**：按本书顺序学习，基础扎实后再跳跃；
- **善用搜索**：遇到问题先查官方文档，再搜索社区，最后提问；
- **阅读源码**：深入一个你常用的库，读懂它的实现，能力会跃升；
- **做项目驱动**：给自己定一个小项目目标，在完成中学习最有效。

附录F 实验器材与工具清单

为方便读者备齐实验器材，本附录汇总全书所需的硬件与工具清单，并给出选购建议。

F.1 核心硬件

器材	建议规格	数量	参考用途
ESP32 开发板	ESP32-WROOM-32 / 38 引脚	1～2	全书主控
面包板	830 孔	1～2	搭建电路
杜邦线	公对公、母对母、公对母	各若干	连接
USB 数据线	可传输数据	1	烧录供电

F.2 基础电子元件

元件	建议规格	数量
LED	红/绿/蓝/白	各 5
电阻	220 Ω、1 kΩ、4.7 kΩ、10 kΩ	各 10
轻触按键	四脚	5
电位器	10 kΩ	2
有源/无源蜂鸣器	5 V	各 1

F.3 传感器与模块

模块	型号	用途
温湿度传感器	DHT11 / DHT22	环境监测
气压传感器	BMP280	气压/海拔
六轴传感器	MPU6050	运动检测
超声波模块	HC-SR04	测距/避障
OLED 显示屏	SSD1306（0.96 寸）	显示
继电器模块	1～4 路	控制强电
Micro SD 卡模块	SPI 接口	存储
电机驱动	L298N / TB6612	驱动电机

模块	型号	用途
直流减速电机	带轮	2～4

F.4 工具与仪器

工具	用途
万用表	测电压、电流、通断，排查电路
USB 电流表	测功耗，低功耗实验必备
电烙铁 + 焊锡	焊接（进阶）
剥线钳、镊子	加工导线、夹取元件
逻辑分析仪（可选）	分析时序，底层开发利器

> 【提示】 选购建议：入门阶段不必一次买齐，先备齐"核心硬件 + 基础元件"即可完成前三部分实验；传感器和模块可按需逐步添置。多数器材在电商平台以"ESP32 入门套件"形式打包出售，性价比高。

附录G 30 天学习路线图

为了让读者更有节奏地学习，本附录提供一份**30 天学习路线图**，每天约投入 1～2 小时，适合有日常工作/学习的读者利用业余时间完成。

第 1 周：打好地基（入门篇）

天数	内容	目标
第 1 天	第 1 章 认识 ESP32	了解芯片家族与开发板
第 2 天	第 2 章 开发环境搭建	装好 IDE 与驱动，跑通串口测试
第 3 天	第 3 章 点亮第一盏灯	完成 LED 点灯与闪烁
第 4 天	第 4 章 电子基础	掌握电路基础与引脚资源
第 5 天	复习 + 流水灯实验	巩固入门篇，独立完成流水灯
第 6 天	第 5 章 GPIO 输入输出	按键控制 LED，理解消抖
第 7 天	第 6 章 PWM	完成呼吸灯实验

第 2 周：夯实外设（基础篇）

天数	内容	目标
第 8 天	第 7 章 ADC	电位器调光
第 9 天	第 8 章 定时器与中断	非阻塞闪烁 + 中断计数
第 10 天	第 9 章 UART	串口指令控制
第 11 天	第 10 章 I2C	地址扫描 + 驱动传感器
第 12 天	第 11 章 SPI	SD 卡读写
第 13 天	复习基础篇	做一个多传感器小综合
第 14 天	第 12 章 传感器实战	温湿度 + 测距

第 3 周：迈向物联网（进阶篇）

天数	内容	目标
第 15 天	第 13 章 Wi-Fi	成功联网并获取 IP
第 16 天	第 14 章 HTTP	GET/POST + JSON 解析
第 17 天	第 15 章 MQTT	发布订阅，远程控制

天数	内容	目标
第 18 天	第 16 章 BLE	手机连接控制
第 19 天	第 17 章 FreeRTOS	多任务编程
第 20 天	第 18～19 章 文件系统与低功耗	配置存储 + 深度睡眠
第 21 天	复习进阶篇	整理联网技术栈

第 4 周：进阶与实战（精通篇 + 实战篇）

天数	内容	目标
第 22 天	第 20 章 OTA	无线升级
第 23 天	第 21 章 ESP-IDF	搭建 IDF 环境，跑 blink
第 24 天	第 22～23 章 内存与驱动	理解内存，手写驱动
第 25 天	第 24～25 章 性能与安全	掌握优化与安全
第 26 天	第 26 章 智能家居	完成第一个项目
第 27 天	第 27 章 环境监测站	多任务数据采集
第 28 天	第 28～29 章 门锁/小车	选择其一完成
第 29 天	第 30 章 云平台	打通端到端
第 30 天	综合复盘	总结知识体系，规划下一步

> 【提示】 提示：这份路线图是参考节奏，实际学习时请根据自己的基础和进度灵活调整。基础好的读者可以压缩前两周；时间紧张的读者可以把第 4 周的实战项目分散到后续完成。**最重要的是坚持动手**，每天哪怕只做一个小实验，累积起来就是巨大的进步。

附录H 开发板选型与购买指南

面对市面上琳琅满目的 ESP32 开发板，新手常常不知如何选择。本附录梳理选型要点，帮助你买到适合自己的开发板。

H.1 主流开发板对比

开发板	主芯片	Flash	特点	适合人群
ESP32-DevKitC	经典 ESP32	4 MB	官方标准板，资料最全	新手首选
NodeMCU-32S（38 引脚）	经典 ESP32	4 MB	兼容 NodeMCU 布局，价格低	新手、面包板
ESP32-WROVER-KIT	经典 ESP32 + PSRAM	4/8 MB	带 PSRAM、带屏幕	需要大内存的进阶用户
ESP32-S3-DevKitC-1	ESP32-S3	8 MB	双核 + AI 指令 + USB	语音、图像、进阶
ESP32-C3-DevKitM-1	ESP32-C3（RISC-V）	4 MB	单核、低功耗、便宜	低功耗、量产

H.2 选型建议

- **纯新手入门**：选一块 38 引脚、芯片为经典 ESP32 的开发板（如 NodeMCU-32S 或 ESP32-DevKitC），价格低廉、教程最多、兼容性最好。本书所有实验均以这类开发板为基准。
- **需要大内存**（图片、音频、大数组）：选带 PSRAM 的模组（如 WROVER 系列）。
- **需要 USB 原生接口**（如做 HID 键盘、U 盘）：选 ESP32-S2/S3。
- **极致低功耗、电池供电**：选 ESP32-C3/C6 等 RISC-V 或低功耗型号。
- **做 AI、语音、图像**：选 ESP32-S3，它增加了面向神经网络的向量指令。

H.3 购买注意事项

- **确认芯片型号**：商品标题里的"ESP32"可能泛指整个家族，务必看清具体芯片（经典 ESP32 / S2 / S3 / C3 等），它们引脚和特性不同；
- **确认引脚数量**：常见有 30 引脚和 38 引脚两种布局，本书以 38 引脚为例，购买时注意；

- **确认 USB 转串口芯片**：CP2102 和 CH340 都需要装驱动，性能差异不大，关键是买到带数据线的套餐；
- **优先买带排针的版本**：已焊接排针的开发板可直接插面包板，省去焊接麻烦；
- **避开"缩水板"**：极低价开发板可能省略稳压、保护电路，供电不稳会导致 Wi-Fi 频繁复位，建议选口碑好的店铺和品牌。

附录I 全书知识点自测题

本附录提供覆盖全书五大部分的复习自测题，帮助读者检验学习效果。建议在学习完对应部分后作答，参考答案可结合正文自行核对。

I.1 入门篇自测

1. 什么是片上系统（SoC）？ESP32 属于哪一类器件?
2. 经典款 ESP32 的处理器架构、主频和内置 SRAM 分别是多少?
3. 列举 ESP32 家族中至少三款衍生芯片，并说明它们与经典款的一个主要区别。
4. 开发板上的 BOOT 键和 EN（复位）键分别有什么作用?
5. Arduino、ESP-IDF、MicroPython 三大框架各有什么特点？本书的推荐学习路线是什么?
6. 安装 ESP32 开发板支持包时，需要在 Arduino IDE 首选项中填写什么地址?
7. 为什么串口输出会出现乱码？如何解决?
8. 点亮 LED 时为什么必须串联限流电阻？请用欧姆定律说明。
9. ESP32 的 GPIO 引脚最高能承受多大的输入电压？超过会怎样?
10. 下列引脚中，哪些不能作为普通输出使用：GPIO2、GPIO3、GPIO5、GPIO12、GPIO34？为什么?

I.2 基础篇自测

1. 上拉电阻和下拉电阻的作用是什么？什么是"悬空"?
2. 什么是按键抖动？软件消抖的基本原理是什么?
3. 简述 PWM 的原理，以及占空比与等效平均电压的关系。
4. ESP32 使用哪三个函数（ledc 系列）产生 PWM？各自作用是什么?
5. 12 位 ADC 的返回值范围是多少？为什么?
6. 为什么在 Wi-Fi 开启时建议优先使用 ADC1（GPIO32～39）?
7. 什么是中断？中断服务函数中应注意哪些事项?
8. `IRAM_ATTR` 和 `volatile` 分别有什么作用?
9. `vTaskDelay()` 与 `delay()` 的本质区别是什么?
10. UART 接线时，TX 和 RX 应该如何连接？波特率不一致会导致什么?
11. I2C 总线的 SDA 和 SCL 为什么需要上拉电阻?
12. SPI 与 I2C 相比，在速度、线数、全双工方面有何差异？各适合什么场景?

I.3 进阶篇自测

1. DHT22 的采样间隔为什么不能太短？读取失败（NaN）时如何处理?
2. HC-SR04 超声波测距的原理是什么？如何由回波时间计算距离?
3. ESP32 的 Wi-Fi 支持哪三种工作模式？各有什么用途?
4. 为什么真实的联网设备必须实现断线自动重连?
5. HTTP 的 GET 和 POST 方法有什么区别？各适合什么场景?
6. 什么是 JSON？ArduinoJson 库的作用是什么?
7. MQTT 的发布/订阅模型中，Broker、Topic、QoS 分别指什么?
8. 使用 PubSubClient 时，为什么 `client.loop()` 必须常驻主循环?
9. BLE 的 GATT 中，Service 和 Characteristic 是什么关系?
10. FreeRTOS 中，任务、队列、互斥锁分别解决什么问题?
11. 深度睡眠唤醒后，程序从哪里开始执行？RAM 中的数据会怎样?
12. LittleFS 与 SPIFFS 相比有哪些优势?

I.4 精通篇自测

1. OTA 升级的原理是什么？"双分区"机制有什么好处?
2. ESP-IDF 相比 Arduino 框架有哪些优势？它的工程入口函数是什么?
3. ESP32 的 SRAM、IRAM、DRAM、PSRAM、Flash 各有什么用途?
4. 什么是内存泄漏？如何用代码监测?
5. 什么是堆碎片？如何减少它的发生?
6. 什么是栈溢出？如何排查和修复?
7. 驱动开发的本质是什么？为什么"先读 ID 寄存器"是验证通信的第一步?
8. 编写健壮的设备驱动，应遵循哪些工程方法（至少列出四点）?
9. 实时性的含义是什么？影响实时性的主要因素有哪些?
10. ESP32 提供哪些安全能力来防止固件被读取和篡改?
11. TLS/HTTPS 提供哪三重安全保障?
12. 为什么生产环境不能使用 `setInsecure()` 跳过证书校验?

I.5 实战篇自测

1. 智能家居控制中心的软件通常分为哪几层？各层职责是什么?
2. 环境监测站的"采集—显示—上报—重传"多任务架构中，互斥锁和队列分别起什么作用?

3. 智能门锁的核心控制逻辑为什么适合用状态机表达？请画出它的状态图。
4. 智能小车中，为什么电机必须独立供电？"共地"的意义是什么？
5. 一个完整的物联网平台通常包含哪四层架构？各层之间用什么协议通信？
6. 设计一套规范的 MQTT 主题命名规范，并说明其优点。
7. 从"原型"到"产品"，需要补齐哪些方面的能力？

结语：从入门到精通的旅程

亲爱的读者，当你翻到这一页，意味着你已经走完了从"ESP32 新手"到"物联网开发者"的完整旅程。让我们花一点时间，回顾这段旅程，也展望前方的路。

回想第一章，我们第一次认识 ESP32 时，它还只是一颗陌生的芯片；第二章里，我们为安装开发环境而手忙脚乱；第三章点亮那盏 LED 时，你或许还不太明白高低电平背后的原理。而如今，你已经能够独立搭建智能家居、设计环境监测站、驱动电机小车，甚至规划一个完整的云端物联网平台。这其中的每一步，都是成长的印记。

在这段旅程中，我们不仅学会了具体的技术——GPIO、PWM、ADC、定时器、中断、I2C、SPI、UART、Wi-Fi、蓝牙、MQTT、FreeRTOS——更重要的是，我们建立了一套**看待嵌入式系统的思维方式**：

第一，硬件与软件是一体两面。 嵌入式开发最独特的地方，在于你必须同时理解电路和代码。一个引脚输出高电平，背后是电压、电流和欧姆定律；一段看似简单的代码，背后是寄存器和时序。只有软硬兼修，才能真正驾驭这颗芯片。

第二，从"能跑"到"跑得好"需要工程化。 入门时，我们满足于"程序能跑起来"；但精通者追求的是"程序稳定、可靠、可维护地跑下去"。断线重连、内存管理、低功耗、OTA 升级、安全加密——这些看似"额外"的工作，才是区分玩具与产品的分水岭。

第三，文档与源码是最好的老师。 本书能带你入门，但技术的世界日新月异。当你遇到书里没有的新芯片、新传感器、新协议时，请记住：乐鑫官方的文档、Arduino 社区的库、开源项目的源码，都是你最可靠的老师。学会查文档、读源码，你就拥有了自我成长的能力。

第四，动手实践是唯一捷径。 嵌入式是一门"手艺"，再多的阅读也替代不了亲手接线、烧录、调试。如果你只是读完了这本书却没有动手，那么你只得到了知识的一半；如果你把每一章的实验都亲手做过一遍，那么你得到的将是真正属于自己的能力。

关于未来，嵌入式与物联网的世界依然广阔。你可以继续深入的方向有很多：

- **向底层深耕**：深入研究 ESP-IDF、RTOS 内核、驱动程序、电源管理，成为嵌入式系统专家；
- **向应用拓展**：结合机器学习（TinyML）、边缘计算、计算机视觉，让设备更"智能"；
- **向产品迈进**：把原型打磨成量产产品，补齐认证、可靠性、供应链等环节；

· **向生态融合**：接入 Matter、HomeKit、鸿蒙等智能家居生态，参与更大的平台。

无论你选择哪条路，本书所建立的基础——扎实的外设知识、清晰的工程思维、敏锐的排错能力——都将是你最坚实的起点。

最后，感谢你读完这本书。技术书籍的写作是一场马拉松，而技术的探索更是一场没有终点的旅程。愿你保持好奇、保持热情，在嵌入式与物联网的世界里，点亮属于自己的那盏灯。

如果你在阅读或实践中遇到问题，欢迎通过社区与我交流；如果你用 ESP32 做出了有趣的作品，也期待听到你的分享。祝你在开发之路上越走越远，越走越稳。

—— **作者，于成书之际**